Advanced Data Structures

CO4

1. **Digital Search Tree**
2. **Binary Trie**
3. **Compressed Trie (Radix Trie)**
4. **Suffix Trie**
5. **Multi-Way Trie**
6. **Suffix Tree**

1 Digital Search Tree:

A Digital Search Tree (DST) is a type of tree-based data structure used to store and search keys based on the digits (bits or characters) of the key values rather than comparing the entire key at once. It is particularly useful for binary keys, strings, and integer keys where each level of the tree represents a digit position in the key. DST is closely related to the concept of a Trie, but it stores complete keys in nodes and uses digit-by-digit decisions for traversal.

In a Digital Search Tree, each level of the tree corresponds to a specific digit position of the key (for example, the first bit, second bit, etc.). When inserting or searching for a key, the algorithm checks the digit at that position and moves either left (0) or right (1) depending on the value of the digit. This structure allows efficient searching for keys with similar prefixes.

1. Structure of Digital Search Tree

- **Each node stores:**
  - **A key value**
  - **A left child pointer**
  - **A right child pointer**
- **Traversal decision is made using individual digits of the key.**
- **For binary representation:**
  - **0 → left child**
  - **1 → right child**

Example :

Create Digital Search Tree by Inserting the following keys: 5, 7, 3, 4

Solution:

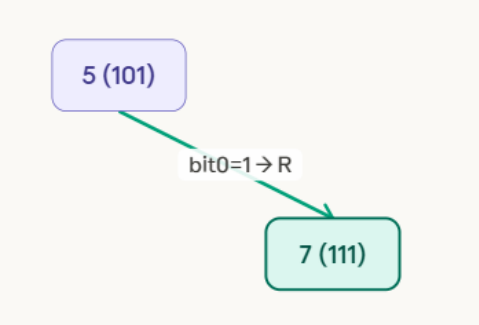
Binary representation:

Key	Binary
5	101
7	111
3	011
4	100

Step 1: Insert 5 (101)

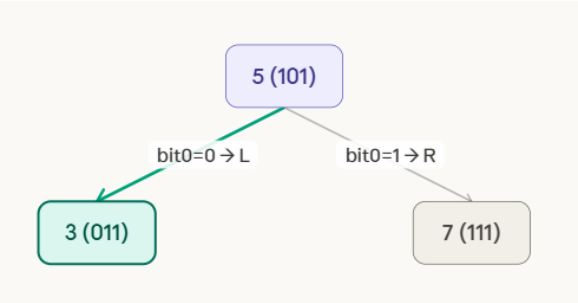
Step 2: Insert 7 (111)

Compare at root (5, depth 0 → check bit 0 = MSB). Bit 0 of 7 = 1 → go RIGHT. Right child of 5 is empty → place 7 there.



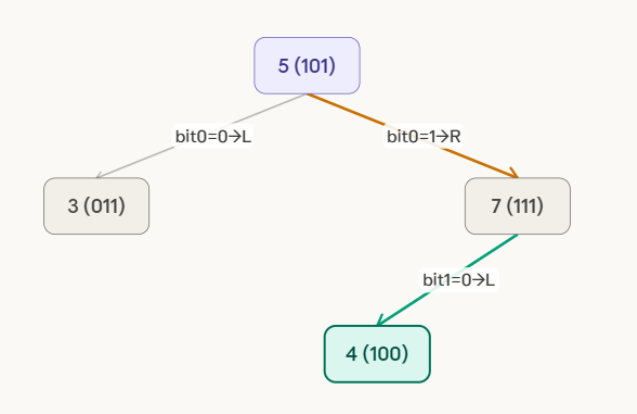
Step 3: Insert 3 (011)

Insert 3 (binary: 011). At root (5, depth 0 → bit 0). Bit 0 of 3 = 0 → go LEFT. Left child of 5 is empty → place 3 there.

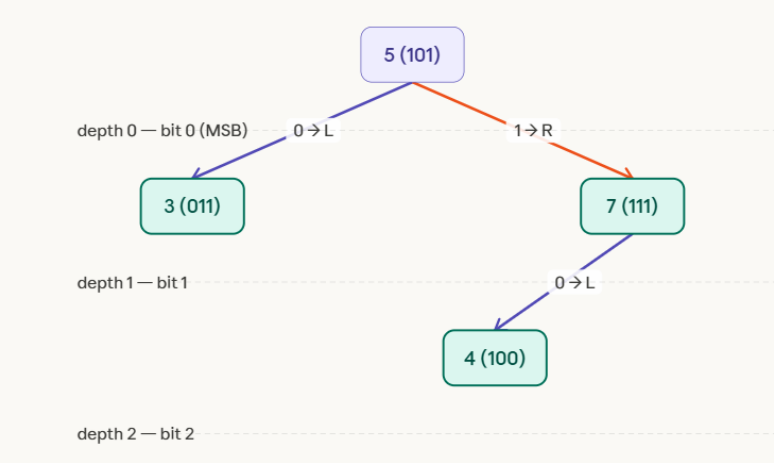


Step 4: Insert 4 (100)

At root (5, depth 0 → bit 0). Bit 0 of 4 = 1 → go RIGHT. Reach node 7 (depth 1 → bit 1). Bit 1 of 4 = 0 → go LEFT. Left child of 7 is empty → place 4 there.



FINAL DIGITAL SEARCH TREE. Root = 5. Left subtree: 3 (bit0=0). Right subtree: 7 (bit0=1), with 4 as left child of 7 (bit1=0). Each node stores its full key. Routing uses bit at current depth: bit0 at depth 0, bit1 at depth 1, bit2 at depth 2.



3. Algorithms

Algorithm 1: DST Insertion

Input: Key K
Output: Insert node in DST

```
DST_Insert(root, K)
1. Convert key K into binary representation
2. Set current = root
3. Set digit position i = 0
4. While current is not NULL
    if bit(K,i) == 0
        move to left child
    else
        move to right child
    i = i + 1
5. Insert new node at that position
6. Store key K
```

Time Complexity

- Average Case → O(log n)
- Worst Case → O(n)

Algorithm 2: Searching in DST

```
DST_Search(root, K)
1. Convert key K into binary
2. Set current = root
3. i = 0
4. while current ≠ NULL
    if current.key == K
        return FOUND
    if bit(K,i) == 0
        current = current.left
    else
        current = current.right
    i = i + 1
5. return NOT FOUND
```

Time Complexity

- Best Case → O(1)
- Average Case → O(log n)
- Worst Case → O(n)

Algorithm 3: Deletion in DST

DST_Delete(root, K)
1. Search node containing key K
2. If node is leaf
delete directly
3. If node has one child
replace node with its child
4. If node has two children
replace with inorder successor
delete successor node

4. Advantages of Digital Search Tree

1. **Faster searching for binary or string keys.**
2. **Efficient for prefix-based data.**
3. **Reduces number of key comparisons.**
4. **Useful in network routing and dictionary implementations.**

5. Disadvantages

1. **Tree can become skewed if keys share long prefixes.**
2. **Requires binary representation of keys.**
3. **May consume more memory than balanced trees.**

6. Applications of Digital Search Tree

1. **IP Routing tables**
2. **Dictionary word searching**
3. **Auto-complete systems**
4. **Symbol tables in compilers**

DST is conceptually related to structures used in prefix searching like Trie and hashing-based indexing systems.

7. Comparison with Binary Search Tree

Feature	Digital Search Tree	Binary Search Tree
Comparison method	Digit-by-digit	Whole key comparison
Search complexity	O(log n) average	O(log n) average
Suitable for	Binary keys, strings	General numeric keys
Structure	Based on digits	Based on ordering

8. Short Example (Search Operation)

Search key 4 (100) in the DST.

Steps:

1. **Start at root 5**
2. **First bit = 1 → go right**
3. **Second bit = 0 → go left**
4. **Node found = 4**

Search successful.

Digital Search Trees are efficient structures for storing keys when bitwise representation is important. By using digit-by-digit traversal, DST reduces comparison cost and enables fast searching for binary keys. This makes it suitable for applications like network routing, indexing systems, and dictionary lookups.

Problem 1

Create DST by inserting following keys 3, 5, 6, 9, 10, 12 and after inserting delete 6 and 10 and draw the final DST.

Solution:

First convert them into binary form (minimum 4 bits for clarity).

Key	Binary
3	0011
5	0101
6	0110
9	1001
10	1010
12	1100

In a Digital Search Tree, traversal depends on binary digits:

- **0 → Left child**
- **1 → Right child**

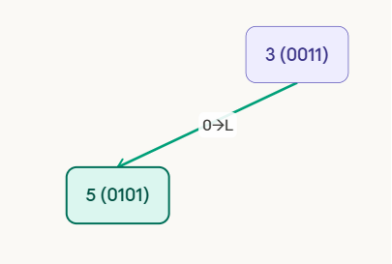
Step 1: Insert 3 (0011)

Step 2: Insert 5 (0101)

At root 3 (depth 0 → bit 0).

Bit 0 of 5 = 0 → go LEFT.

Left of 3 is empty → place 5 there.



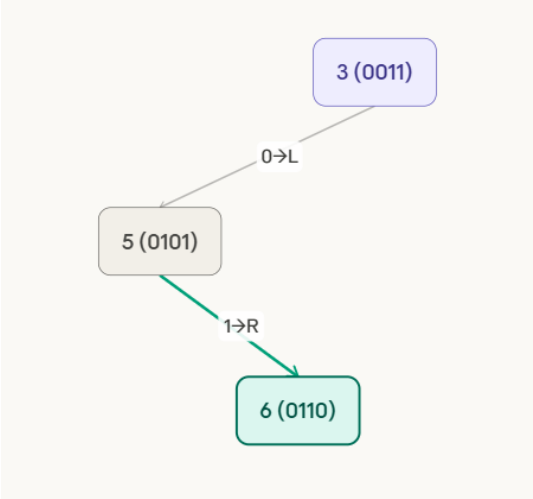
Step 3: Insert 6 (0110)

At root 3 (depth 0, bit 0). Bit 0 of 6 = 0 → go LEFT → reach 5.

At 5 (depth 1, bit 1).

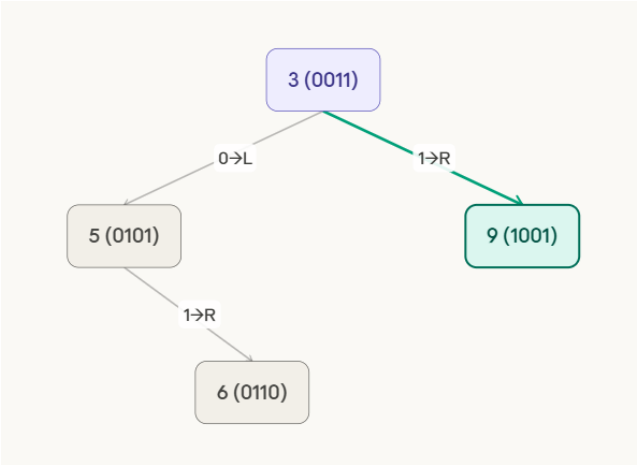
Bit 1 of 6 = 1 → go RIGHT.

Right of 5 is empty → place 6 there.



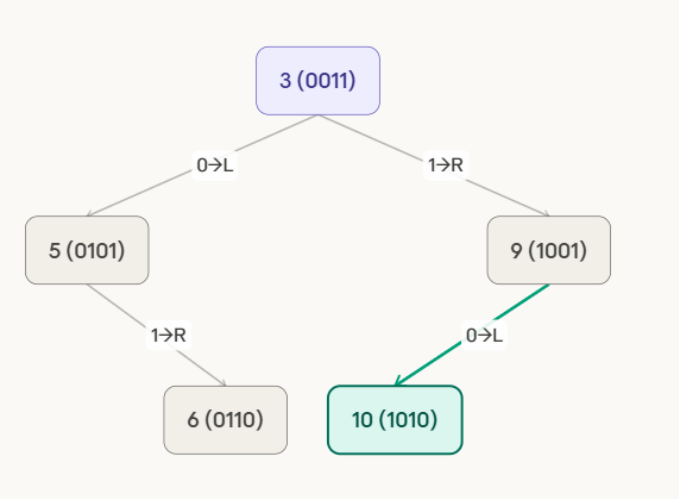
Step 4: Insert 9 (1001)

At root 3 (depth 0, bit 0). Bit 0 of 9 = 1 → go RIGHT. Right of 3 is empty → place 9 there.



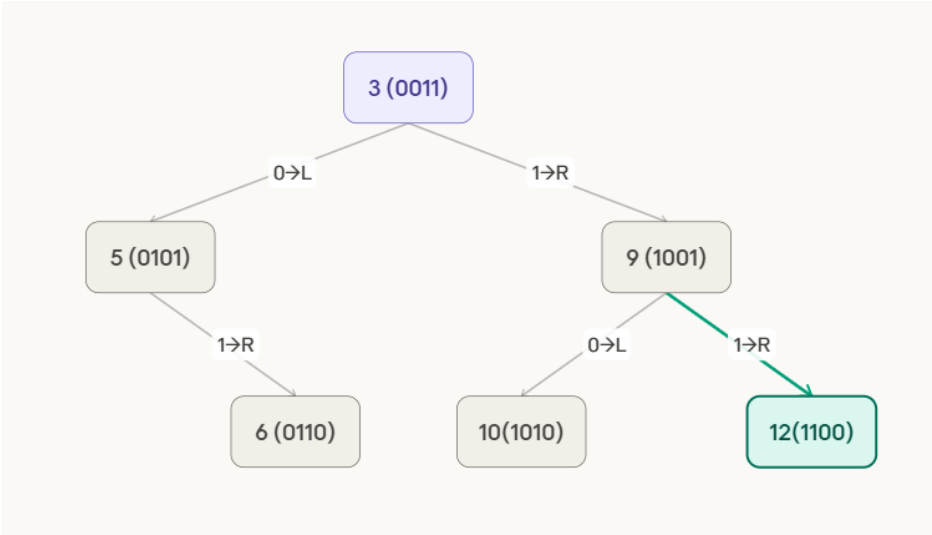
Step 5: Insert 10 (1010)

At root 3 (depth 0, bit 0). Bit 0 of 10 = 1 → RIGHT → reach 9. At 9 (depth 1, bit 1). Bit 1 of 10 = 0 → LEFT. Left of 9 is empty → place 10 there.



Step 6: Insert 12 (1100)

At root 3 (depth 0, bit 0). Bit 0 of 12 = 1 → RIGHT → reach 9. At 9 (depth 1, bit 1). Bit 1 of 12 = 1 → RIGHT. Right of 9 is empty → place 12 there.

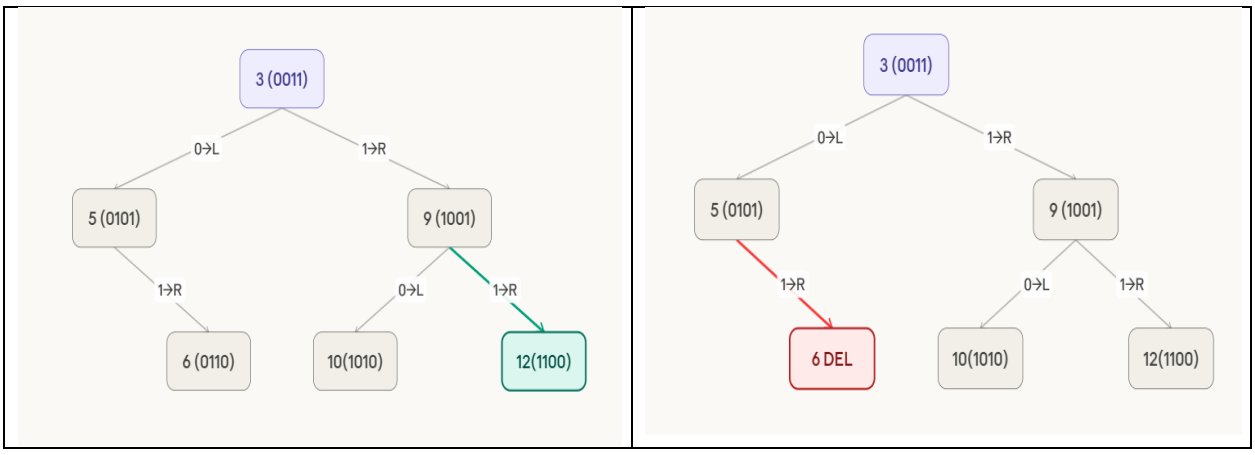


Deletion Operation

Now delete two keys: 6 and 10

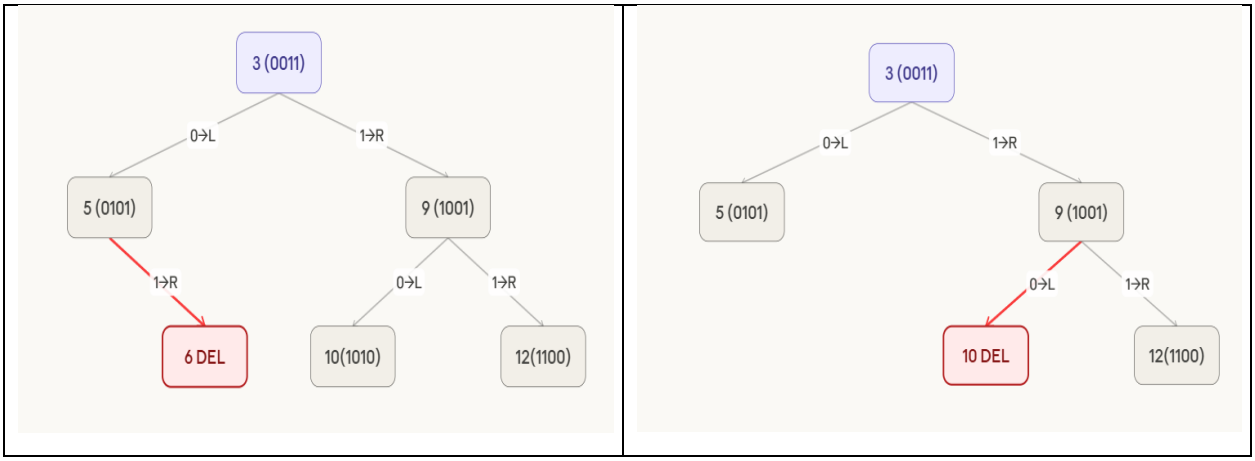
Delete 6

Locate 6 — path: root(3)→bit0=0→L→5→bit1=1→R→6. Node 6 is a LEAF (no children). Simply remove it. Parent 5 loses its right child. No replacement needed.



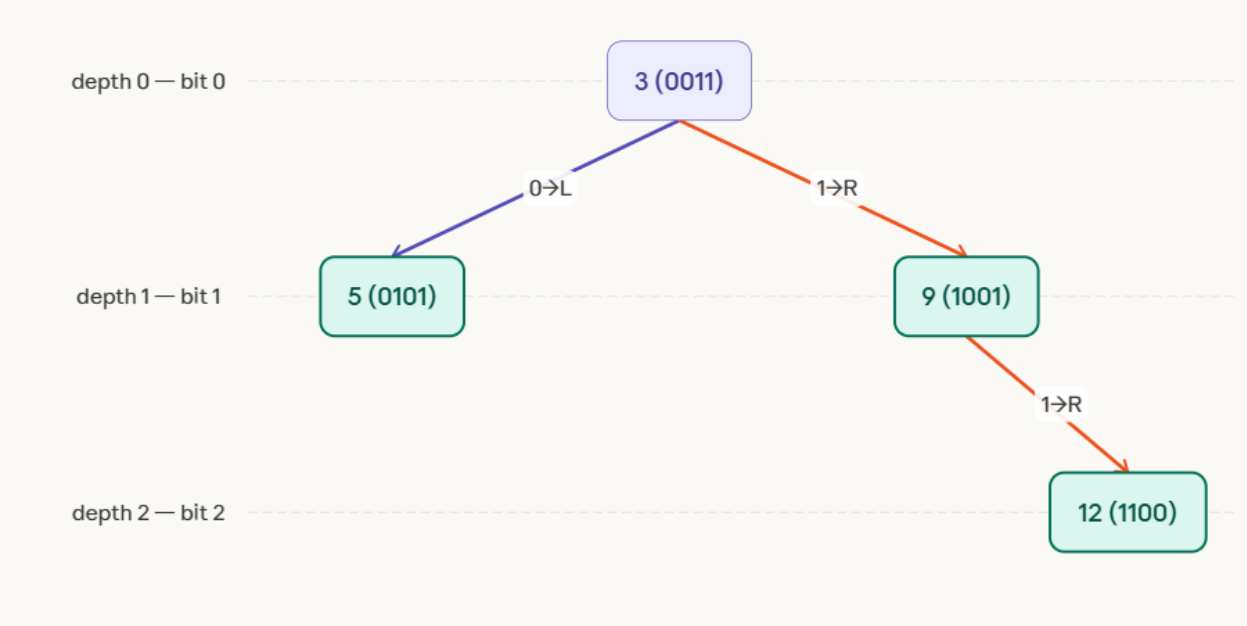
Delete 10

Locate 10 — path: root(3)→bit0=1→R→9→bit1=0→L→10. Node 10 is a LEAF (no children). Simply remove it. Parent 9 loses its left child. No replacement needed.



Final Tree after Deletions

After deleting 6 and 10. Remaining keys: 3 (root), 5 (left of 3), 9 (right of 3), 12 (right of 9). Both deletions were simple leaf removals — no restructuring required. Tree has 4 nodes across 3 levels.



Problem 2

Create Digital Search Tree by inserting following keys 3, 4, 5, 8, 10, 12 ,9,7and after inserting delete 8 and draw the final DST.

Solution

Digital Search Tree (DST) Construction – Step by Step

Given Keys:
Insert → 3, 4, 5, 8, 10, 12, 9, 7

First convert numbers into 4-bit binary (for consistent comparison).

Key	Binary
3	0011
4	0100
5	0101
7	0111
8	1000
9	1001
10	1010
12	1100

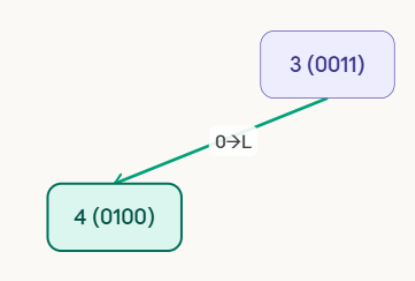
Rule of Digital Search Tree

- **0 → Left child**
- **1 → Right child**
- **Each level corresponds to the next bit position.**

Step 1: Insert 3 (0011)

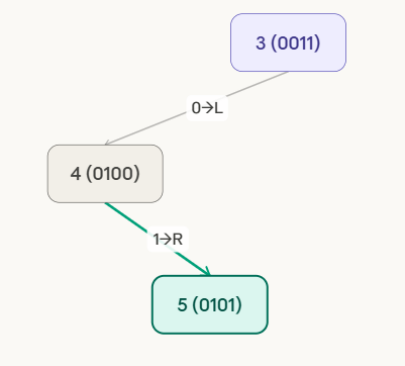
Step 2: Insert 4 (0100)

At root 3, depth 0 → bit0 of 4 = 0 → go LEFT. Left of 3 is empty → place 4 there.



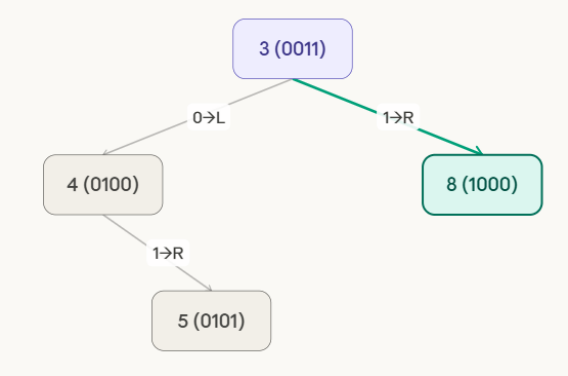
Step 3: Insert 5 (0101)

At root 3, depth 0 → bit0 of 5 = 0 → LEFT → reach 4. At 4, depth 1 → bit1 of 5 = 1 → RIGHT. Right of 4 is empty → place 5 there.



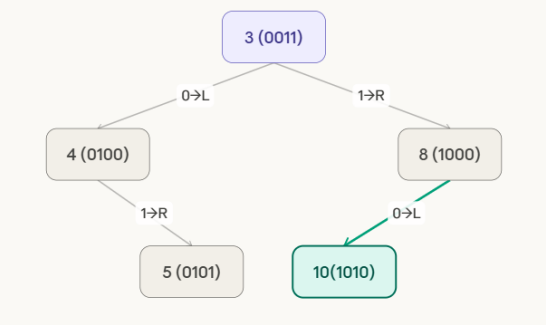
Step 4: Insert 8 (1000)

At root 3, depth 0 → bit0 of 8 = 1 → go RIGHT. Right of 3 is empty → place 8 there.



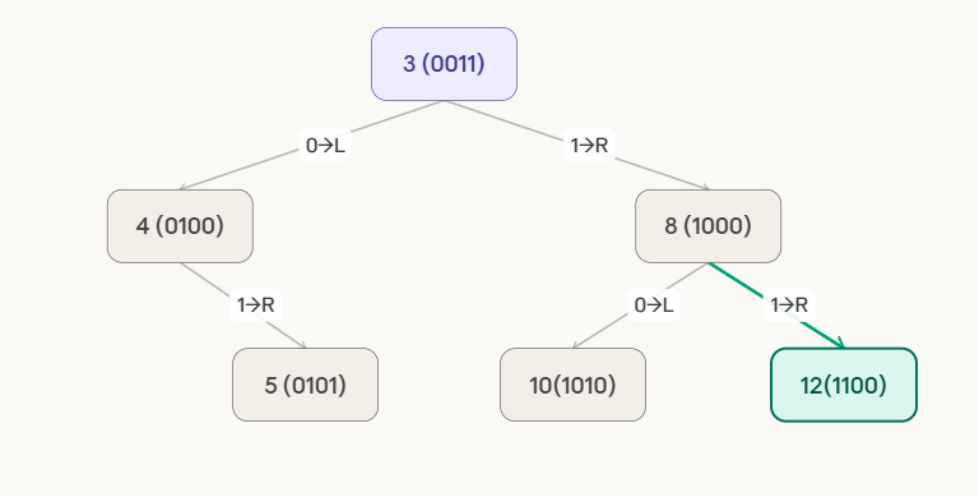
Step 5: Insert 10 (1010)

At root 3, depth 0 → bit0=1 → RIGHT → reach 8. At 8, depth 1 → bit1 of 10 = 0 → LEFT. Left of 8 is empty → place 10 there.



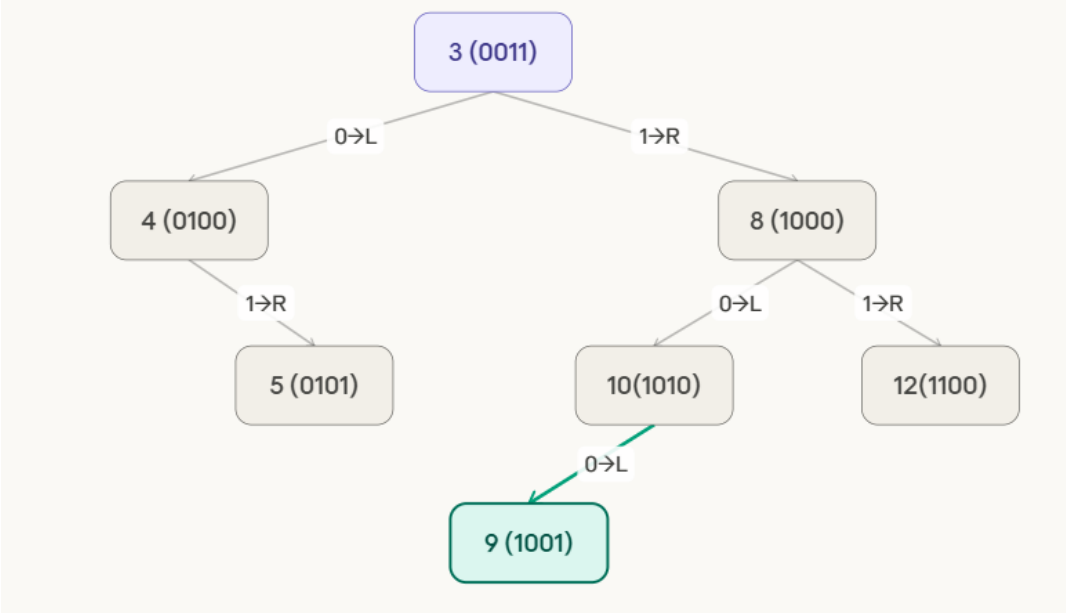
Step 6: Insert 12 (1100)

At root 3 → bit0=1 → RIGHT → 8. At 8, depth 1 → bit1 of 12 = 1 → RIGHT. Right of 8 is empty → place 12 there.



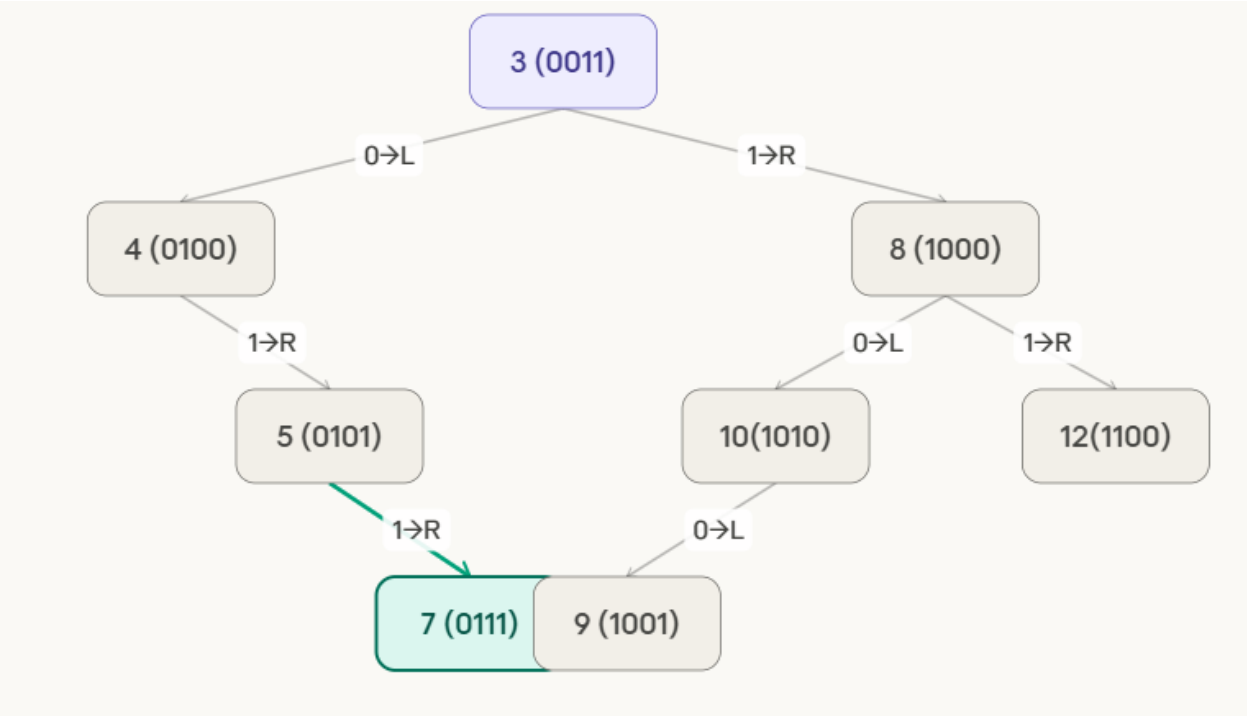
Step 7: Insert 9 (1001)

root→bit0=1→R→8. At 8, depth 1→bit1 of 9=0→L→10. At 10, depth 2→bit2 of 9=0→L. Left of 10 is empty → place 9 there.



Step 8: Insert 7 (0111)

root→bit0=0→L→4. At 4, depth 1→bit1 of 7=1→R→5. At 5, depth 2→bit2 of 7=1→R. Right of 5 is empty → place 7 there.



delete node 8.

Step 1: Identify the Node to Delete

Locate node 8. Path: root(3)→bit0=1→RIGHT→found 8.

Node 8 has TWO children: left child=10, right child=12.

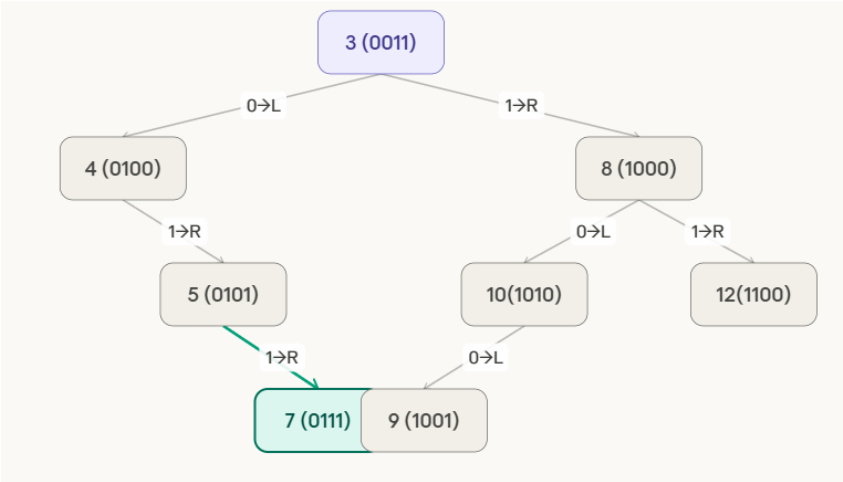
→ Case 3: Find in-order successor (smallest key in right subtree).

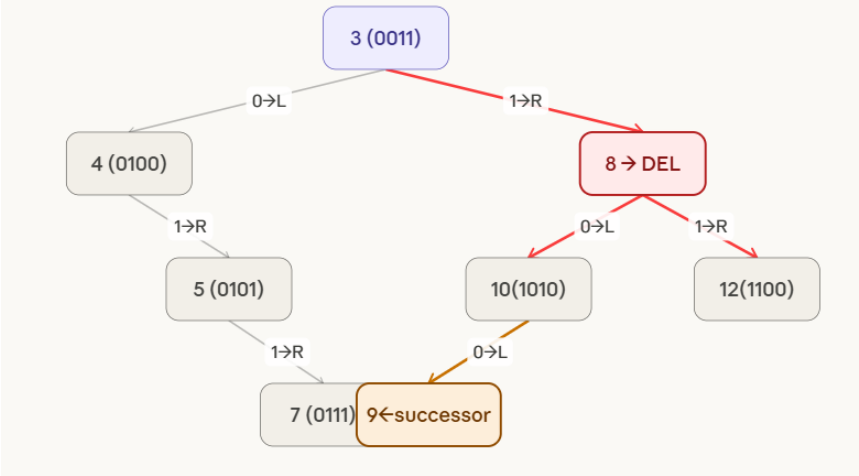
Right subtree root = 12 → 12 has no left child

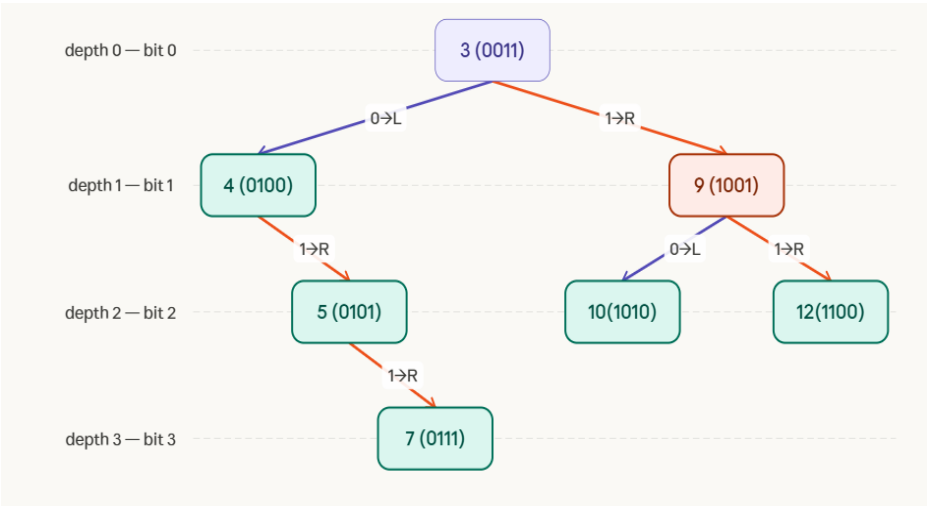
→ IN-ORDER SUCCESSOR = 9? No.

We use the standard DST deletion: replace 8 with its in-order successor from the right subtree. Smallest in right subtree of 8 = 9 (going 12→ no, 10→left→9).

Actually the in-order successor = 9 (leftmost of right subtree). Copy 9 into node 8. Delete old node 9.


3 (0011)
0→L
1→R
4 (0100)
8 (1000)
1→R
0→L
1→R
5 (0101)
10(1010)
12(1100)
1→R
0→L
7 (0111)
9 (1001)


3 (0011)
0→L
1→R
4 (0100)
8 → DEL
1→R
0→L
1→R
5 (0101)
10(1010)
12(1100)
1→R
0→L
7 (0111)
9←successor


depth 0 — bit 0
3 (0011)
0→L
1→R
depth 1 — bit 1
4 (0100)
9 (1001)
1→R
0→L
1→R
depth 2 — bit 2
5 (0101)
10(1010)
12(1100)
1→R
depth 3 — bit 3
7 (0111)

2. Binary Trie

A **Binary Trie** is a special type of **Trie data structure** used to store **binary numbers (0s and 1s)**. In this structure, each node represents a **bit (0 or 1)** of a binary number. Since binary digits are only **0 and 1**, each node in a Binary Trie can have **at most two children**.

Binary Tries are widely used in problems involving **bitwise operations**, especially **maximum XOR queries**, **IP routing**, and **network prefix matching**.

Structure of Binary Trie

- The tree starts from a **root node**.
- Each level represents a **bit position** in the binary number.
- Each node has **two possible children**:
    - **Left child → represents bit 0**
    - **Right child → represents bit 1**
- A complete path from **root to leaf** represents a **binary number**.

Operations in Binary Trie

1. Insertion

Insertion involves adding bits of the binary number one by one.

Algorithm

1. Start from the **root node**.
2. Convert the number into **binary form**.
3. For each bit in the binary number:
    - If bit = **0**, move to **left child**.
    - If bit = **1**, move to **right child**.
4. If the node does not exist, **create a new node**.
5. After inserting all bits, **mark the end of the number**.

Time Complexity:

$$O(L)$$

Where **L = number of bits in the number**.

2. Search

Search checks whether a binary number exists in the Trie.

Steps

1. Start at the **root node**.
2. For each bit in the number:
    - Move to **left child for 0**

- Move to **right child for 1**

3. If the path exists and the final node is marked as **end**, the number is found.

Time Complexity:

$$O(L)$$

3. Deletion

Deletion removes a number from the Binary Trie.

Steps

1. Traverse the tree following the bits.
2. Unmark the **end-of-number node**.
3. Remove nodes that are **not used by other numbers**.

5. Applications of Binary Trie

1. **Maximum XOR Pair Problems**
2. **IP Routing Tables**
3. **Network Prefix Matching**
4. **Bit Manipulation Problems**
5. **Competitive Programming Algorithms**

Binary Tries allow efficient computation of **maximum XOR values between numbers in an array**.

6. Advantages

- Efficient **bitwise operations**
- Fast **search and insertion**
- Useful for solving **XOR optimization problems**
- Time complexity depends only on **number of bits**, not number of elements.

7. Disadvantages

- Requires **extra memory** for nodes.
- Not efficient if numbers have **very large bit length**.

8. Comparison with Standard Trie

Feature	Standard Trie	Binary Trie

Data stored	Strings	Binary numbers
Children per node	Many (alphabet size)	2
Edge labels	Characters	Bits
Applications	Dictionary search	Bitwise algorithms

A **Binary Trie** is an efficient tree-based data structure designed to store **binary representations of numbers**. It supports fast **insertion, searching, and deletion operations** with time complexity **O(L)**, where **L is the number of bits**. Due to its efficiency in **bitwise operations**, Binary Trie is widely used in **network routing, cryptography, and XOR-based algorithmic problems**.

Problem:

Create binary trie of the following elements 2,4,6,8,10 and delete 4 and 6 and draw the final trie.

Solution:

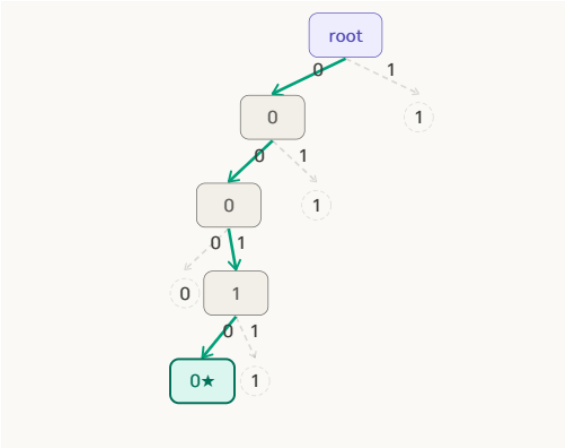
Elements: **2, 4, 6, 8, 10**
Operations: **Insert all elements → Delete 4 and 6**

For simplicity, convert numbers into **4-bit binary**.

Number	Binary
2	0010
4	0100
6	0110
8	1000
10	1010

Each level represents a **bit position** and each node has **two children (0 and 1)**.

Step 1: Insert 2 (0010)

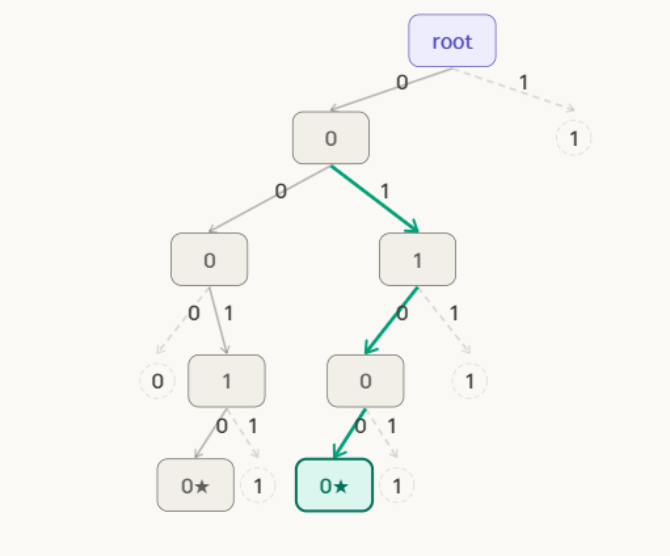


Step 2: Insert 4 (0100)

Binary: **0100**

Path:
0 → 1 → 0 → 0

First bit **0 already exists**, so we reuse it.

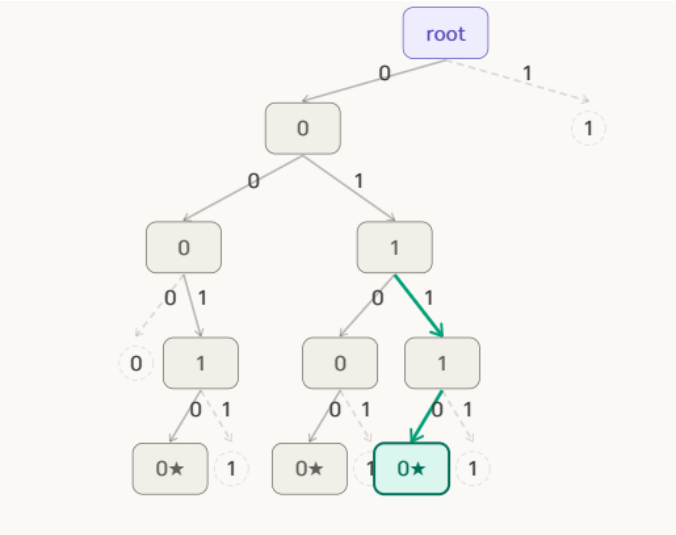


Step 3: Insert 6 (0110)

Binary: **0110**

Path:
0 → 1 → 1 → 0

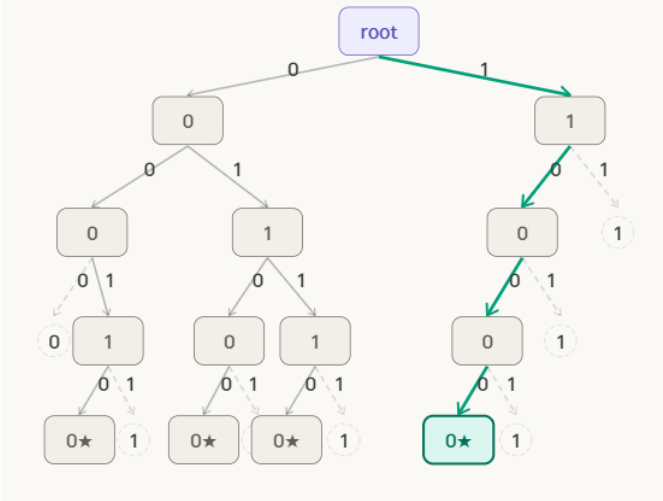
Prefix **0 → 1 already exists**.



Step 4: Insert 8 (1000)

Binary: **1000**

Path:
1 → 0 → 0 → 0

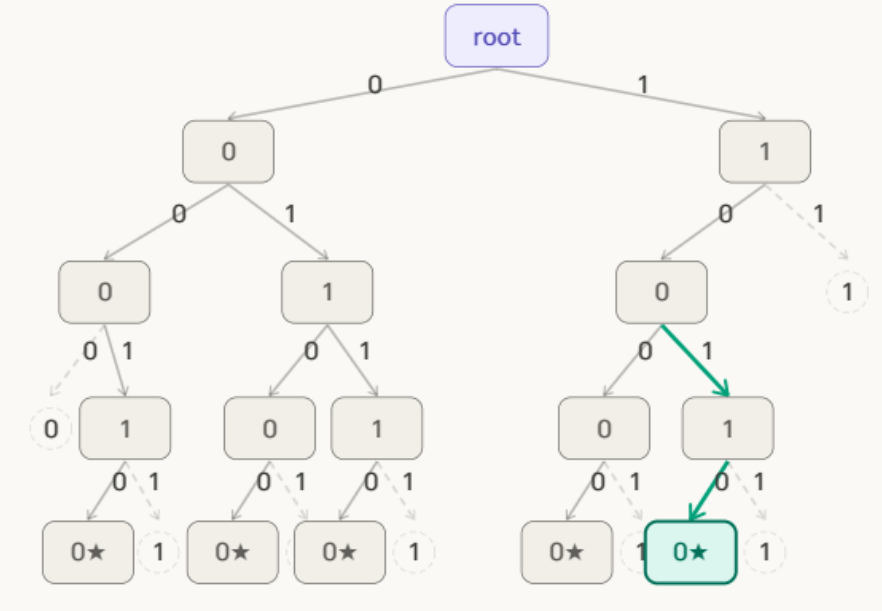


Step 5: Insert 10 (1010)

Binary: **1010**

Path:
1 → 0 → 1 → 0

Prefix **1 → 0 already exists**.

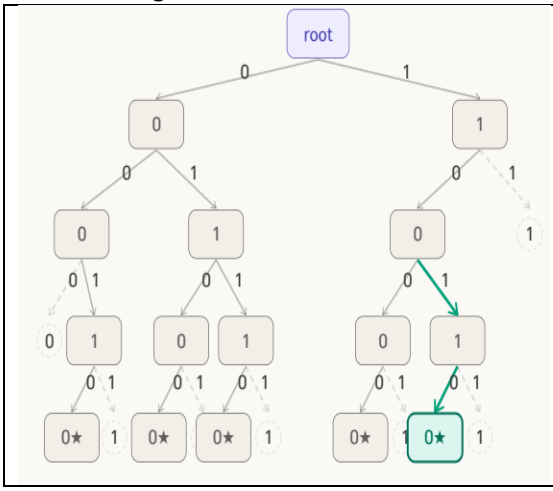


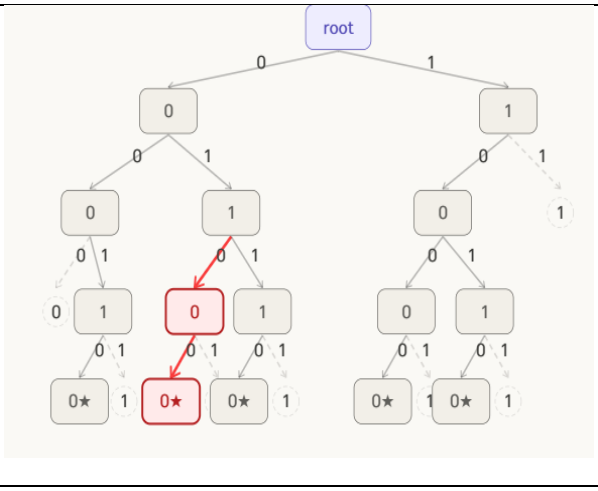
Step 6: Delete 4 (0100)

Binary: **0100**
Path:
0 → 1 → 0 → 0
Steps:

1. Traverse to **0100**.
2. Remove the **end marker**.
3. Check if node is used by another number.
4. Since no other number uses **0100**, delete that branch.

After deleting **4**:



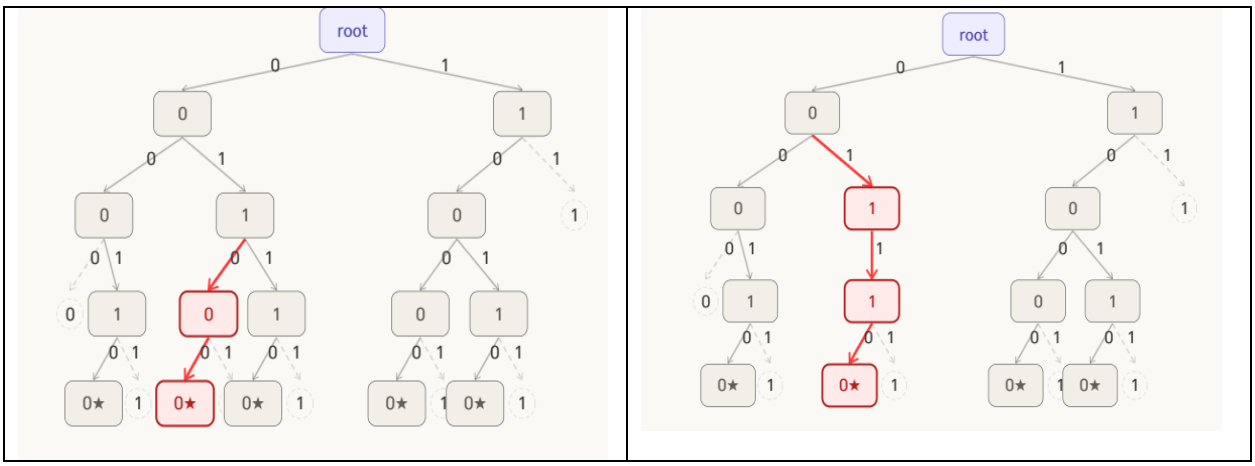


Step 7: Delete 6 (0110)

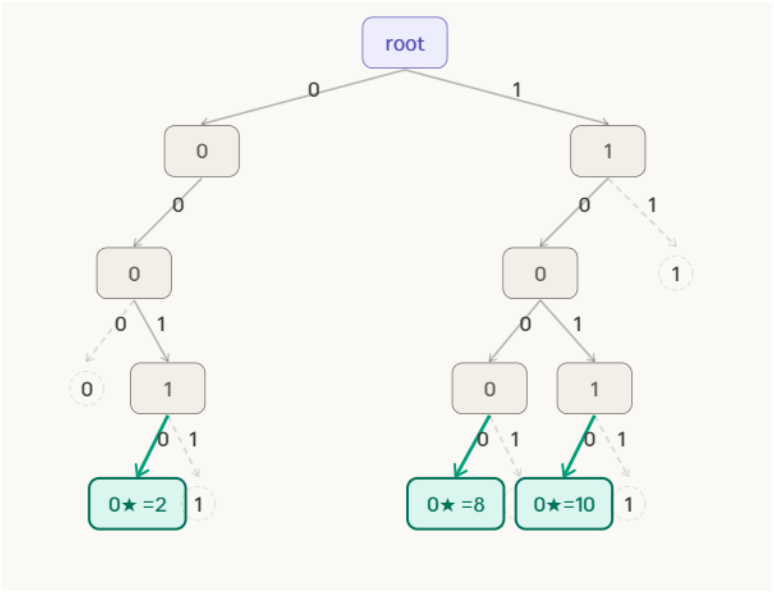
Binary: **0110**
Path:
0 → 1 → 1 → 0
Steps:

1. Traverse to **0110**.
2. Remove **end marker**.
3. Check if node has children.
4. Since no other number shares this path, remove nodes.

Final Trie:



FINAL BINARY TRIE — After deleting 4 and 6. Remaining elements: 2 (0010), 8 (1000), 10 (1010). The "1" branch under left "0" subtree is fully pruned. Tree is clean and valid. Depth = 4 (4-bit numbers).



2 Trie (Prefix Tree) or Standard Trie

A Trie, also called a Prefix Tree, is a tree-based data structure used to store a collection of strings (words) so that they can be searched efficiently. In a Trie, each node represents a character, and the path from the root node to a particular node forms a prefix or complete word. This structure is very useful for applications where many words share common prefixes, such as dictionary search, autocomplete systems, and spell checking. Instead of comparing full strings, the Trie processes one character at a time, which makes searching very fast.

Structure of Trie

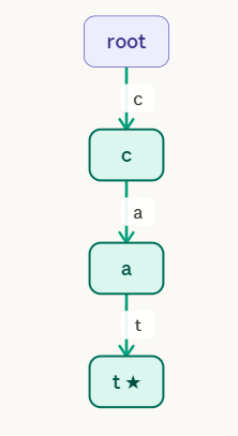
- **Root Node** – starting point of the tree (usually empty).
- **Edges** – represent characters.
- **Nodes** – represent prefixes of words.
- **End-of-word marker** – indicates that a word ends at that node.

Example: Create Trie or prefx Trie of the following strings { cat, car, dog }

Solution:

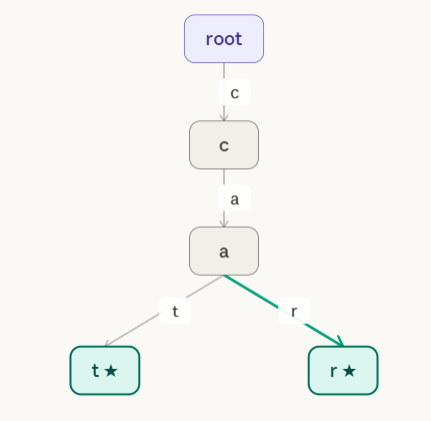
Step 1: Insert cat

Tree is empty. Create root → c → a → t. Mark "t" as end-of-word (★). Each character gets its own node — this is the key property of a character-level trie.



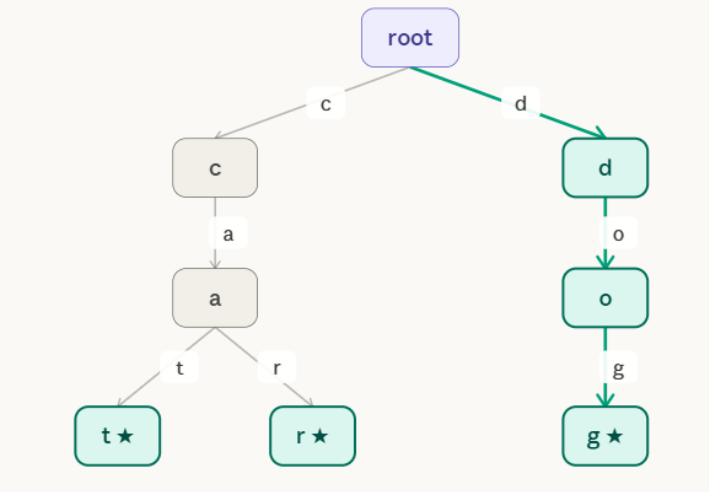
Step 2: Insert car

Follow existing path root→c→a (shared with "cat"). At node "a", "r" is a new edge. Create new leaf "r ★". The prefix "ca" is now shared by both "cat" and "car".



Step 3: Insert dog

At root, "d" is a completely new edge (no shared prefix with "cat" or "car"). Create root→d→o→g. Mark "g" as end-of-word (★). "dog" forms an independent branch.



Algorithms

1. Insert Algorithm

Insert(root, word)
1. current = root
2. for each character ch in word
3. if ch not in current.children
4. create new node
5. move to that node
6. mark end of word

Time Complexity:
O(L) where L = length of word

2. Search Algorithm

Search(root, word)
1. current = root
2. for each character ch in word
3. if ch not present
4. return FALSE
5. move to next node
6. if end-of-word marker found
7. return TRUE

3. Delete Algorithm (Basic Idea)

1. Search the word.
2. Remove the end-of-word marker.
3. Delete nodes that are not used by other words.

Advantages of Trie

Advantage	Explanation
Fast Searching	Searching depends only on word length
Prefix Matching	Efficient for prefix-based queries
Memory Sharing	Common prefixes are stored once
Autocomplete	Very useful in search engines

Problem : Construct a Trie (prefix trie)for the strings {pen, pencil, pending, penguin}. Explain the insertion process and show how deletion of pencil affects internal and leaf nodes.

Solution:

Trie Construction Example

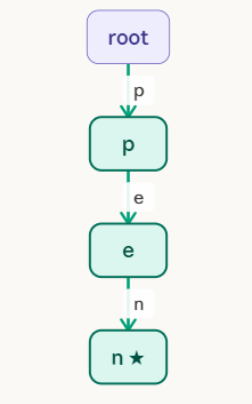
We need to construct a Trie for the strings:

{pen, pencil, pending, penguin}

A Trie stores characters in nodes, and each path from the root to a leaf represents a word.

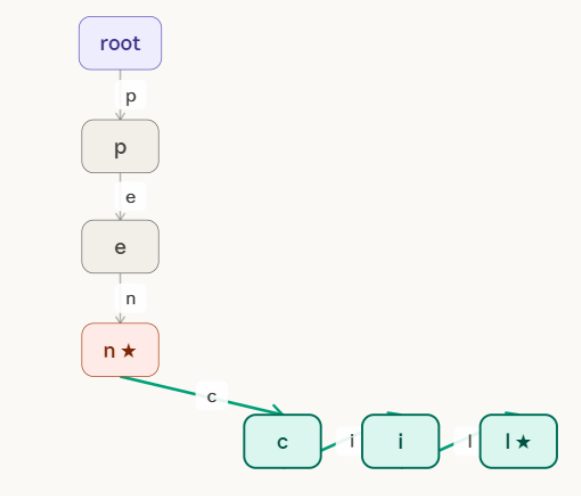
Step 1: Insert pen

Tree is empty. Create nodes p → e → n in order. Mark "n" as end-of-word (★). Three new nodes created.



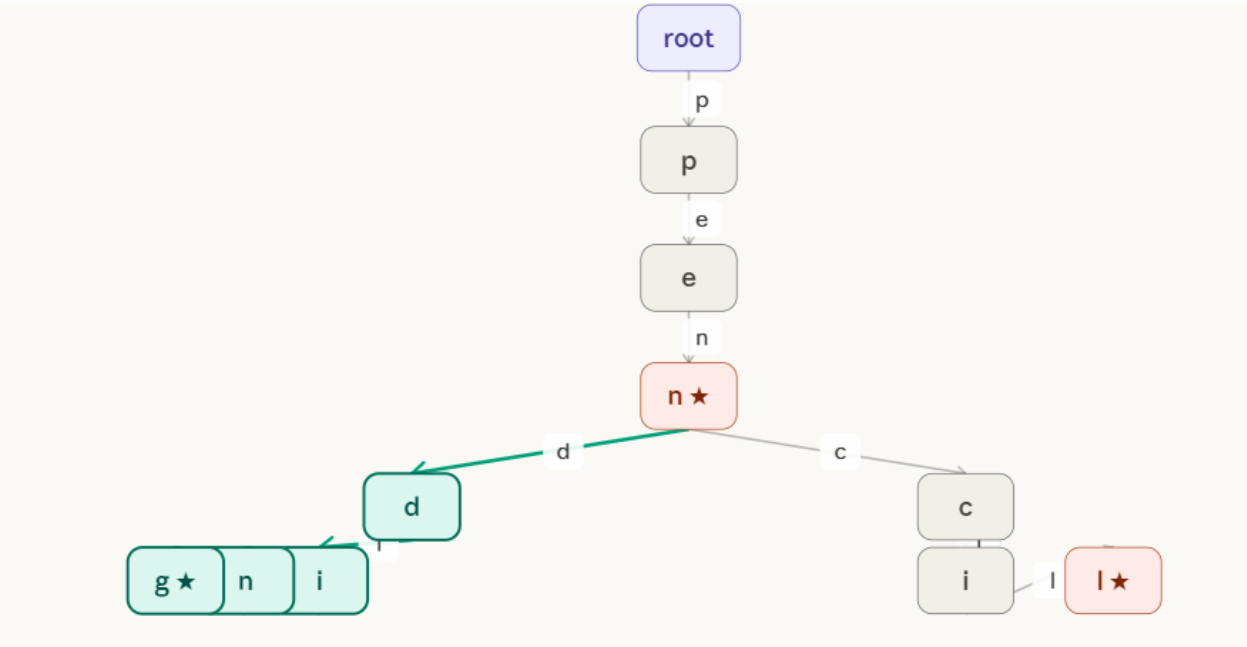
Step 2: Insert pencil

p → e → n already exist (shared with "pen"). At "n", "c" is new. Add c → i → l. Mark "l ★". The prefix "pen" is fully shared.



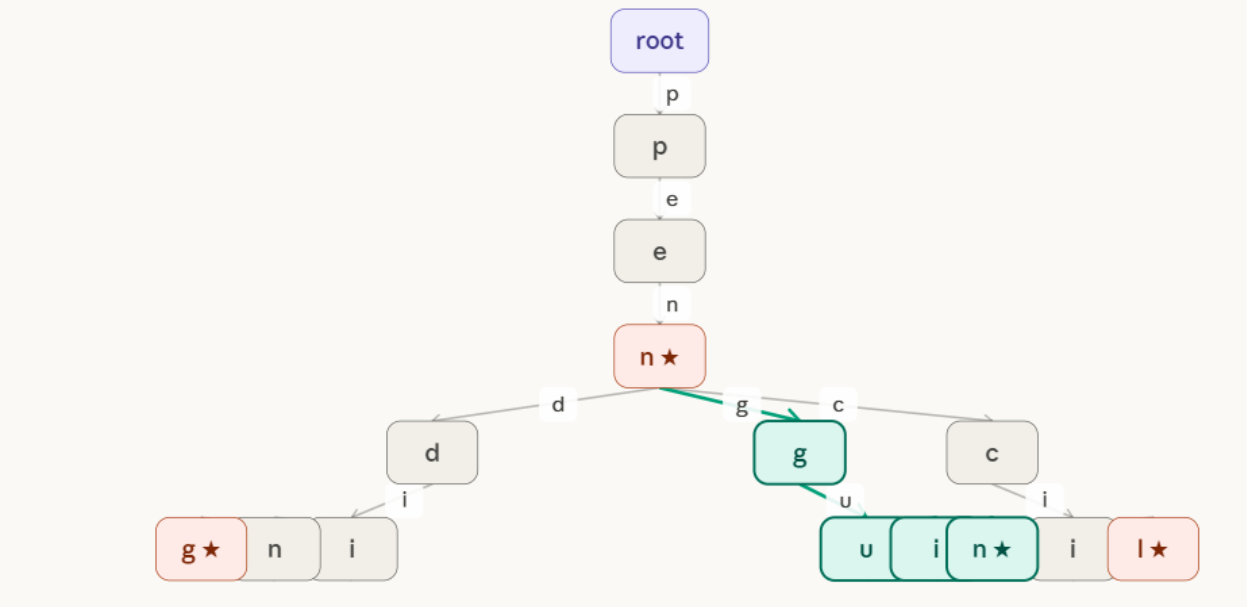
Step 3: Insert pending

p → e → n shared. At "n", "d" is new (sibling of "c"). Add d → i → n → g. Mark "g ★". Three words now share "pen".

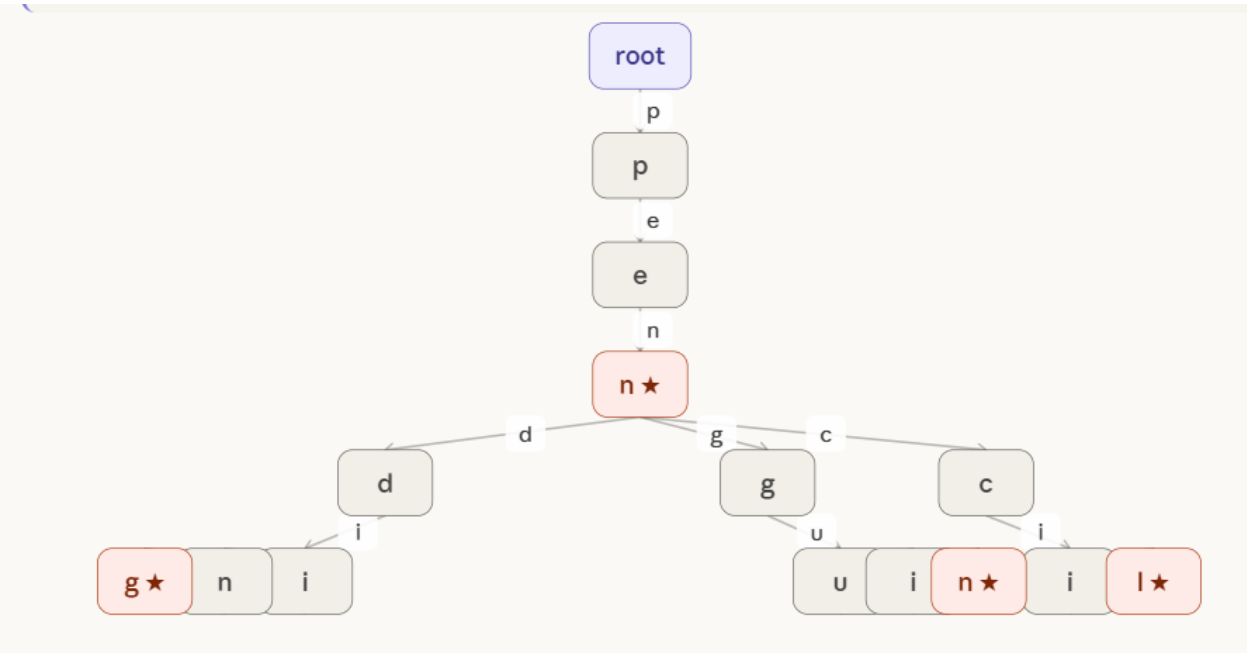


Step 4: Insert penguin

p → e → n shared. At "n", "g" is new (third sibling). Add g → u → i → n. Mark "n ★". Now "pen" branches into three children: c, d, g.

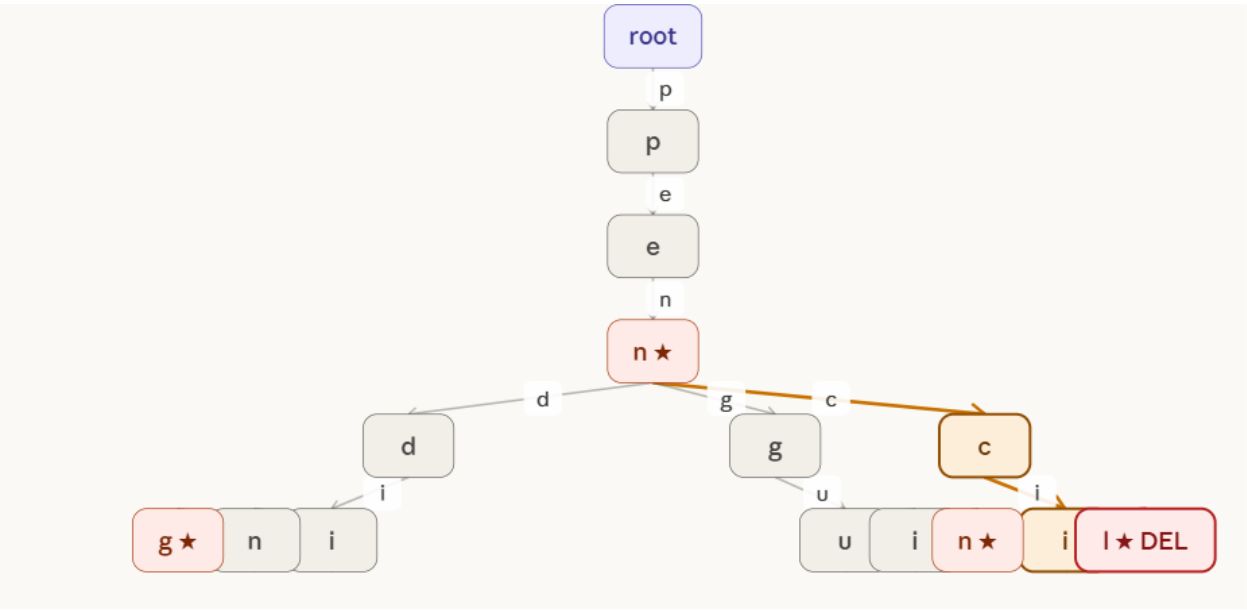


pen" is the shared trunk for all four words. At node "n ★", three branches diverge: c→pencil, d→pending, g→penguin. Shared prefix "pen" = 3 nodes. Total nodes = 14.

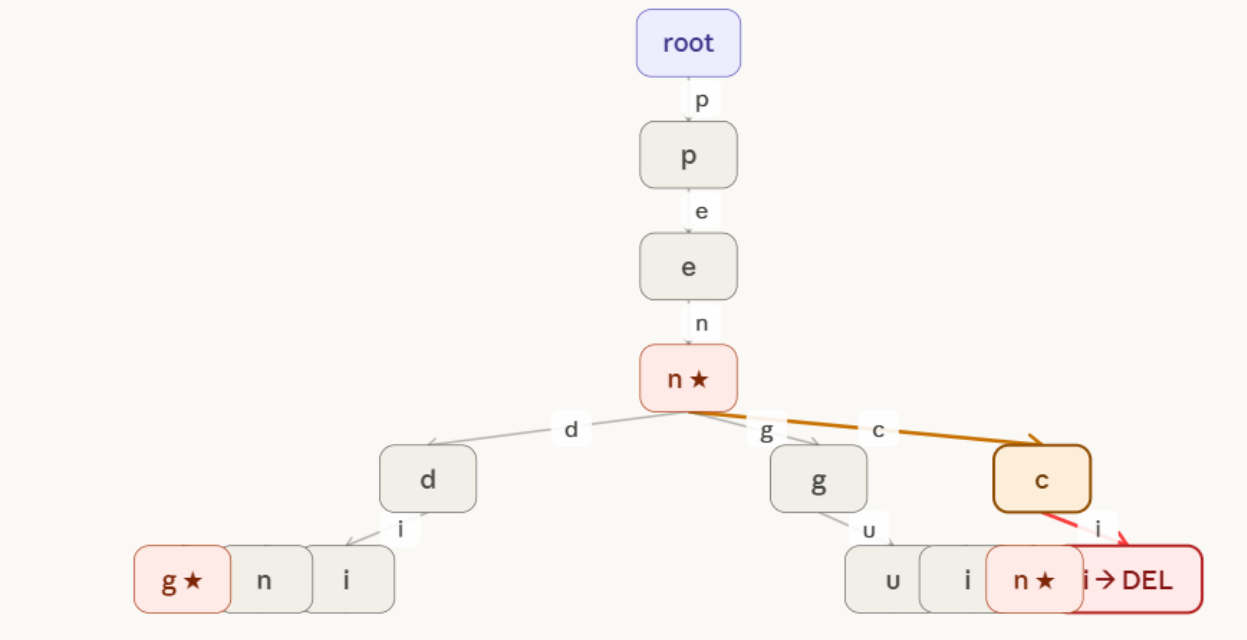


Deletion of pencil

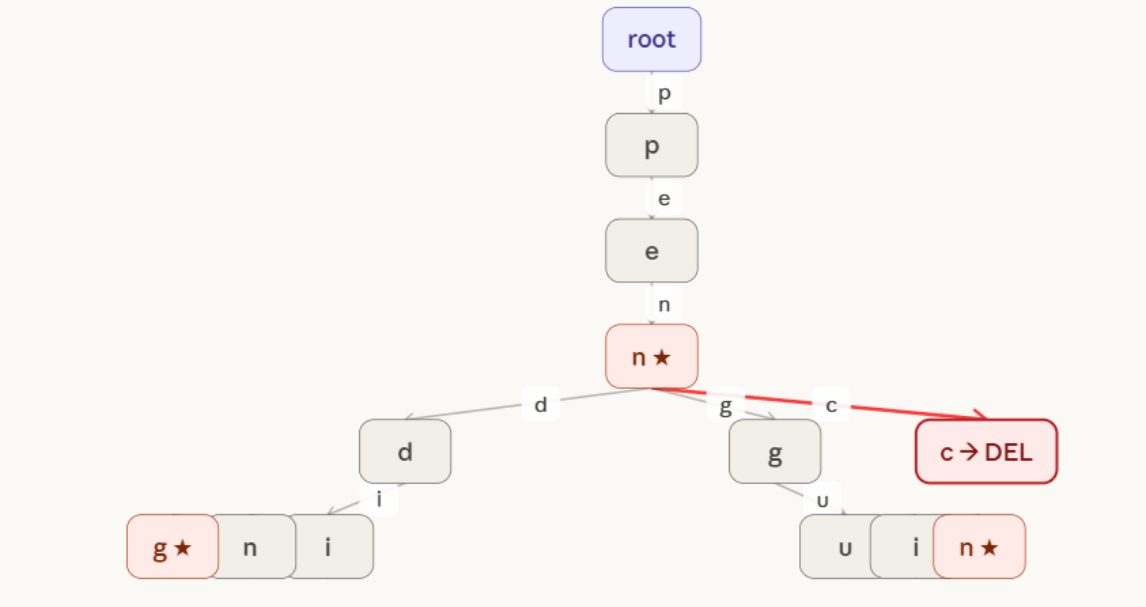
Step 1: Locate the word. Follow path: root → p → e → n → c → i → l ★. Leaf "l ★" is found. It is a LEAF with no children → safe to remove directly.



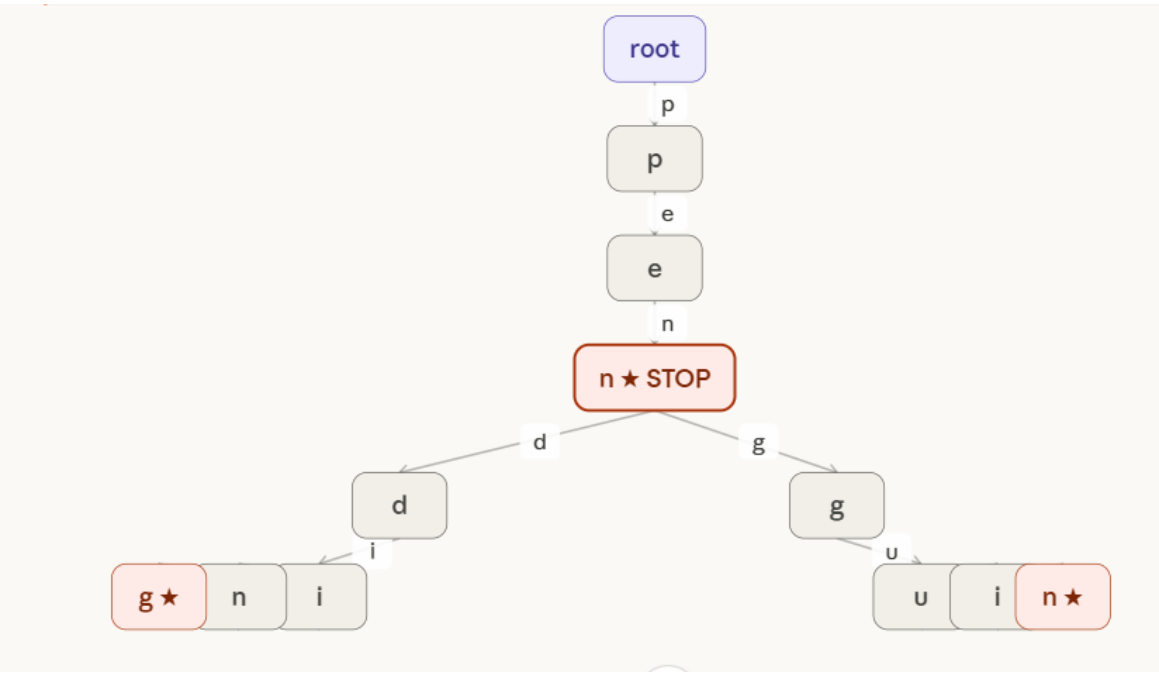
Step 2: Remove leaf "l ★". Check parent "i": it now has 0 children and is NOT end-of-word → REMOVE "i" too. Walk up the chain.



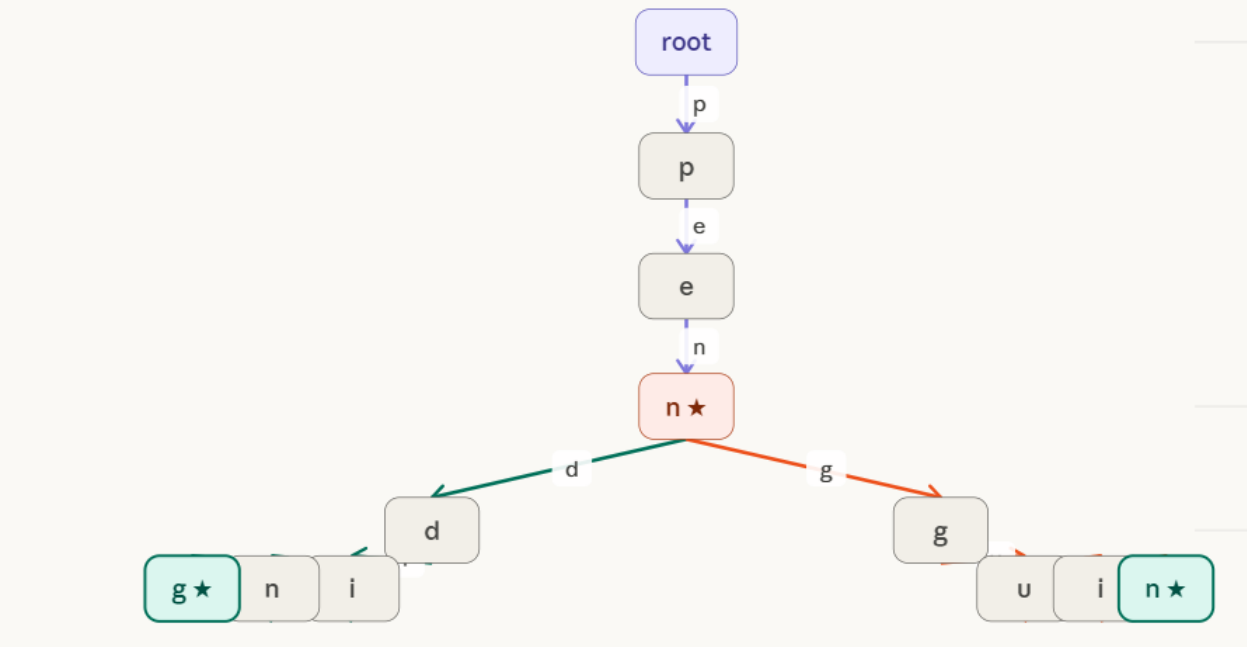
Step 3: Remove "i". Check parent "c": 0 children, NOT end-of-word → REMOVE "c" too. Walk up to "n ★".



Step 4: Remove "c". Check parent "n ★": it IS end-of-word (marks "pen") → STOP. Do NOT remove "n". The deletion cascade halts here. "pen" is preserved.



FINAL TRIE after deleting "pencil". Nodes c, i, l removed (3 nodes deleted). Node "n ★" kept because it marks "pen". "d" and "g" branches (pending, penguin) completely unaffected. Total nodes: 11.



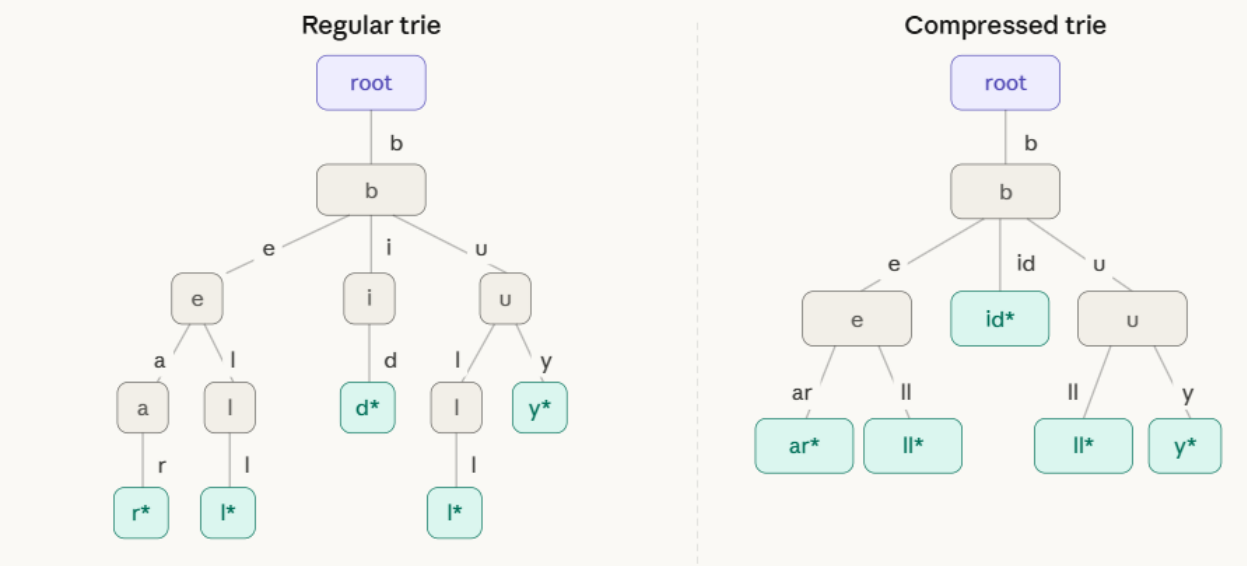
3 Compressed Trie or Radix Tree / Patricia Trie

A **compressed trie** (also called a Patricia trie or radix tree) is an optimised version of a standard trie where every node that has only one child is merged with that child. Instead of storing one character per edge, each edge can carry a whole string label (a multi-character substring).

The key insight: in a regular trie, long chains of single-child nodes waste space and time. Compressing these chains reduces the number of nodes from O(n × m) down to O(n), where n is the number of stored keys.

Trie vs Compressed Trie — side by side

Inserting {"bear", "bell", "bid", "bull", "buy"} into both structures:



Notice how the compressed trie on the right merges b→i→d into a single edge labeled "id", and b→e→a→r into "e" then "ar". Fewer nodes, same information.

Insertion Algorithm:

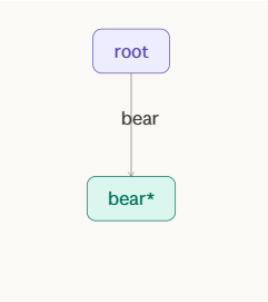
INSERT(root, w):

1. Start at root, current node = root
2. While characters of w remain:
   a. Look for an edge whose label starts with current char of w
   b. If NO matching edge found:
      → Create a new leaf node, attach with remaining w as edge label. DONE.
   c. If a matching edge IS found:
      → Walk along the edge comparing w with the edge label, char by char
      d. If w matches the ENTIRE edge label:
         → Move to child node; continue with remaining w
      e. If mismatch at position k (partial match):
         → SPLIT the edge at position k:
            i. Create a new internal node
            ii. The first k characters become the edge to the new node
            iii. Old child hangs below new node with remaining old label
            iv. New leaf hangs below new node with remaining new key
3. Mark final node as end-of-word

Example : Create Compressed Trie of the following strings {**“bear", "bell", "bid", "bull", "buy", then "be”}**

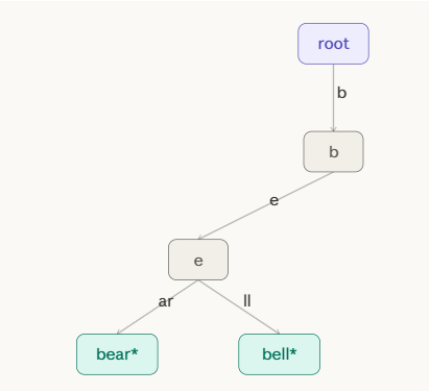
Solutions :

Step 1: Insert "bear" — tree is empty, create root → leaf with edge "bear".

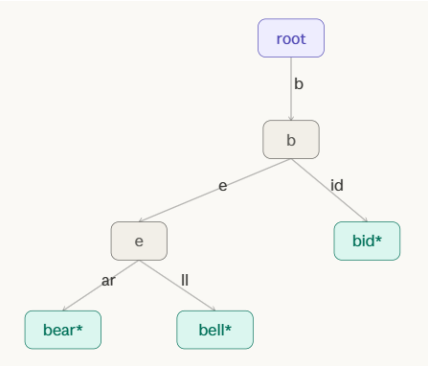


Step 2: Insert "bell" — match "be" with "bear", mismatch at position 2 (a vs l). Split:

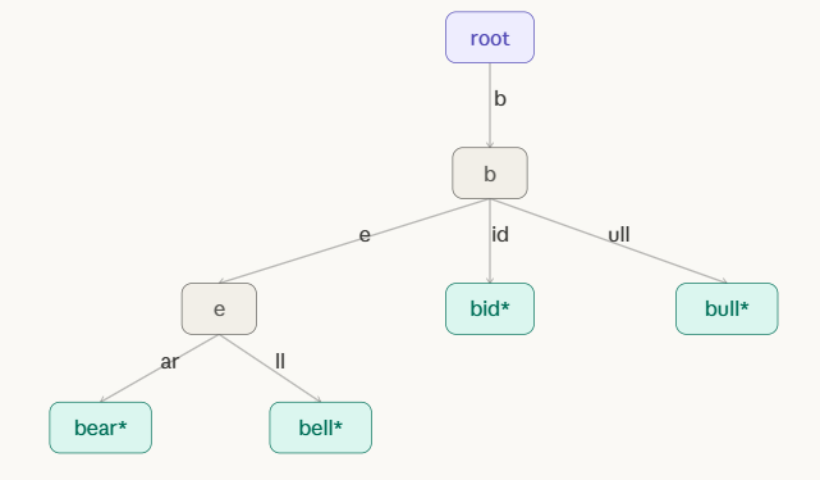
- Edge "bear" splits into "be" → internal node, then "ar" → leaf "bear", and "ll" → leaf "bell".



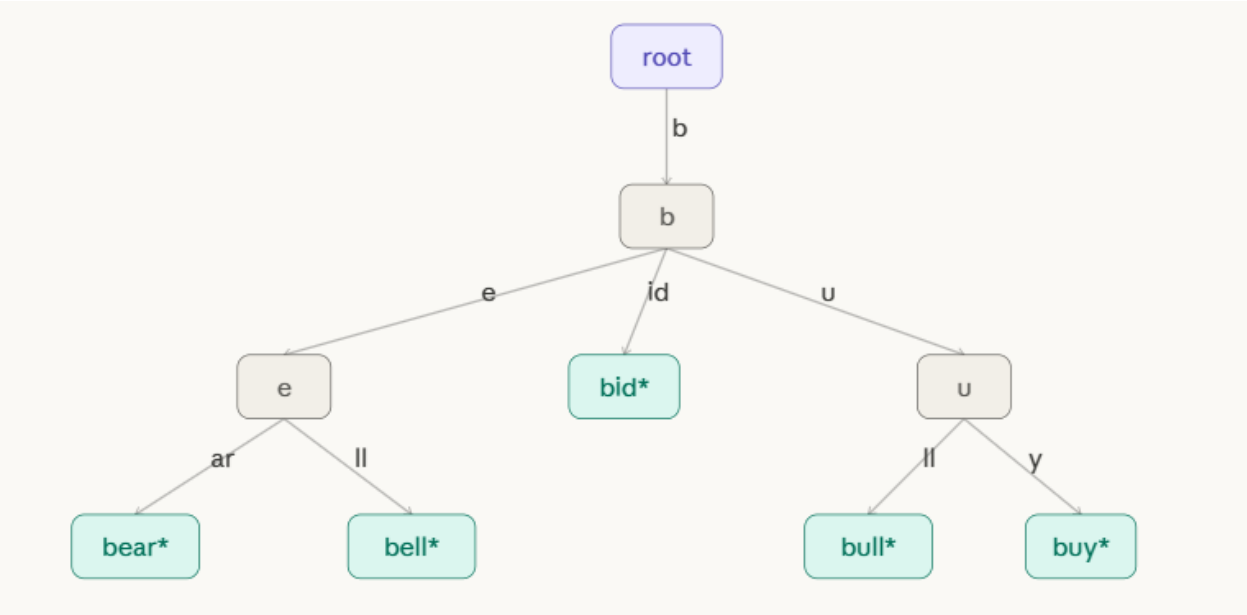
Step 3: Insert "bid" — at root, b matches. Reach internal node after "b". No edge starts with i. Create edge "id" → leaf.



Step 4: Insert "bull" — at root, b matches. Reach b node. No edge starts with u. Create edge "ull" → leaf.

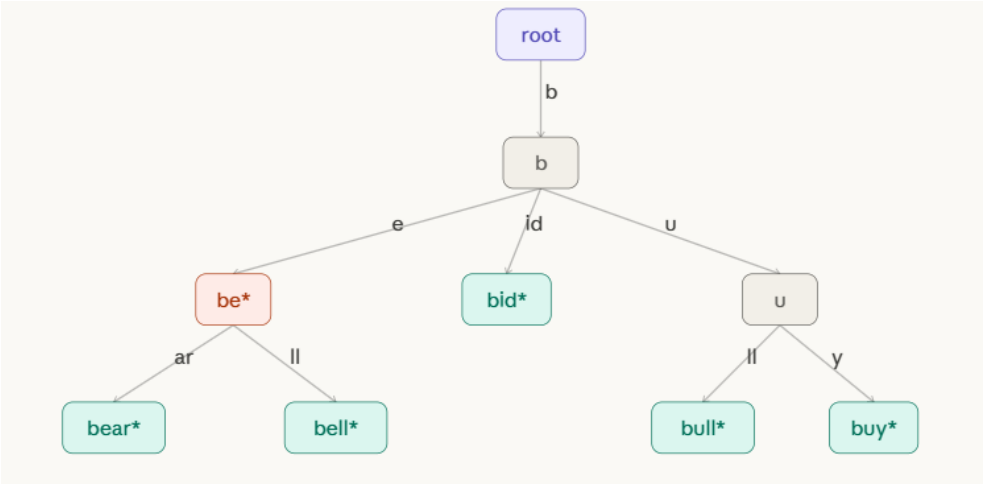


Step 5: Insert "buy" — at root, b matches. Reach b node. Edge "ull" starts with u. Match u, then mismatch (l vs y) at position 1. Split "ull" into "u" → internal, "ll" → "bull" leaf, "y" → "buy" leaf.



Step 6: Insert "be" — follow "b", then edge "e...". Match "e" fully (since key ends). Mark the "e" internal node as end-of-word.

The interactive stepper below walks through each insert with the tree state shown at every stage:



Deletion Algorithm

Goal: Delete key w from a compressed trie.

Algorithm

DELETE(root, w):

1. Search for the node that represents the end of w.

   If not found → key doesn't exist, return error.

2. Unmark the end-of-word flag at that node.

3. Case A — Deleted node is a LEAF:

   → Remove the leaf.

   → Move up to parent. If parent now has only ONE child

   AND parent is not root AND parent is not an end-of-word node:

   → MERGE parent + its remaining child:

   Concatenate edge labels, remove parent, attach grandparent

   directly to child with merged label.

   → Repeat merge check upward until no more single-child

   non-word internal nodes remain.

4. Case B — Deleted node is an INTERNAL node (has children):

   → Simply unmark end-of-word. Do NOT remove the node.

   → No merging needed (the node still has children).

Three cases of deletion

Case	Condition	Action
1	Node is a leaf	Remove leaf + potentially merge parent
2	Node is internal with 2+ children	Only unmark end-of-word
3	Node is internal with 1 child after sibling removed	Merge with sole remaining child

Example:

Starting from the final tree above containing: bear, bell, be, bid, bull, buy.

Delete "bear" (leaf, parent has 2 children):

- Locate leaf "ar" under "be" node.
- Remove leaf "ar". Parent "be" still has child "ll", so no merge needed. Done.

Delete "bell" (leaf, parent now has 1 child):

- Remove leaf "ll". Parent "be" (end-of-word for "be") now has 0 children.
- But "be" is an end-of-word node — cannot merge. Remove it as a leaf.
- Parent "b" now has fewer children — check: "b" still has "id" and "u", so no merge.

Delete "be" (internal node with children):

- Node "e" is marked as end-of-word. Unmark it. It still has children "ar" and "ll", so keep the node. Done.

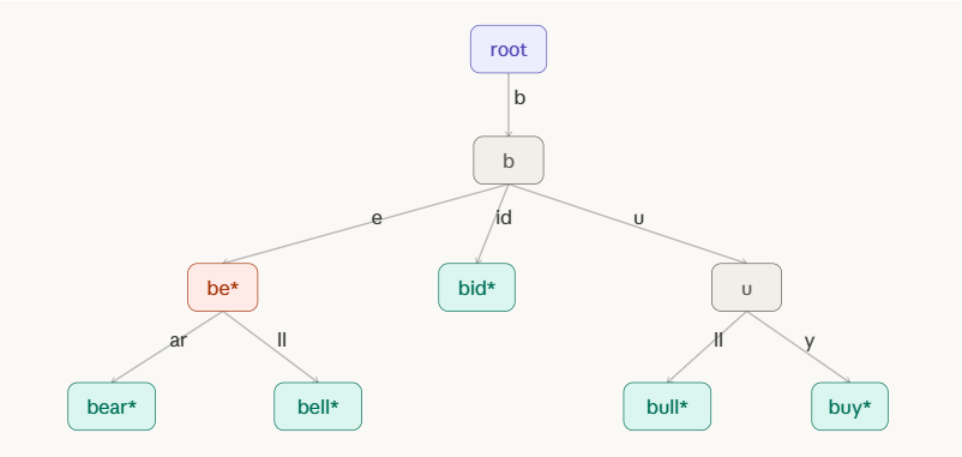
Delete "bid" (leaf, parent has 2+ children):

- Remove leaf "id". Parent "b" still has "e" and "u". No merge.

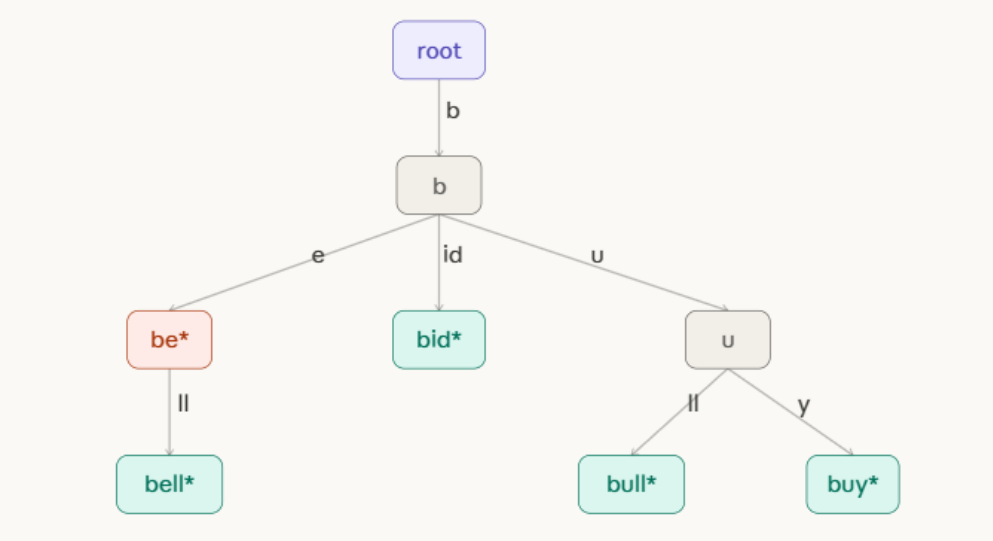
Delete "bull" (leaf, triggers merge):

- Remove leaf "ll" under "u" node. Parent "u" now has only child "y" (buy).
- "u" is internal, not end-of-word, has only 1 child → MERGE: collapse "u" + "y" into single edge "uy" directly from "b".

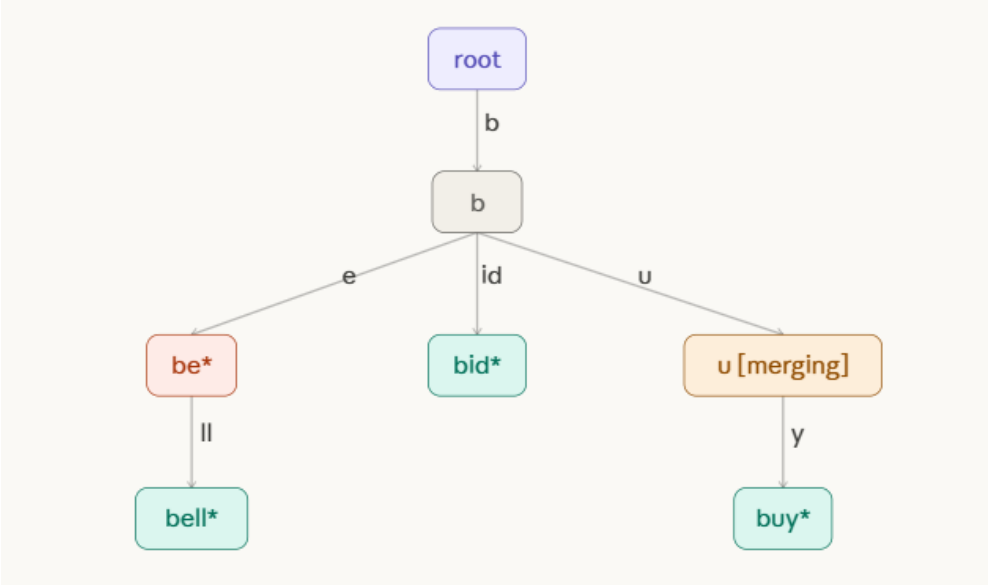
Initial tree with: bear, bell, be, bid, bull, buy. We will delete "bear", then "bull" to demonstrate the merge.



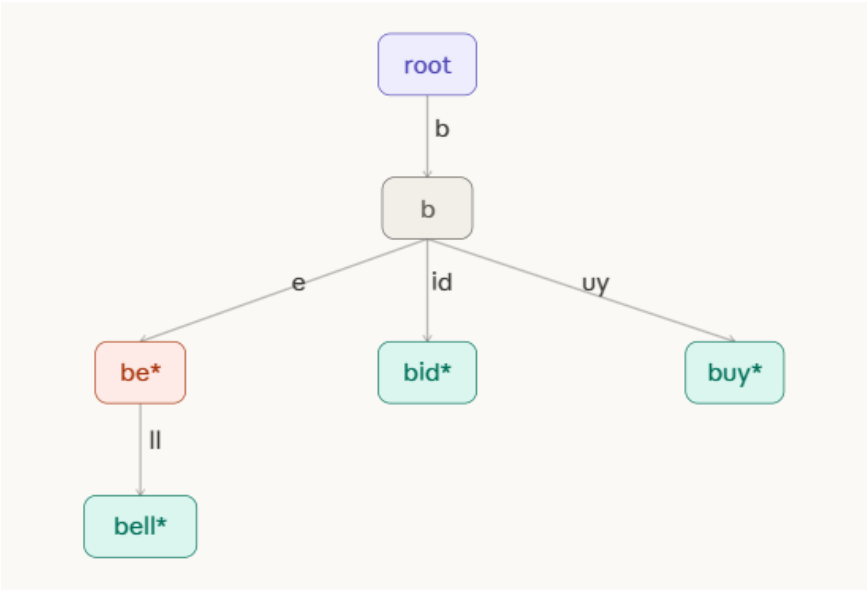
Delete "bear": leaf node "ar" removed. Parent "be" still has child "ll" → no merge needed. "be" remains (it is an end-of-word).



Delete "bull": remove leaf "ll" under "u". Parent "u" now has only ONE child ("y") and is NOT end-of-word → MERGE triggered!



MERGE complete: node "u" is removed. Edge "u" + "y" merge into single edge "uy" directly from "b" → "buy" leaf. Tree is now compressed again.



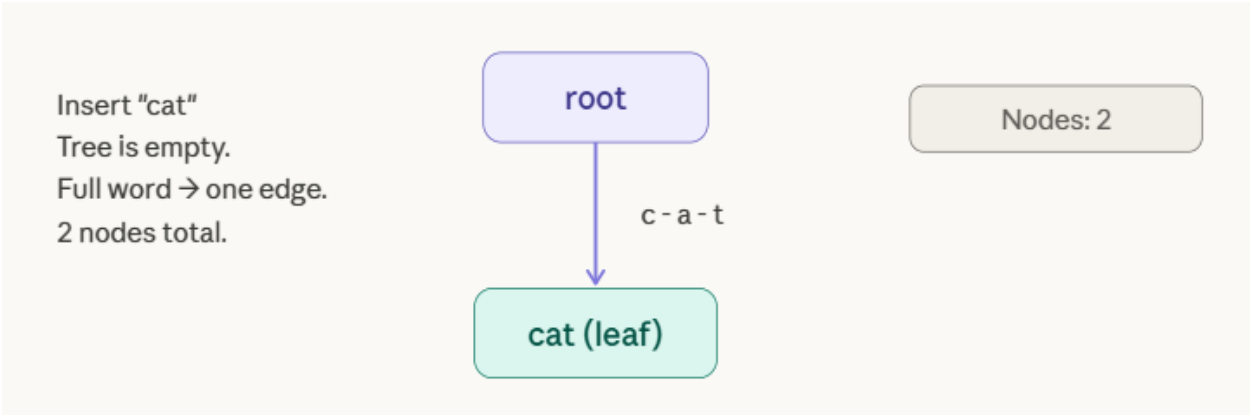
Problems:

1. **Construct a Compressed Trie by inserting the strings {cat, car, cart, care} and explain how path compression reduces the number of nodes. Delete the word cart and describe the changes in the trie structure.**

Solution:

Step 1 : Insert "cat"

The tree is empty. Create the root node and a single leaf. The entire word "cat" becomes the edge label. No compression needed yet.

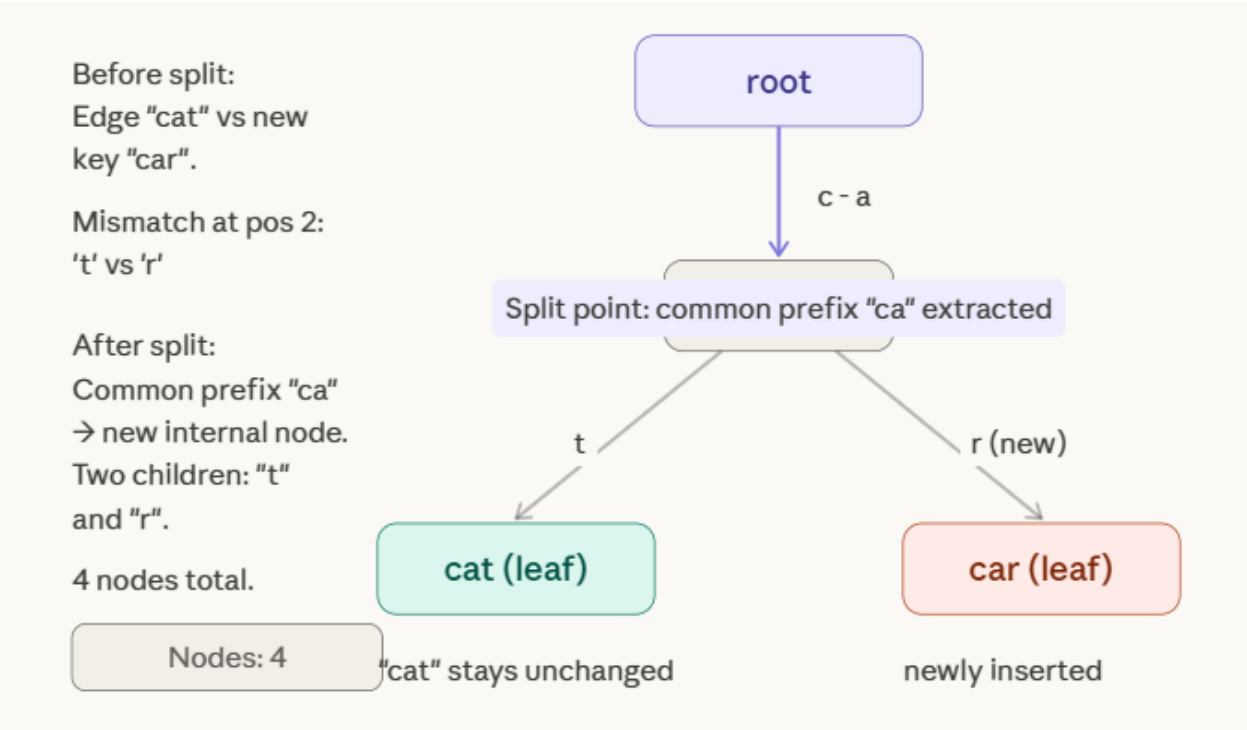


Step 2 : Insert "car"

Compare "car" against the existing edge label "cat" character by character:

- **c == c ✓**
- **a == a ✓**
- **r ≠ t ← mismatch at position 2**

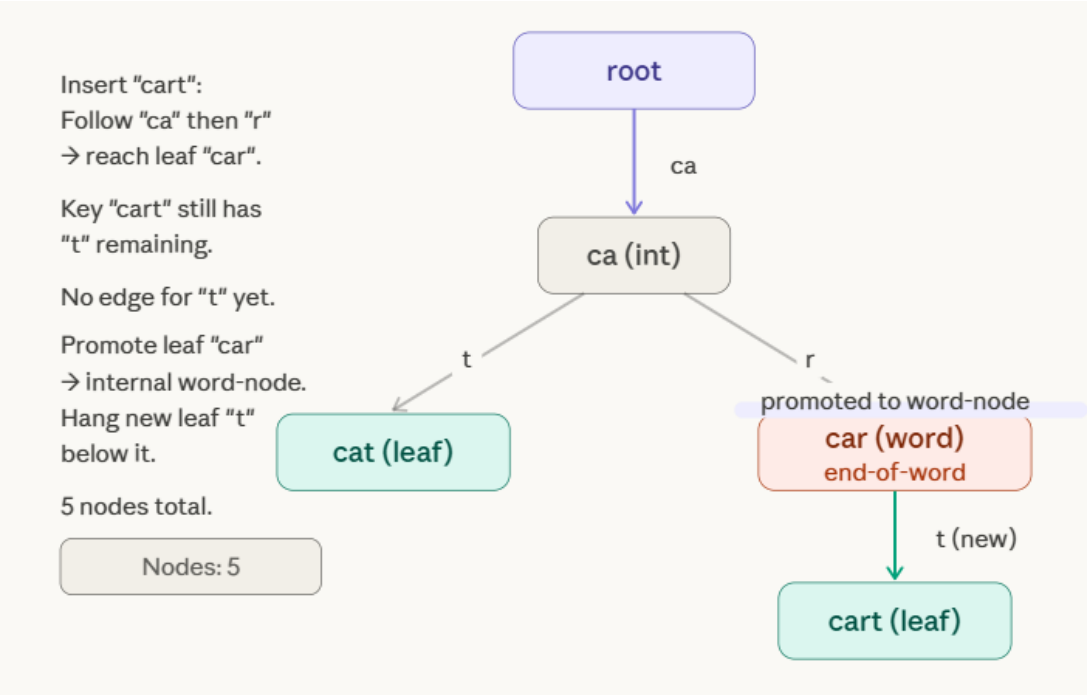
Action — SPLIT: The common prefix "ca" becomes a new internal node. The old leaf keeps edge "t". The new leaf "car" gets edge "r".



Step 3 — Insert "cart"

Follow the tree: root → edge "ca" → internal node → edge "r" → leaf [car]. The word "cart" matches "car" fully, but still has one character "t" left.

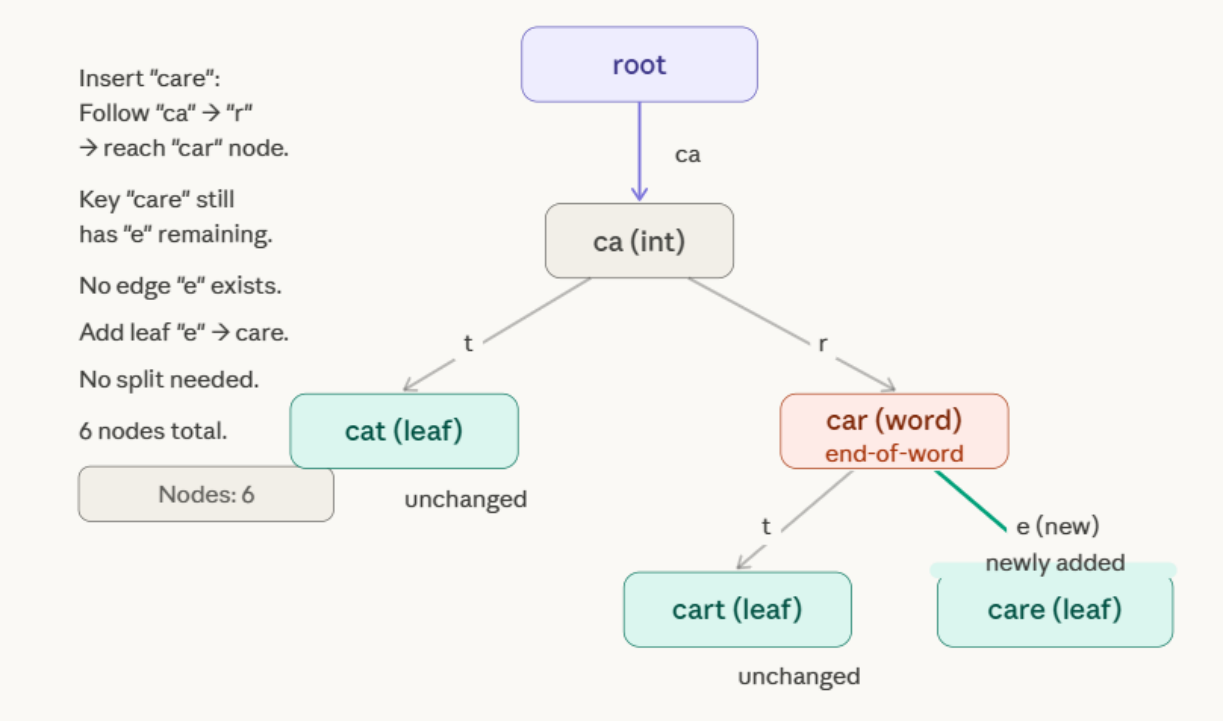
Action: The leaf [car] is promoted to an internal word-node (it still marks end of "car"), and a new leaf is attached with edge "t".



Step 4 **Insert "care"**

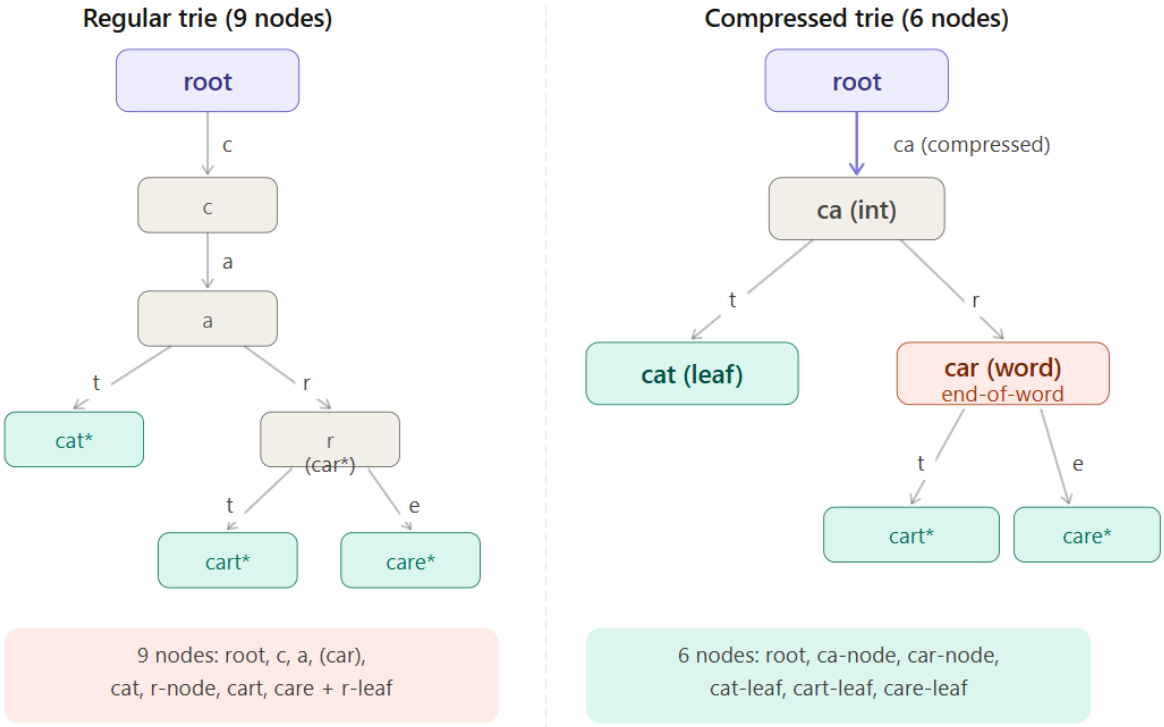
Follow: root → "ca" → internal node → "r" → word-node [car]. The word "care" still has "e" remaining. The node [car] has no child with edge starting "e".

Action: Simply add a new leaf [care] with edge "e" below [car]. No split needed.



Path Compression — How Many Nodes Were Saved?

Before seeing the deletion, here's a side-by-side comparison showing exactly what compression eliminates.

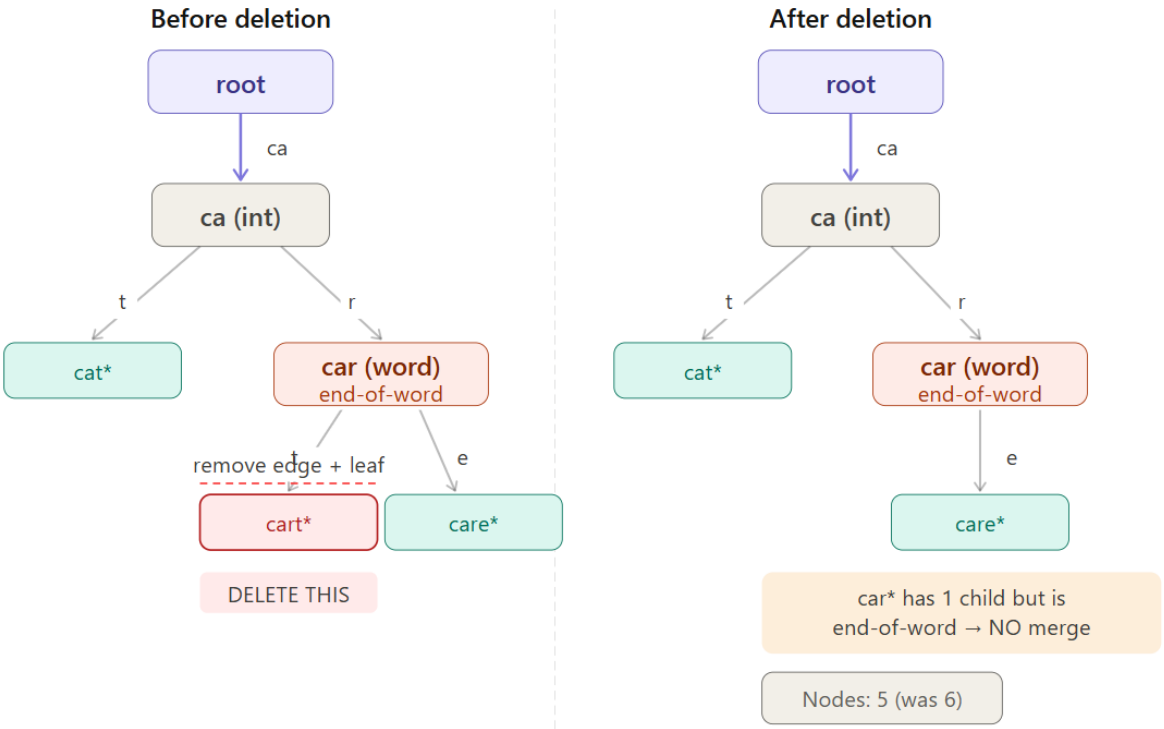


The regular trie needs individual nodes for c, a, and a separate r branching node — 3 extra nodes just to spell out the shared prefix. Path compression collapses the entire chain c → a into one edge labeled "ca", and makes r an edge label rather than a node. That is what compression means: chains of single-child nodes become single labeled edges.

Step 5 — Delete "cart"

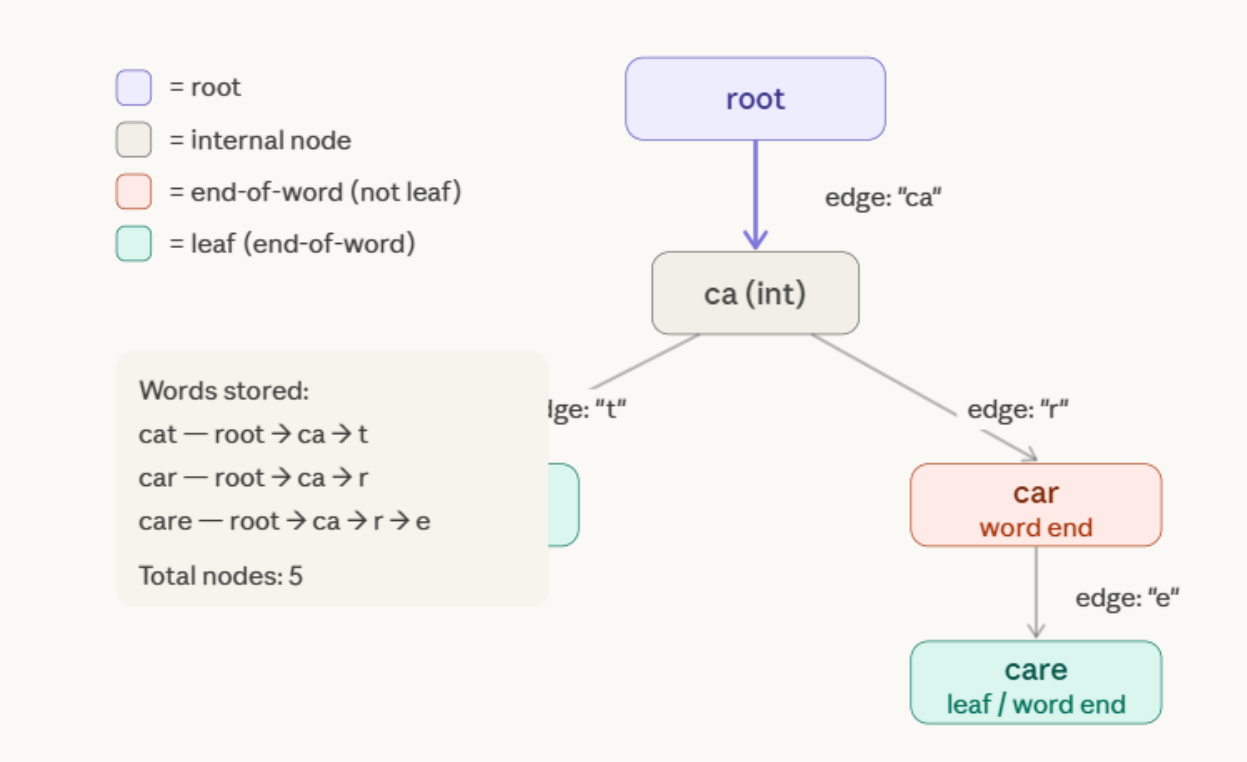
Three things must be checked in order:

1. **Find the node — follow root → "ca" → "r" → "t" → found leaf [cart].**
2. **Remove the leaf — detach it from its parent [car].**
3. **Check parent for merge — [car] now has only one child ([care]). BUT [car] is marked as an end-of-word. The merge rule says: *only merge if the node is NOT an end-of-word*. So no merge happens.**



Final State — Clean Compressed Trie

This is the validated final tree containing {cat, car, care}.



Problem:

Insert the words {apple, apply, apt, ape} into a Compressed Trie and draw the resulting structure. Explain how deletion of the word apt is handled without affecting other compressed paths.

Solutions:

Prefix analysis before we start

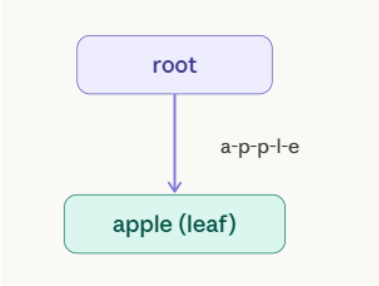
All four words share the prefix "ap". After that they diverge:

- **apple → ap + ple**
- **apply → ap + ply**
- **apt → ap + t**
- **ape → ap + e**

So the root edge will be "ap", branching into four children from a shared internal node.

Step 1 — Insert "apple"

Tree is empty. Create root → leaf with full label "apple".

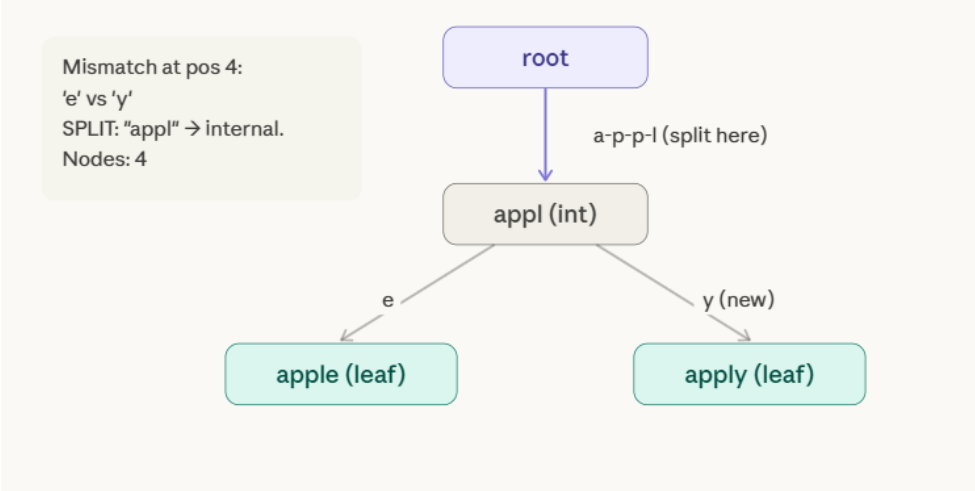


Step 2 — Insert "apply"

Compare "apply" vs edge "apple":

- **a==a, p==p, p==p, l==l, y ≠ e → mismatch at position 4**

SPLIT at position 4. Common prefix "appl" → new internal node. Old leaf keeps "e", new leaf gets "y".

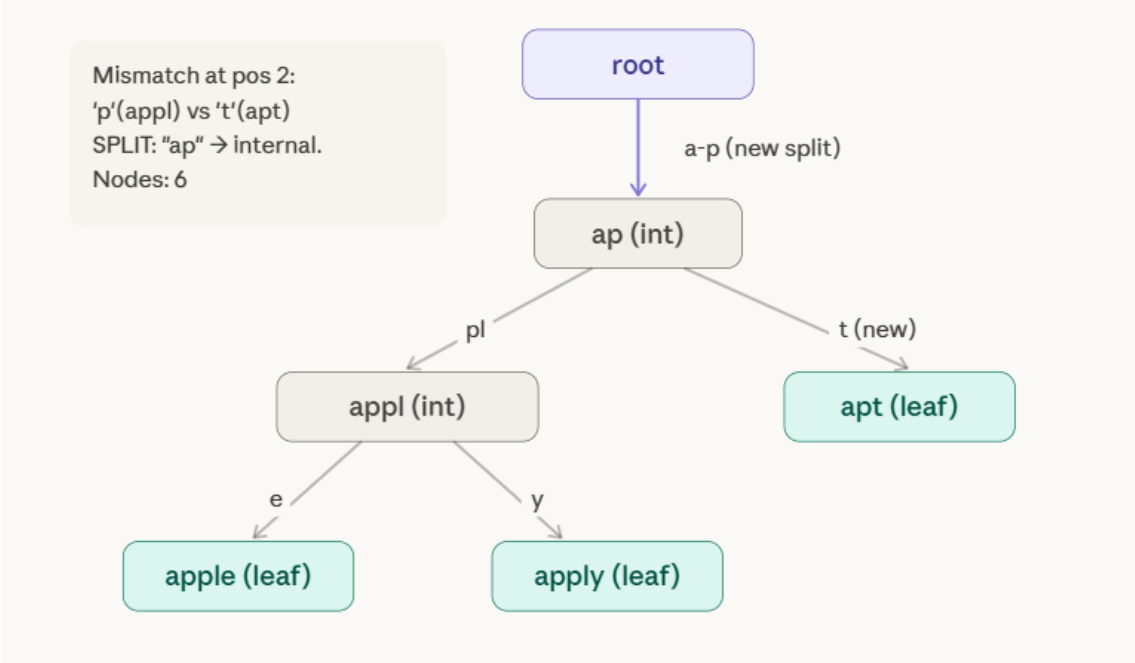


Step 3 — Insert "apt"

Compare "apt" vs edge "appl":

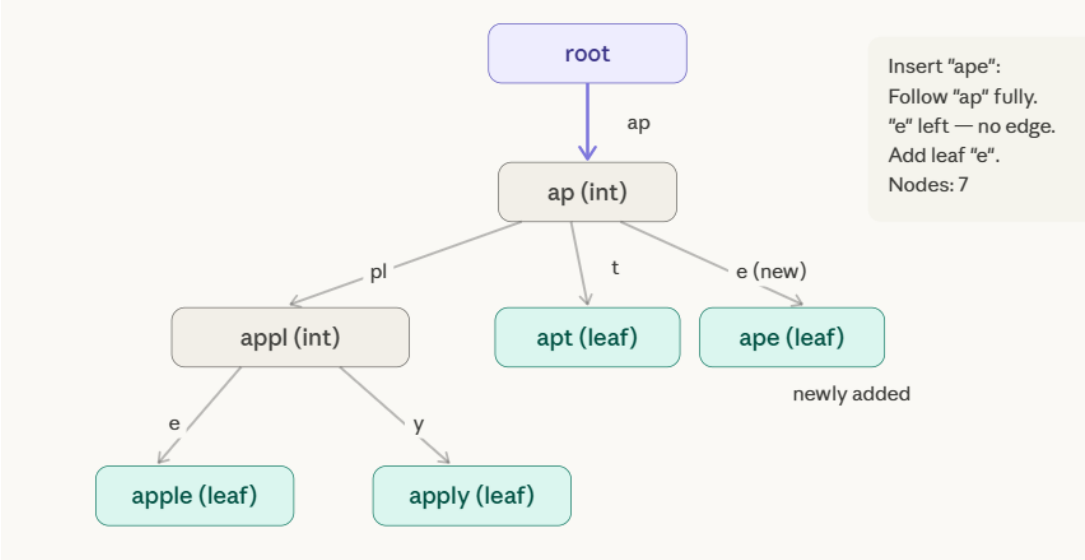
- **a==a, p==p, t ≠ p → mismatch at position 2**

SPLIT at position 2. Common prefix "ap" → new internal node. Old subtree keeps "pl" as its edge, new leaf "apt" gets "t".



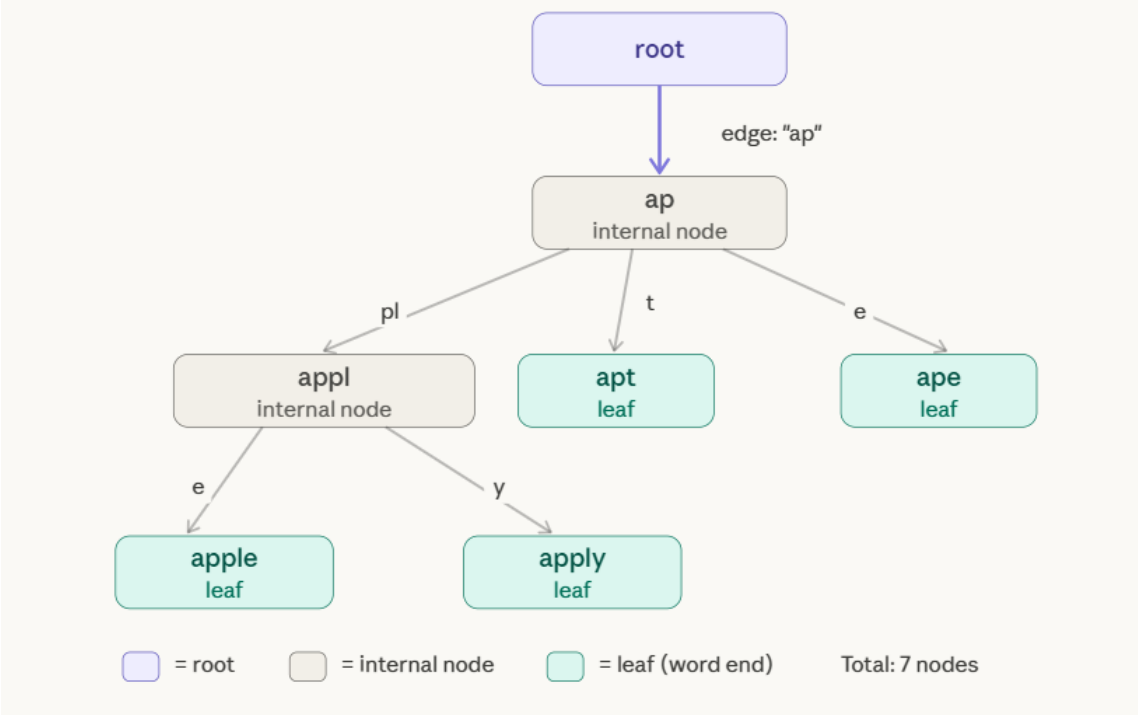
Step 4 — Insert "ape"

Follow: root → "ap" → internal node [ap]. Key "ape" still has "e" remaining. No child edge starts with "e" — simply add a new leaf with edge "e". No split needed.



Final Compressed Trie — all 4 words

Here is the complete, clean structure with color-coded node types and every edge label shown clearly.



How path compression reduced nodes

A regular (uncompressed) trie for the same four words needs a separate node for every individual character. Let's count:

Path in regular trie	Nodes used
root → a → p (shared)	2
→ p → l → e (apple)	3
→ p → l → y (apply)	1 new (l,p shared)
→ t (apt)	1
→ e (ape)	1
Total	~10 nodes

Delete Operation:

Delete "apt" :

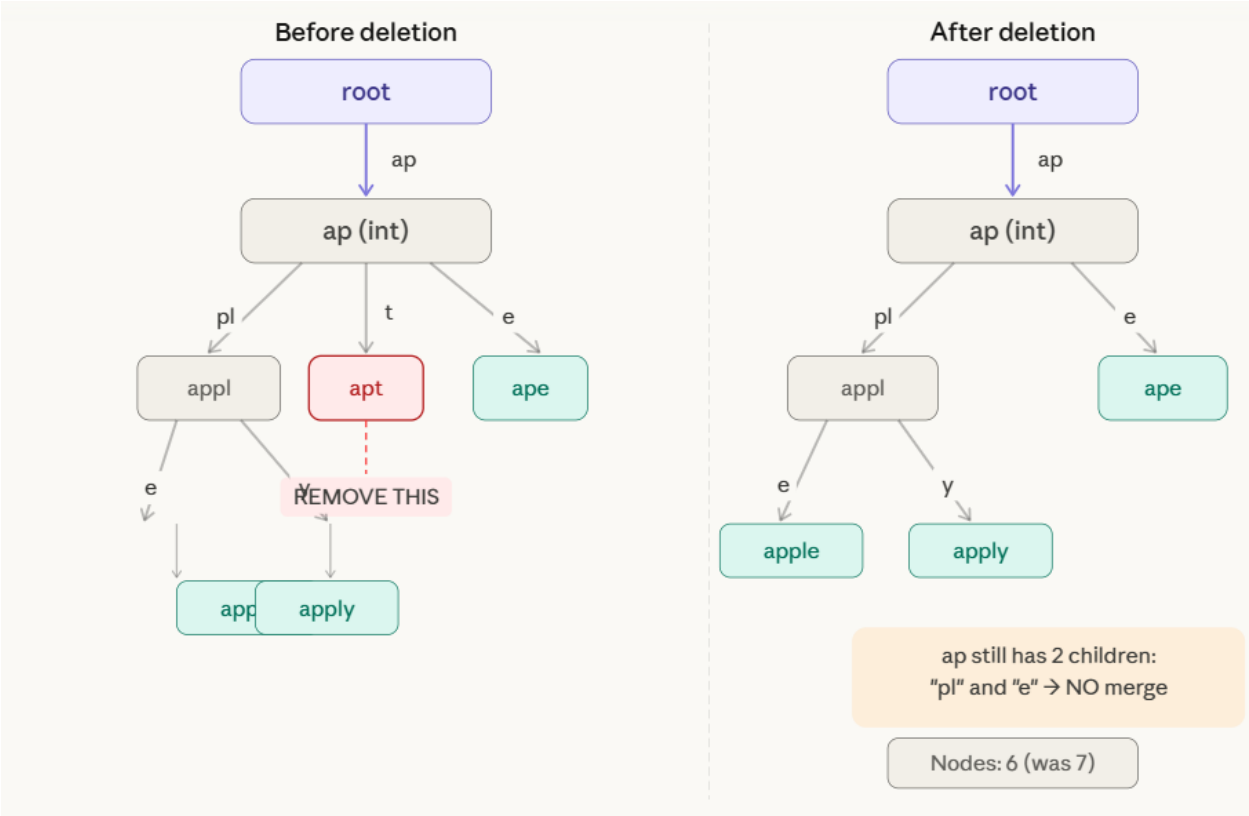
Now we delete "apt" from the final tree. Here is what must be checked at each stage.

Locate the node: Follow **root → "ap" → [ap] → "t" → leaf [apt].** Found.

Node type check: [apt] is a leaf (no children). Proceed with leaf removal.

Remove the leaf: Detach [apt] and its incoming edge "t". Node count drops from 7 to 6.

Check parent for merge: After removing [apt], the parent node [ap] now has 2 children remaining — edge "pl" to [appl], and edge "e" to [ape]. Since [ap] still has 2 children, the merge condition (exactly 1 child, not end-of-word) is not triggered. No structural change beyond the leaf removal.



Problems:

2. **A search engine stores keywords using a Compressed Trie. Insert the strings {data, date, database, dawn} and show how common prefixes are compressed. Delete the keyword date and explain how node merging is performed after deletion.**
3. **Construct a Compressed Trie for the strings {pen, pencil, pending, penguin}. Explain the insertion process and show how deletion of pencil affects internal and leaf nodes.**

4 Suffix Trie

A **Suffix Trie** is a trie data structure that stores **all suffixes** of a given string. For a string S of length n, it stores exactly n suffixes — each suffix starting at position i runs from S[i] to the end of S.

The key difference from a regular trie: a suffix trie is built from **one string**, inserting all its suffixes simultaneously. Every path from root to a leaf spells out a complete suffix of S. Every path from root to any node (leaf or internal) represents a **substring** of S. This is what makes suffix tries so powerful for substring search.

Example:

We use the string **S = "banana"** with a terminal character **$** appended, giving us "banana$". The $ is a unique sentinel that does not appear in S — it guarantees every suffix ends at a distinct leaf.

The 7 suffixes to insert are:

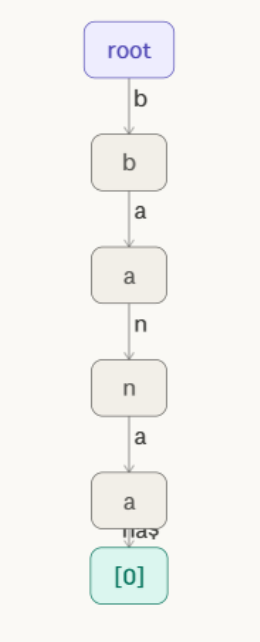
#	**Suffix**
0	banana$
1	anana$
2	nana$
3	ana$
4	na$
5	a$
6	$

Insertion

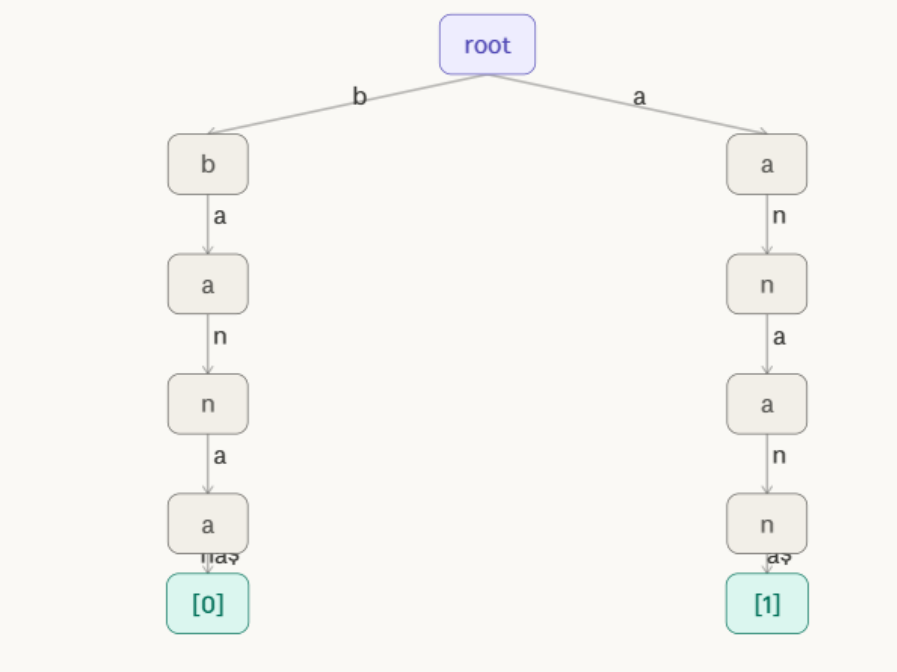
Each suffix is inserted exactly like a word into a normal trie: starting from the root, follow (or create) one edge per character. The leaf reached at the end is labeled with the suffix's starting position.

The interactive stepper below walks through all 7 insertions one at a time, showing exactly how the trie grows after each suffix is added.

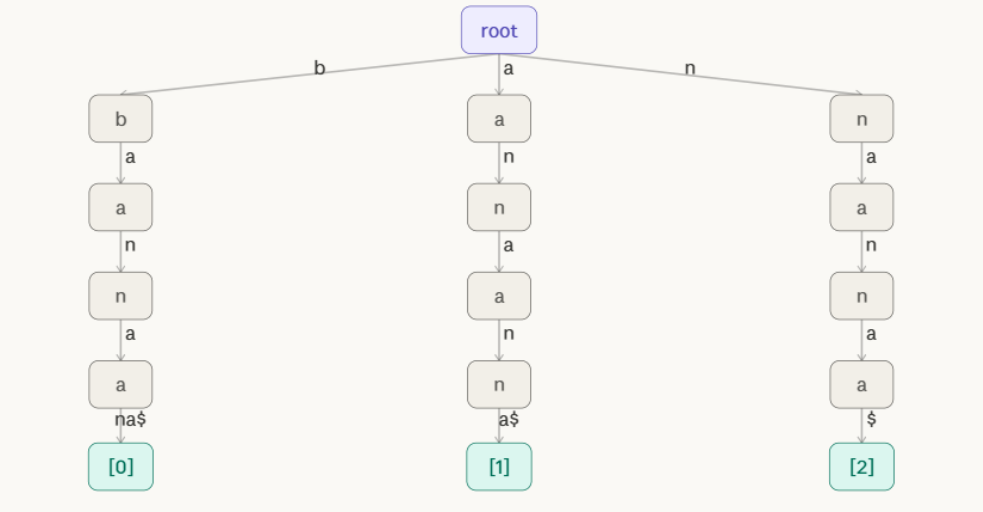
Step1: Insert "banana$" (suffix 0): tree empty. Create one path root→b→a→n→a→n→a→$ with leaf [0].



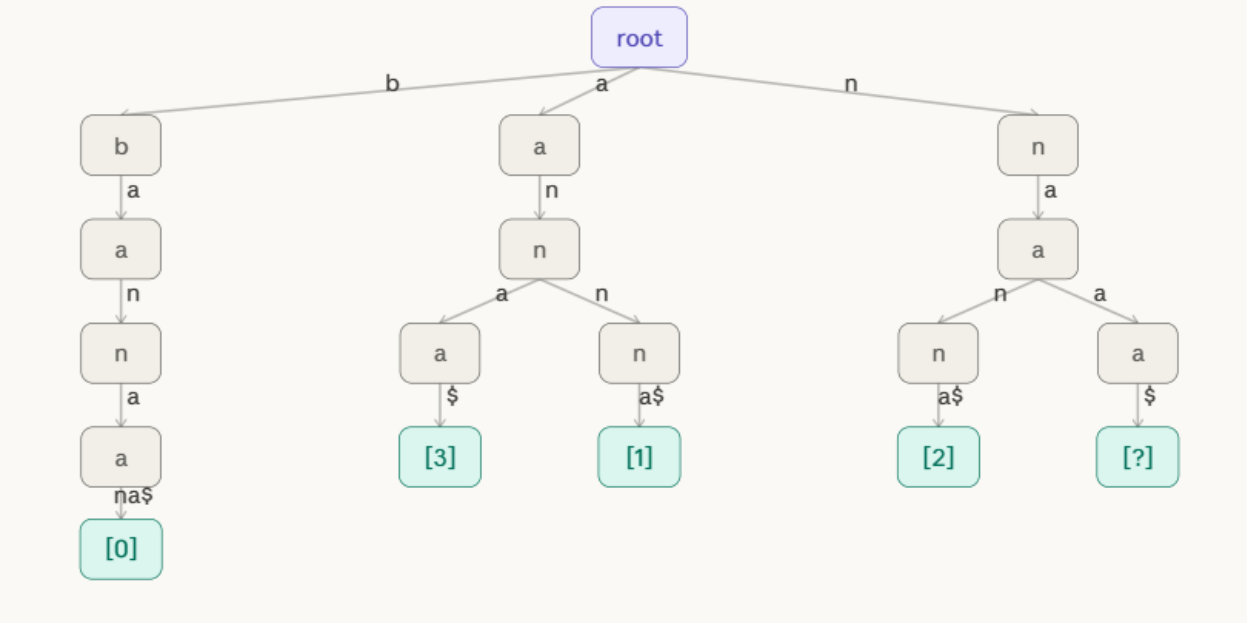
Step 2: Insert "anana$" (suffix 1): no "a" edge from root, create new branch. Leaf [1] added.



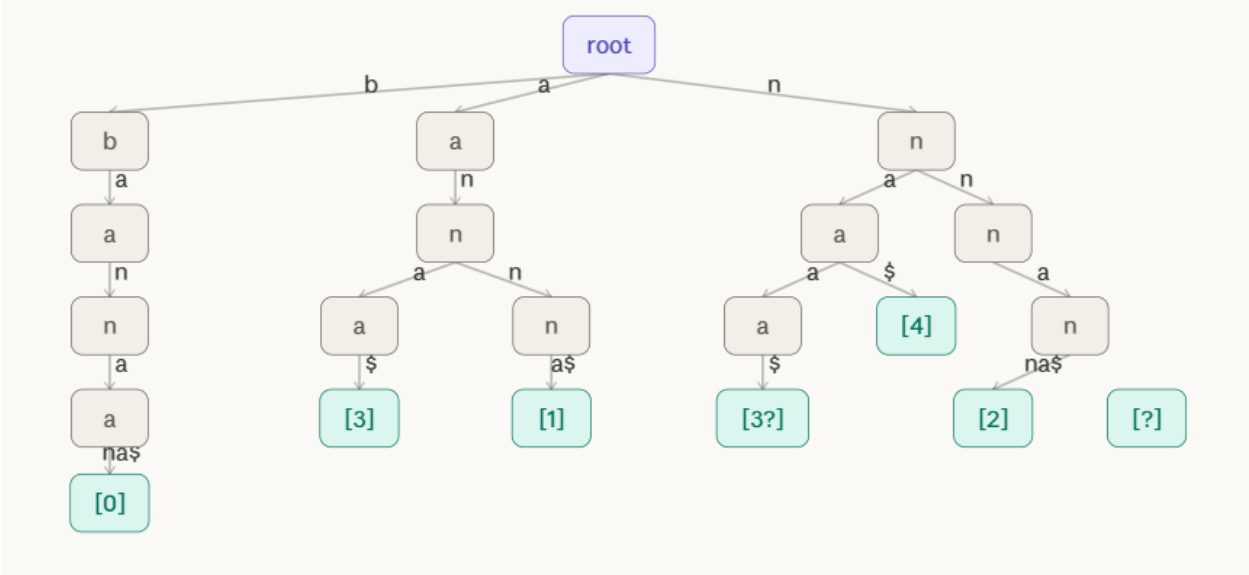
Step 3: Insert "nana$" (suffix 2): no "n" edge from root, create new branch n→a→n→a→$ with leaf [2].



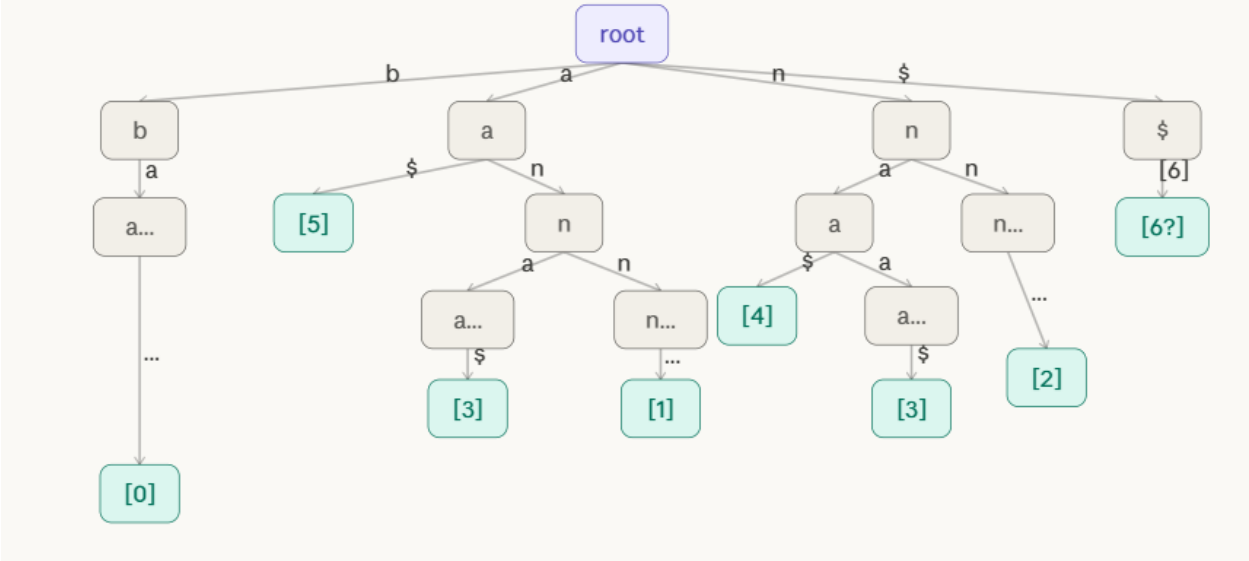
Step 4: Insert "ana$" (suffix 3): "a" edge exists from root. Follow a→n→a, then "$" not found → add leaf [3] at a→n→a→$ node.



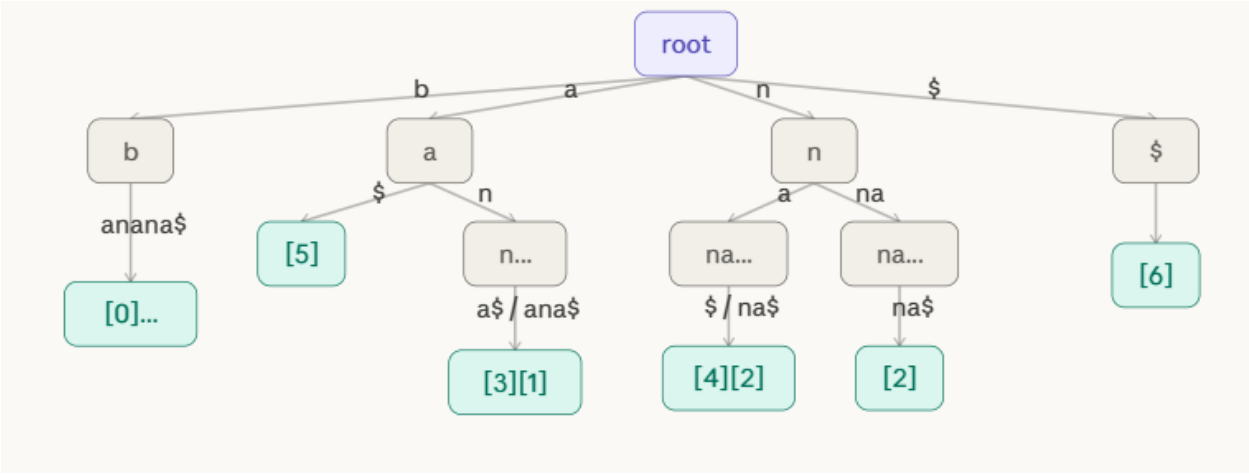
Step 5; Insert "na$" (suffix 4): "n" edge exists. Follow n→a, then "$" missing → add leaf [4]. Also adds "a$" for na$ path.



Step 6: Insert "a$" (suffix 5): "a" edge from root exists. Follow "a", then "$" not yet present → add leaf [5].

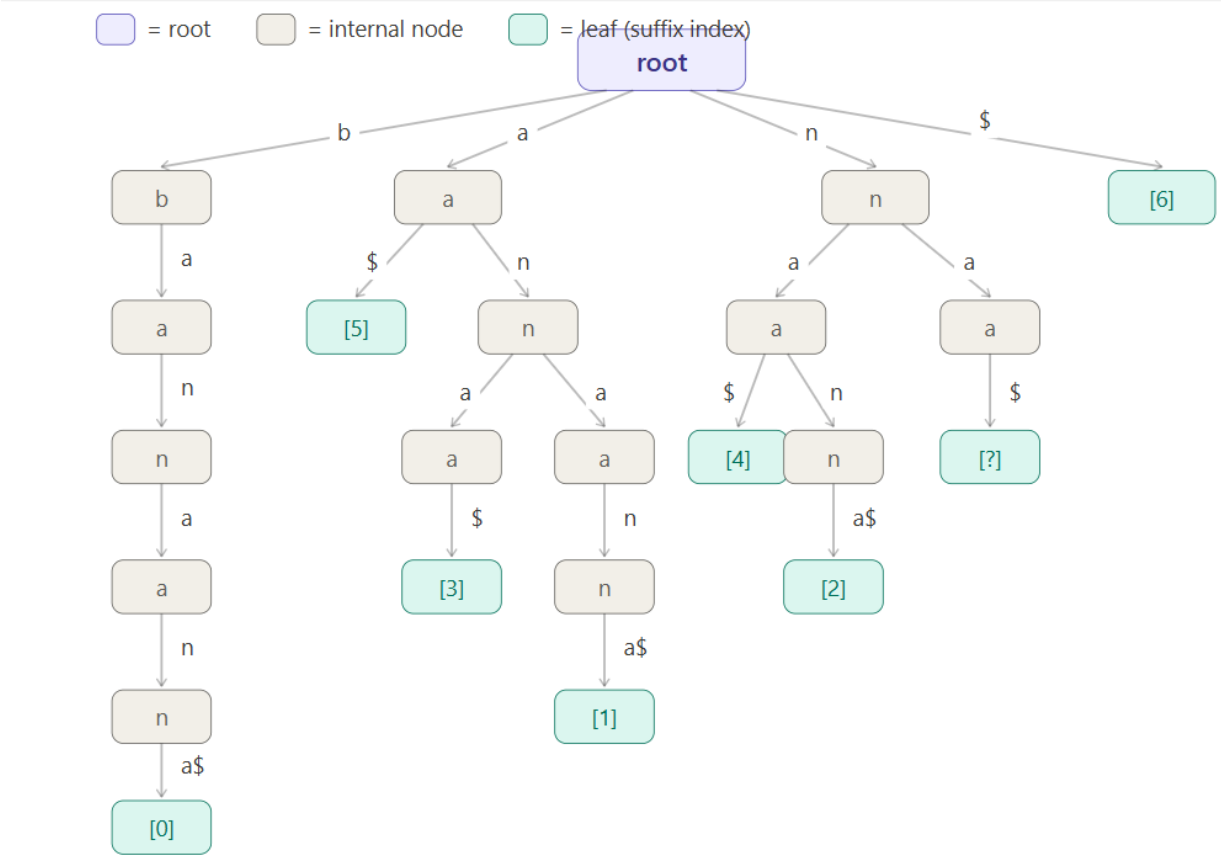


Step 7: Insert "$" (suffix 6): create root → "$" edge → leaf [6]. All 7 suffixes inserted. Every root-to-leaf path spells a complete suffix.



The Complete Suffix Trie for "banana$"

Once all 7 suffixes are inserted, the complete structure looks like this. Each root-to-leaf path spells one complete suffix; each root-to-node path spells one substring.



Search

Searching in a suffix trie is the most powerful operation. There are two kinds:

Substring search (does pattern P appear in S?): Follow the trie from the root, one character of P at a time. If you successfully consume all characters of P and reach any node — leaf or internal — then P is a substring of S. Count the leaves in the subtree below to find all occurrence positions.

Full suffix search (does suffix starting at position i exist?): Follow the path and check if the final node is a leaf labeled [i].

Example: Search for "ana" in "banana$"

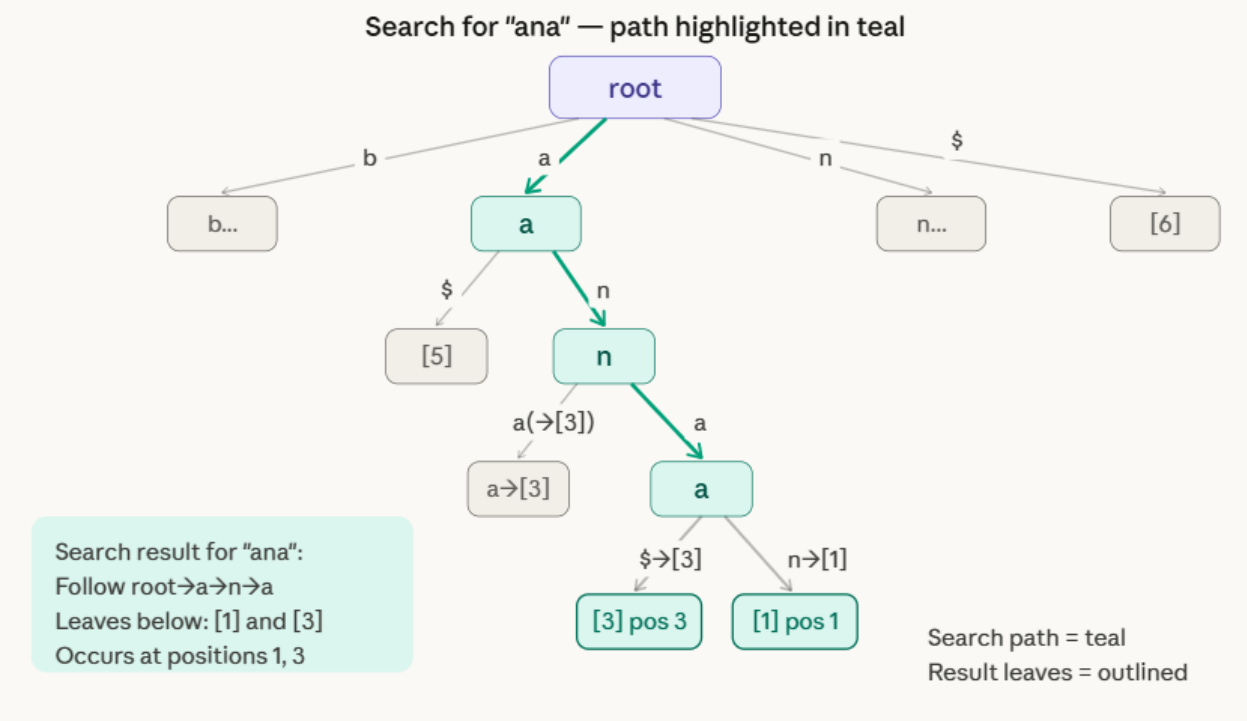
Follow root → a → n → a. We reach an internal node successfully. Every leaf in the subtree below spells a suffix that contains "ana" as a prefix. The leaves found are [1] (suffix anana$, "ana" at pos 1) and [3] (suffix ana$, "ana" at pos 3). So "ana" occurs at positions **1** and **3**.

Example: Search for "nan" in "banana$"

Follow root → n → a → n. Successfully reaches a node. Leaf below: [2] (suffix nana$). So "nan" occurs at position **2** only.

Example: Search for "xyz" in "banana$"

Follow root → x. No edge labeled x exists from root. Search fails immediately. "xyz" is not a substring.



Deletion

Deleting a suffix from a suffix trie means removing suffix S[i..]$ and cleaning up any nodes that become redundant. The rules mirror a standard trie deletion:

Algorithm:

DELETE(root, suffix_at_position_i):

1. Follow the path of suffix S[i..]$ from root to its leaf [i].
2. If path not found → suffix does not exist, error.
3. Remove the leaf [i].
4. Walk back up toward the root:

   For each ancestor node just above the removed node:

   If that ancestor now has 0 children AND is not a valid suffix endpoint itself:

   Remove it too.

   Else: STOP — node is still needed.

Example: Delete suffix "na$" (suffix at position 4)

Path to follow: root → n → a → $ → leaf [4].

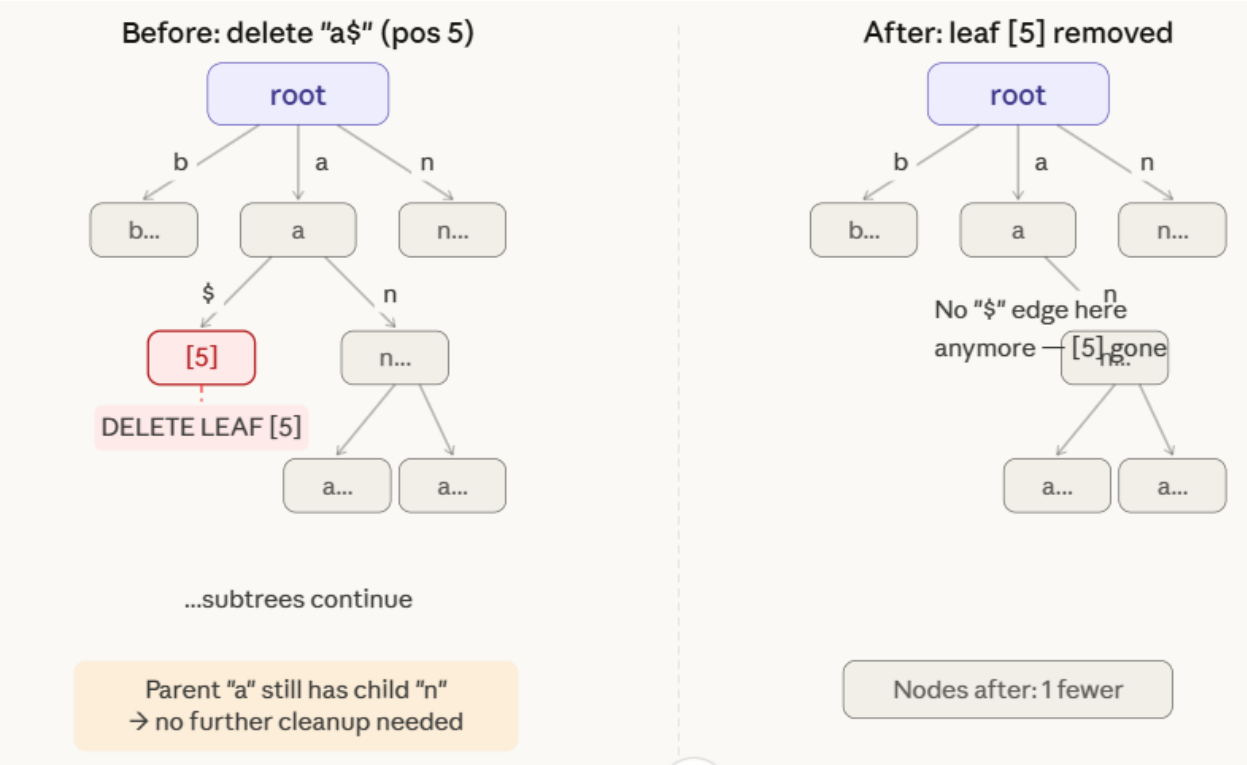
- Remove leaf [4].
- Check parent node a (under n): it still has a child leading to nana$. So it stays.
- No further cleanup needed. Only 1 node removed.

Example: Delete suffix "a$" (suffix at position 5)

Path: root → a → $ → leaf [5].

- Remove leaf [5].
- Check parent node a (from root): it still has the n branch (for ana$, anana$). So it stays.
- Only 1 node removed.

The diagram below shows the before/after for deleting suffix "a$" (position 5)



5 Multi-Way Trie

A multi-way trie (also called an m-ary trie or alphabet trie) is a tree in which:

- each node can have up to **m children**, where m is the size of the alphabet (e.g. 26 for lowercase English)
- each edge is labeled with a single character
- each node may be marked as an **end-of-word** (★)
- the path from root to any ★-node spells exactly one stored key

Unlike a binary search tree which stores keys in nodes, a trie encodes keys in the **paths** themselves. Two words that share a prefix share the same path until they diverge.

2. Operations

Insert — O(m) where m = key length

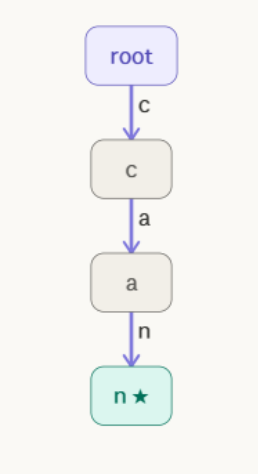
Walk the trie character by character. For each character of the key, follow the matching child edge if it exists, or create a new child node if it doesn't. After consuming all characters, mark the final node as end-of-word (★).

Three scenarios arise during insertion (all shown in the Insert tab above):

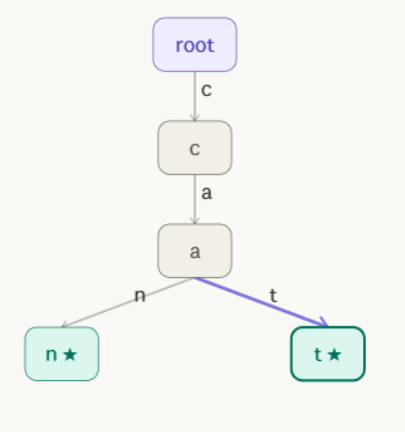
- Character's edge already exists → just follow it (shared prefix re-use)
- Character's edge is missing → create a new node and edge
- Key ends at an internal node → mark that existing node as ★

Example:

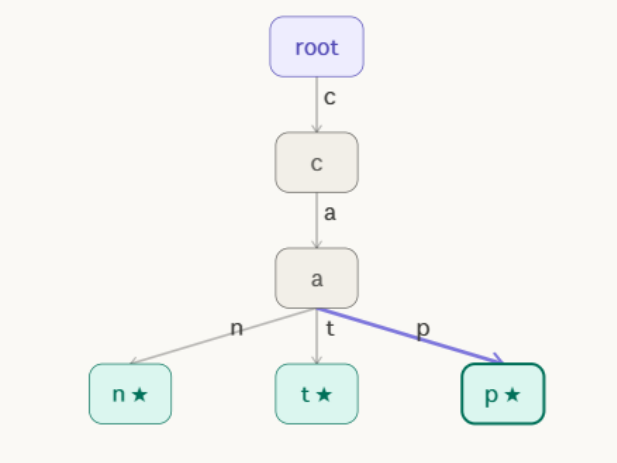
Step 1: Start: empty trie. Insert "can" — create root, then nodes for c→a→n. Mark n as end-of-word (★).



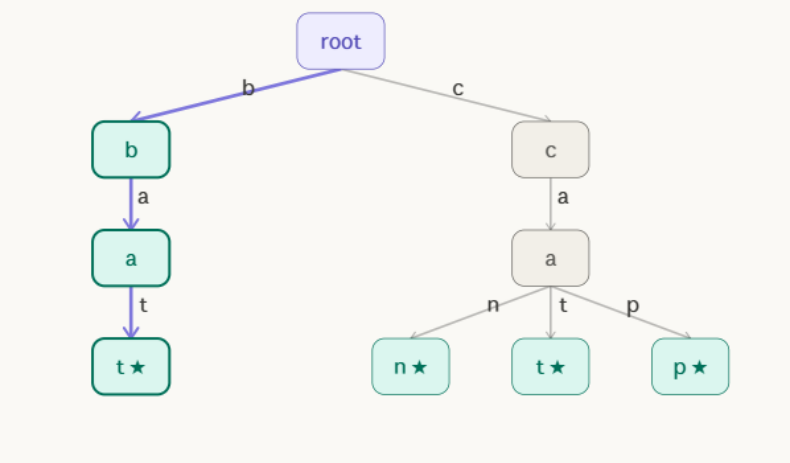
Step 2: Insert "cat": root→c already exists. a already exists. "t" is new — create leaf [t★] as sibling of [n★].



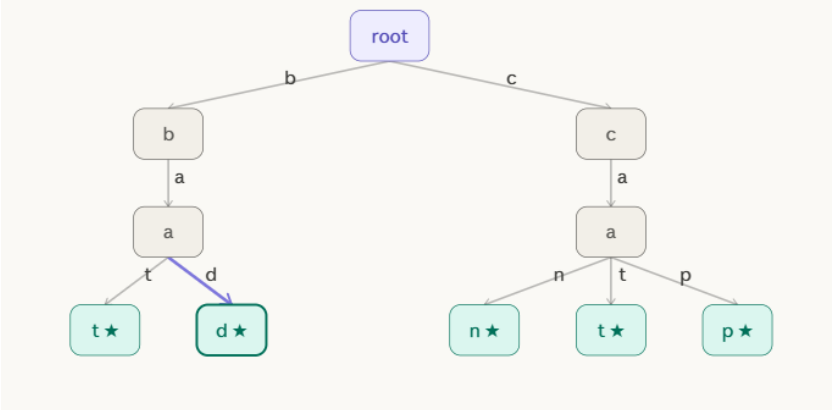
Step 3: **Insert "cap": root→c→a exist. "p" is new — add leaf [p★] under [a], 3rd child of [a].**



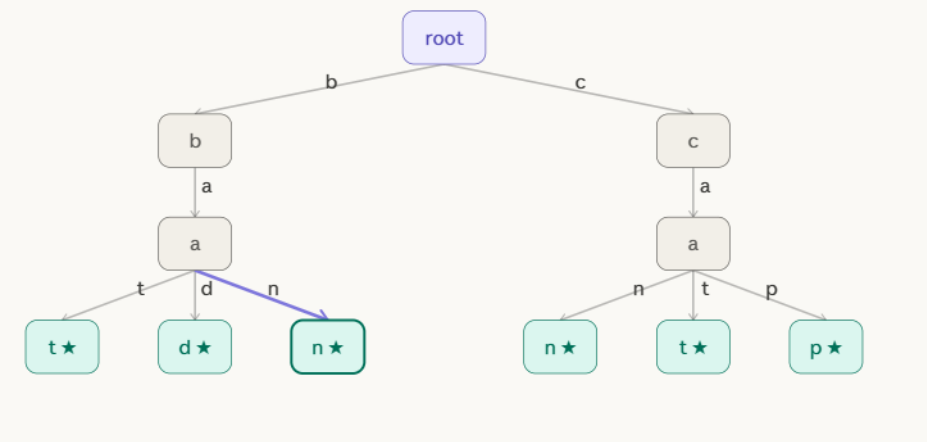
Step 4: **Insert "bat": "b" is new at root — create b→a→t★. This is a separate branch; root now has 2 children (c, b).**



Step 5: **Insert "bad": root→b→a exist. "d" is new — add d★ as sibling of t★ under [a]. Multi-way: [a] now has 2 children (t, d).**



Step 6: **Insert "ban": root→b→a exist. "n" new — add n★ under [a]. Final trie: 2 root-children, 6 end-of-word leaves. Height = 3.**



Search — O(m) where m = key length

Walk from root following the key's characters. The search **succeeds** only if both conditions hold: every character's edge exists, and the final node is marked ★. Reaching a dead end (missing edge) or reaching an unmarked internal node both count as failure.

Delete — O(m), may require cleanup

Locate the ★ node for the key and unmark it. If the now-unmarked node has no children, it is a useless internal node — remove it and walk upward, removing any ancestors that are also childless and non-★, stopping as soon as a node either has remaining children or is itself a ★.

3. Height

The height of a multi-way trie is defined as the number of edges on the longest root-to-leaf path, which equals the **length of the longest stored key**. This is the fundamental property that separates tries from comparison-based trees:

Structure	**Height depends on**
Binary search tree	Number of keys n
AVL / Red-black tree	$\log_2(n)$
Multi-way trie	Length of longest key L

For a set of n keys with maximum length L:

Height = L

Search time = O(L) (independent of n)

If all your keys are 5-character strings, search is always exactly 5 steps — whether you store 10 words or 10 million words. This is what makes tries so attractive for dictionary and autocomplete use cases.

The branching factor at every internal node is at most m (the alphabet size). For a 26-character alphabet, each node has up to 26 child pointers. This is the space cost: O(n × m) in the worst case, which can be large but is often reduced with compressed tries or hash maps per node.

4. Prefix Search

Prefix search is the operation where tries outperform every other data structure. To find all words beginning with a given prefix P:

1. Walk from root following each character of P — O(|P|) steps.
2. If the path ends at a node, perform a **subtree traversal** collecting every ★ node below.
3. Return all collected words.

The subtree traversal only visits the matching portion of the trie — all non-matching branches are skipped entirely. Time cost is O(|P| + k) where k is the number of matching words.

Compare this to a sorted array, which requires binary search plus a linear scan through matches. The trie goes directly to the matching region in O(|P|) and never looks at anything else.

5. Applications

Multi-way tries are the foundation of several widely-used systems:

Autocomplete / type-ahead search — Every search engine and keyboard suggestion system uses a trie variant. As you type each character, the system narrows to the subtree matching your prefix and returns the most frequent leaves.

Spell checking — Dictionaries are stored as tries. A word is valid if and only if a ★-terminated path exists. Suggestions come from prefix search plus edit-distance neighbors.

IP routing tables — Network routers use bitwise tries (binary tries) to match destination IP addresses against routing prefixes. The longest matching prefix determines the next hop. Hardware routers implement this in silicon (TCAM).

DNA sequence search — Genomic databases use tries over a 4-character alphabet {A, C, G, T} to index subsequences. Prefix search finds all sequences sharing a given motif.

File system path lookup — Directory trees are essentially multi-way tries over path components. Each node is a directory; edges are file/folder names.

Phone contact search — Android and iOS contact search builds a trie over contact names and phone number digits, enabling instant prefix matching as you dial.

Lexicographic sorting — An in-order traversal (visiting children alphabetically) of a trie yields all stored keys in sorted order — no separate sort step needed.

Summary

Operation	Time	Space
Insert (key length L)	O(L)	O(L × m) new nodes worst case
Search (key length L)	O(L)	O(1) extra
Delete (key length L)	O(L)	O(1) extra
Prefix search (prefix P, k results)	O(|P| + k)	O(1) extra
Build trie (n keys, total chars C)	O(C)	O(C × m)
Height	L (longest key)	—
Branching factor	m (alphabet size)	—

6 Suffix Tree

A **Suffix Tree** is the compressed trie of all suffixes of a string S$ (where $ is a unique terminal character). It is the most space-efficient and time-efficient suffix data structure:

- Every root-to-leaf path spells exactly one complete suffix of S
- Every internal node has at least 2 children
- Edge labels are **substrings** of S stored as (start, end) index pairs — not copies of characters — giving O(n) total space
- The key innovation over a suffix trie: chains of single-child nodes are compressed into single labeled edges

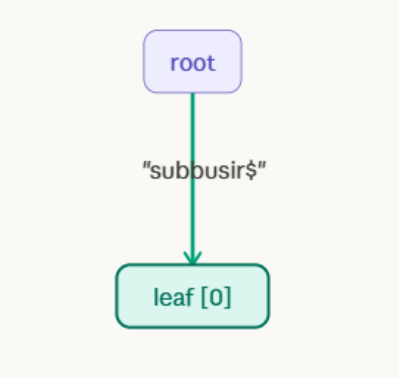
Example : **Create Suffix Tree of the following string S = "subbusir$".**

Solution :

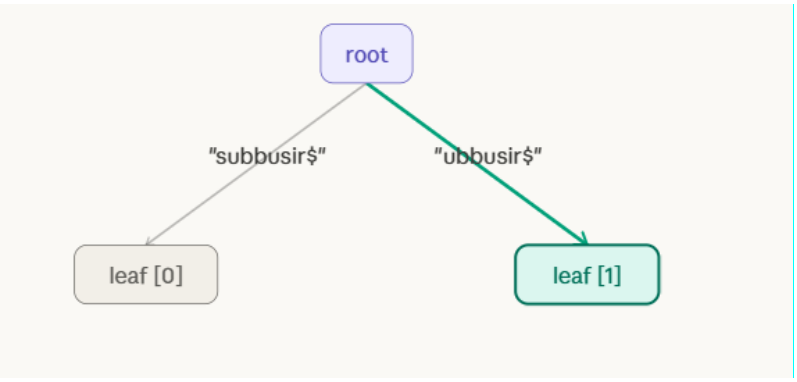
The string has 9 characters (including $). Every suffix starting at each position:

Index	Suffix
0	**subbusir$**
1	**ubbusir$**
2	**bbusir$**
3	**busir$**
4	**usir$**
5	**sir$**
6	**ir$**
7	**r$**
8	**$**

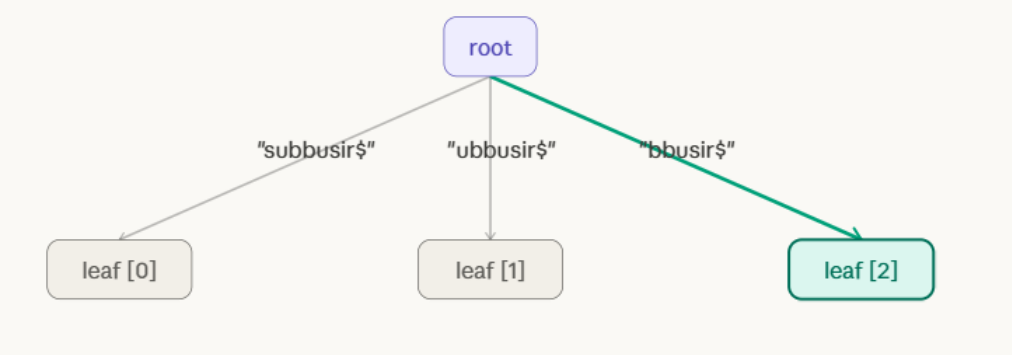
Step 1: Insert "subbusir$" (suffix 0): Tree is empty. No edge from root at all. → Case 1: Create new leaf directly. Root → leaf[0], edge label = full suffix "subbusir$".



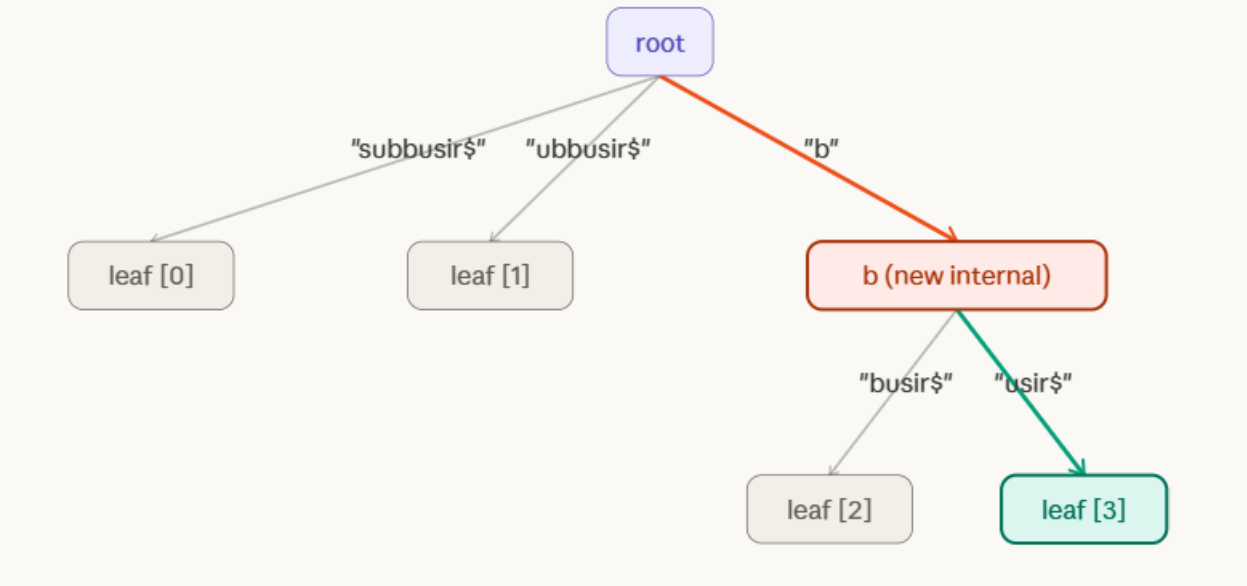
Step 2: Insert "ubbusir$" (suffix 1): At root, look for edge starting with "u". No "u" edge exists. → Case 1: Create new leaf. Root → leaf[1], edge = "ubbusir$".



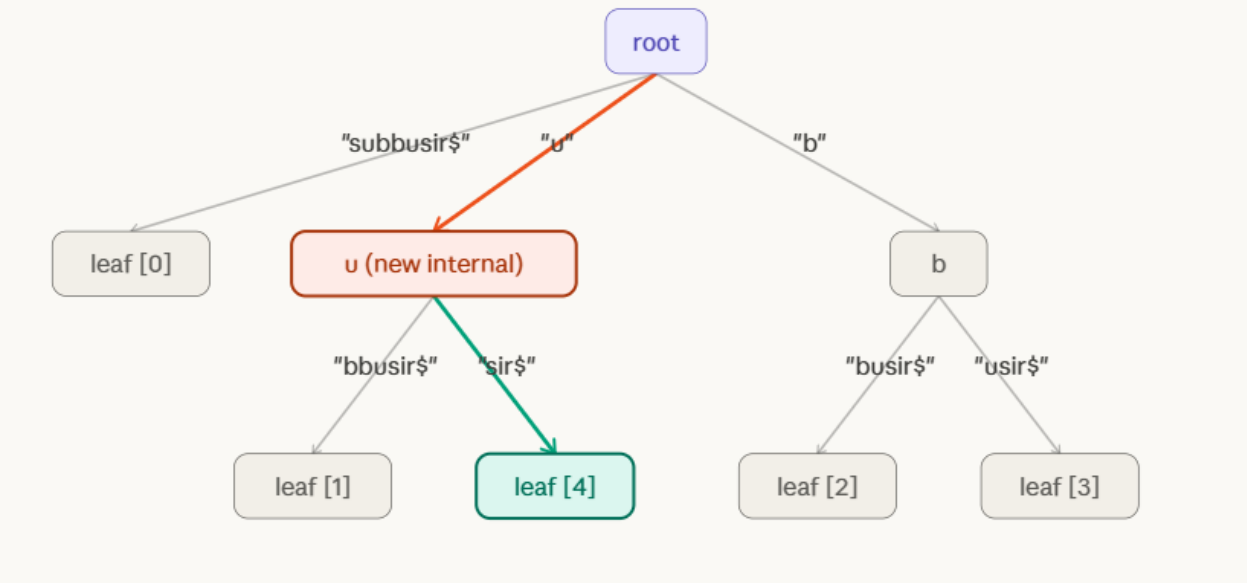
Step 3: Insert "bbusir$" (suffix 2): At root, look for edge starting with "b". No "b" edge exists. → Case 1: Create new leaf. Root → leaf[2], edge = "bbusir$".



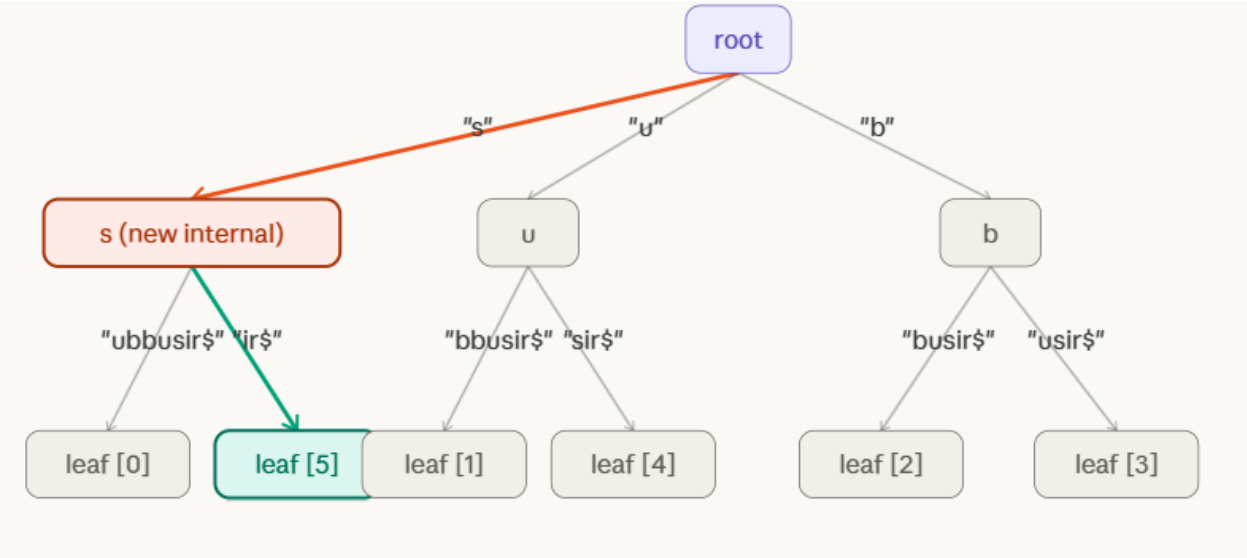
Step 4: Insert "busir$" (suffix 3): At root, edge "bbusir$" starts with "b" — partial match! Walk edge: b==b ✓, then u≠b ✗ (mismatch at pos 1). → Case 2: SPLIT. New internal node at "b". Old leaf[2] keeps "busir$". New leaf[3] gets "usir$".



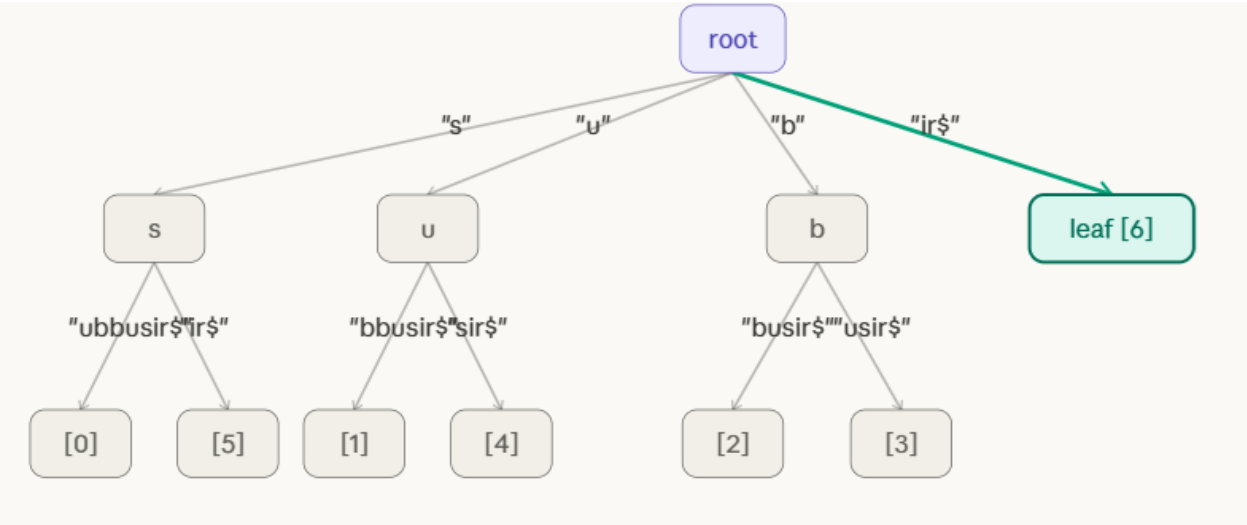
Step 5: Insert "usir$" (suffix 4): At root, edge "ubbusir$" starts with "u" — match! Walk: u==u ✓, then b≠s ✗ (mismatch at pos 1). → Case 2: SPLIT. New internal node "u". Old leaf[1] keeps "bbusir$". New leaf[4] gets "sir$".



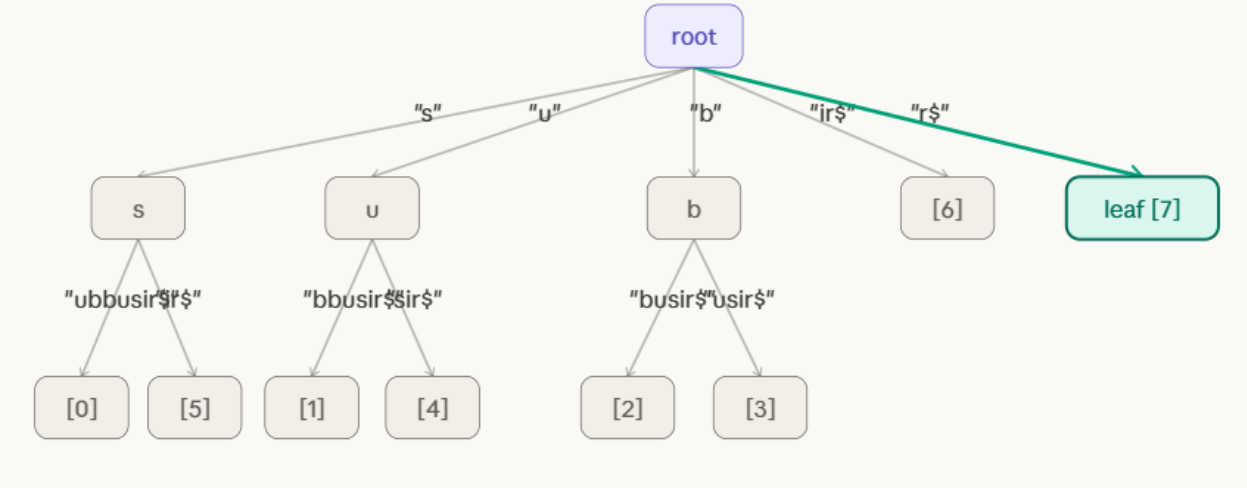
Step 6: Insert "sir$" (suffix 5): At root, edge "subbusir$" starts with "s" — match! Walk: s==s ✓, then u≠i ✗ (mismatch at pos 1). → Case 2: SPLIT. New internal node "s". Old leaf[0] keeps "ubbusir$". New leaf[5] gets "ir$".



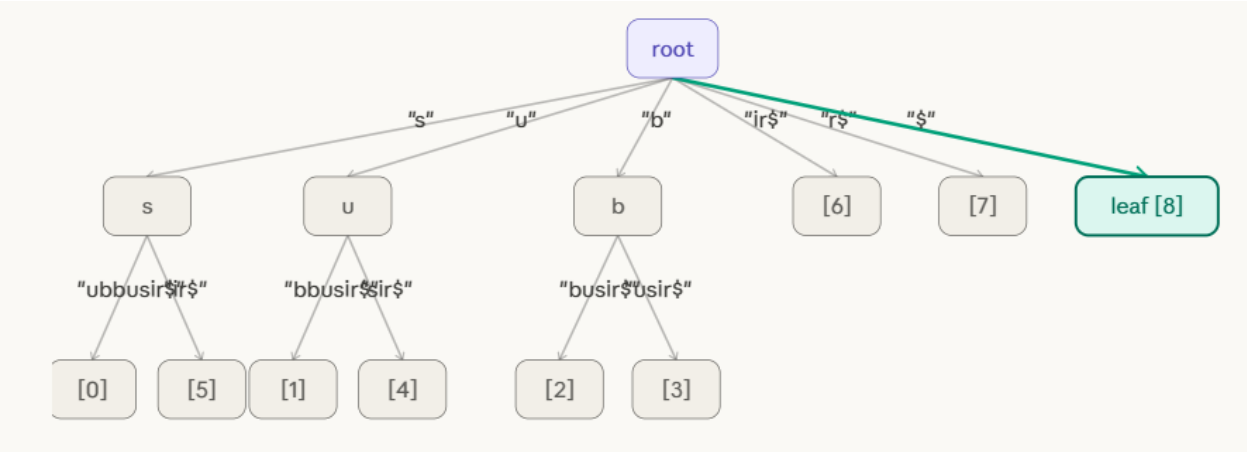
Step 7: Insert "ir$" (suffix 6): At root, no "i" edge exists. → Case 1: New leaf. Root → leaf[6], edge = "ir$".



Step 8: Insert "r$" (suffix 7): At root, no "r" edge exists. → Case 1: New leaf. Root → leaf[7], edge = "r$".



Step 9: Insert "$" (suffix 8): At root, no "$" edge exists. → Case 1: New leaf. Root → leaf[8], edge = "$". ALL 9 SUFFIXES INSERTED. Final suffix tree is complete!



Search

Phase 2 — Substring searches (Steps 10–18)

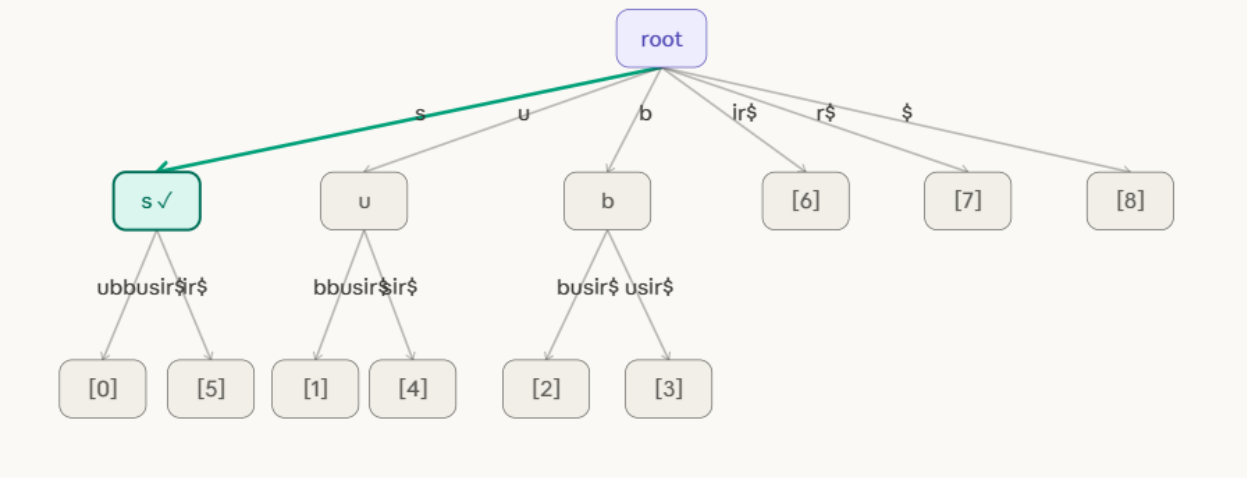
Search "sub" → FOUND at position 0 Root → edge s → node [s] → edge ubbusir$ → walk u, b along edge → all 3 chars matched → leaf [0] below = occurrence at index 0.

Search "bus" → FOUND at position 3 Root → edge b → node [b] → edge usir$ → walk u, s → all 3 chars matched → leaf [3] below = occurrence at index 3.

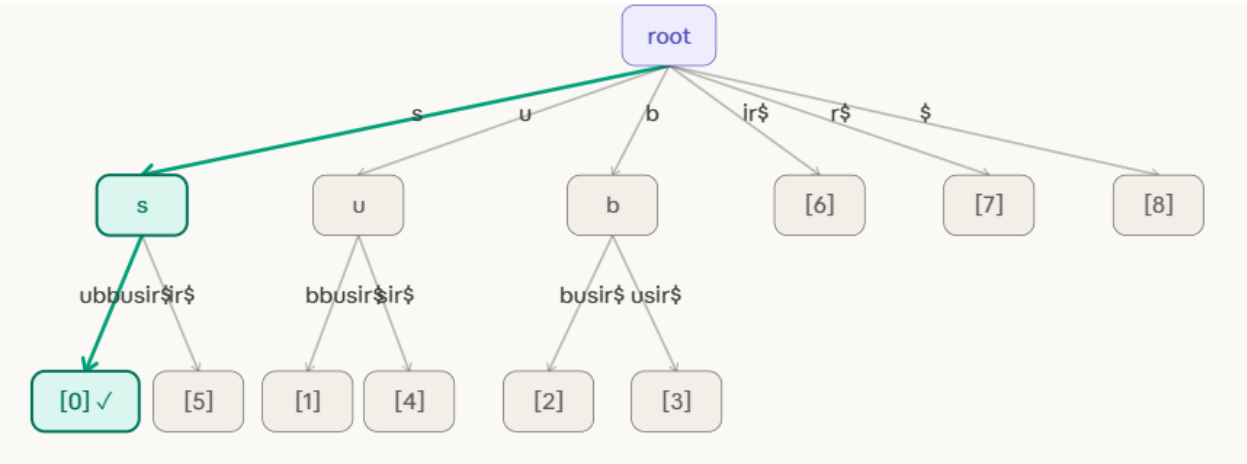
Search "sir" → FOUND at position 5 Root → edge s → node [s] → edge ir$ → walk i, r → all 3 chars matched → leaf [5] below = occurrence at index 5.

Search "xyz" → NOT FOUND Root has no edge starting with x. Fails in O(1) without touching any other part of the tree.

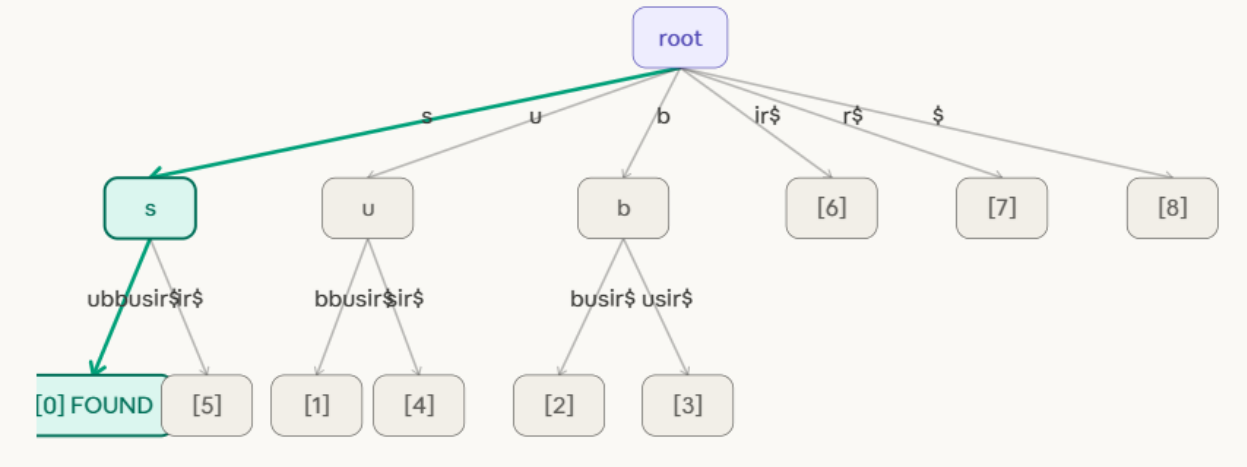
Step 1: Search "sub" — Step 1: at root, look for edge starting with "s". Found edge "s" → internal node. Consume "s". Move to [s] node. 1 char matched.



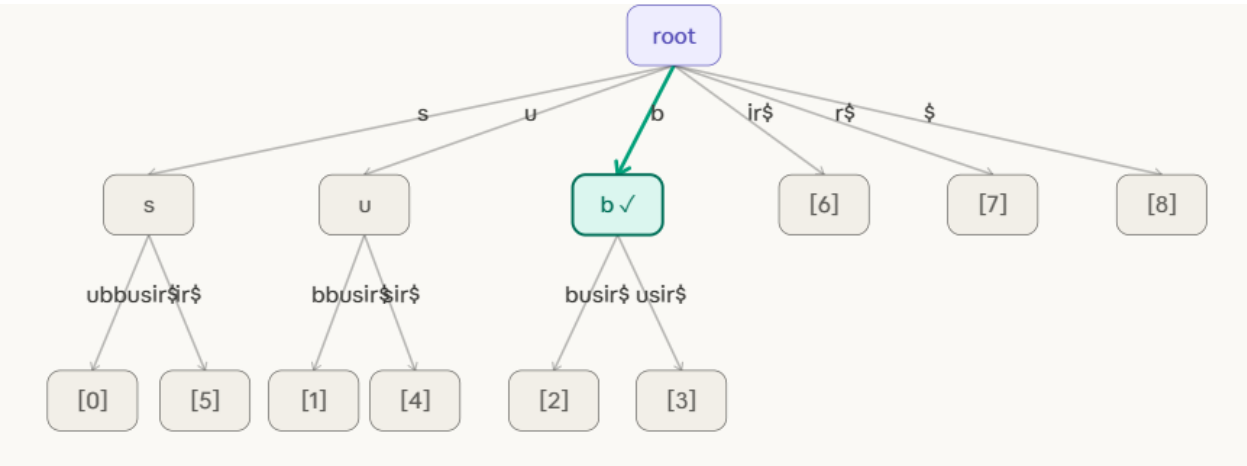
Search "sub" — Step 2: at [s] node, look for edge starting with "u". Found edge "ubbusir$" → leaf [0]. Consume "u" from edge. Now on edge "ubbusir$", 1 char into it. 2 chars matched.



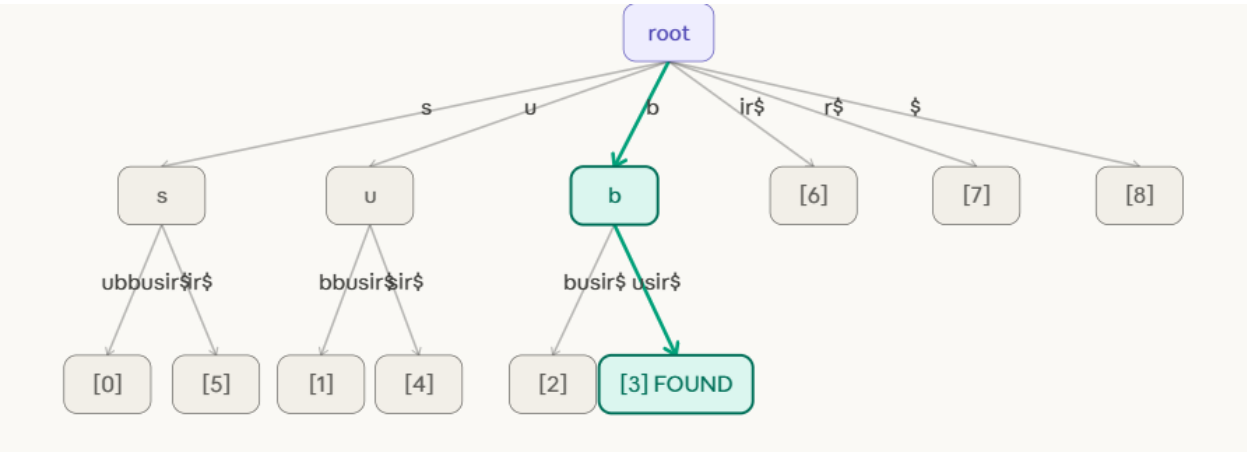
Search "sub" — Step 3: still on edge "ubbusir$". Next edge char = "b" = P[2]. Match! All 3 chars of "sub" consumed. Pattern FOUND mid-edge. Leaf below = [0]. "sub" occurs at position 0.



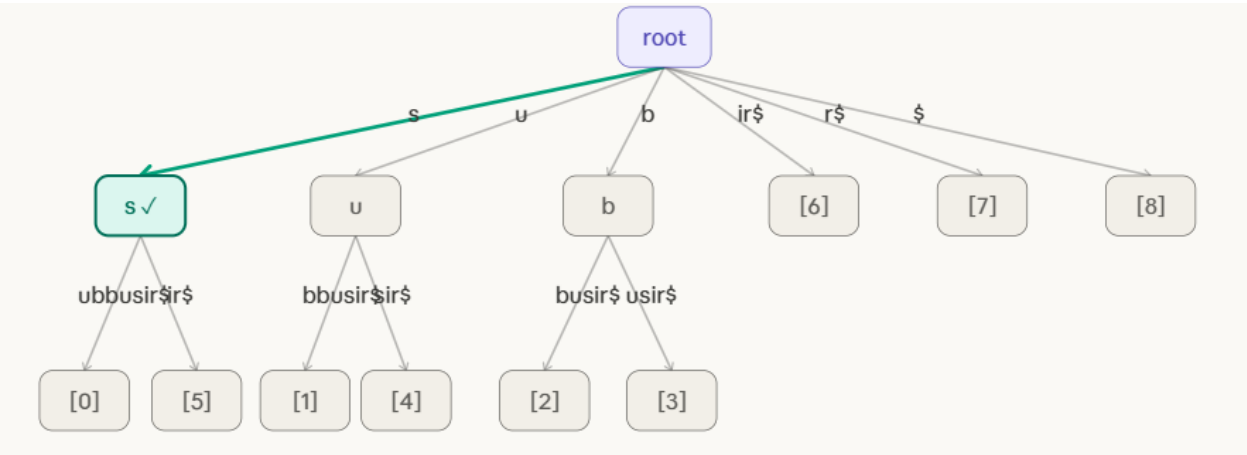
Search "bus" — Step 1: at root, look for edge starting with "b". Found edge "b" → internal node [b]. Consume "b". Move to [b] node. 1 char matched.



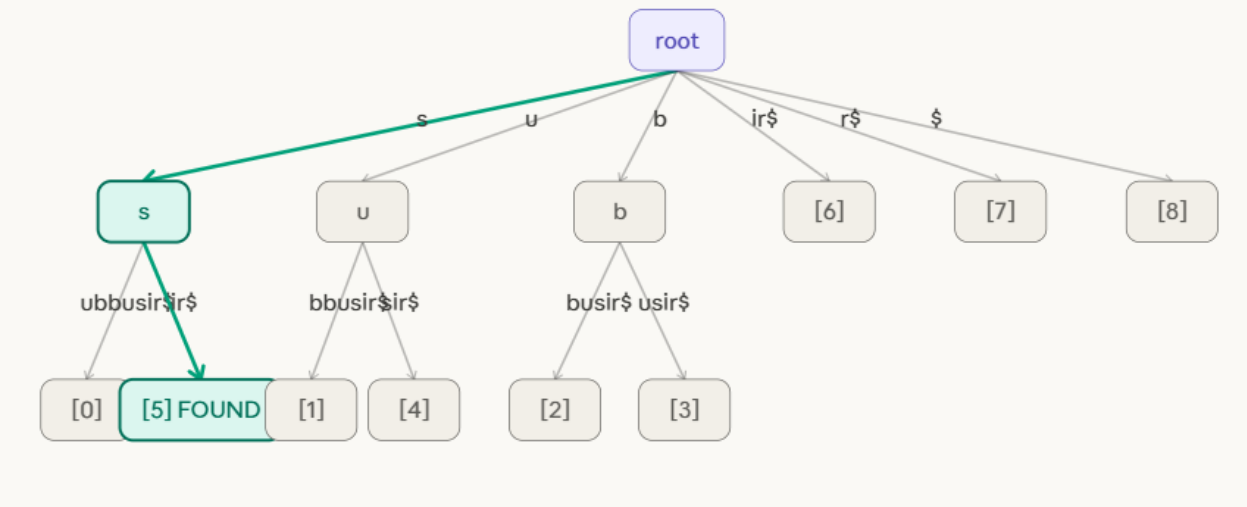
Search "bus" — Step 2 & 3: at [b] node, find edge "usir$" → leaf [3]. Walk along it: "u"=P[1]✓, "s"=P[2]✓. All 3 chars matched. Pattern FOUND. Leaf below = [3]. "bus" occurs at position 3.



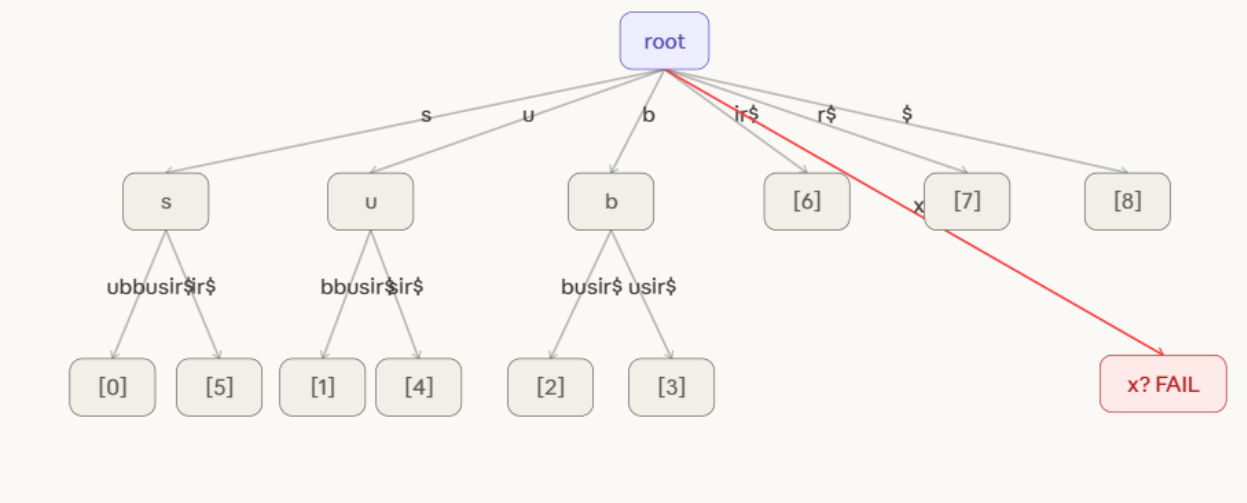
Search "sir" — Step 1: at root, find "s" edge → [s] node. Consume "s". Move to [s] internal node.



Search "sir" — Step 2 & 3: at [s] node, find edge "ir$" → leaf [5]. Walk: "i"=P[1]✓, "r"=P[2]✓. All 3 chars matched mid-edge. Pattern FOUND. Leaf = [5]. "sir" occurs at position 5.



Search "xyz" — Step 1: at root, look for edge starting with "x". No such edge exists in the tree. Search FAILS immediately in O(1). "xyz" is not a substring of "subbusir$".



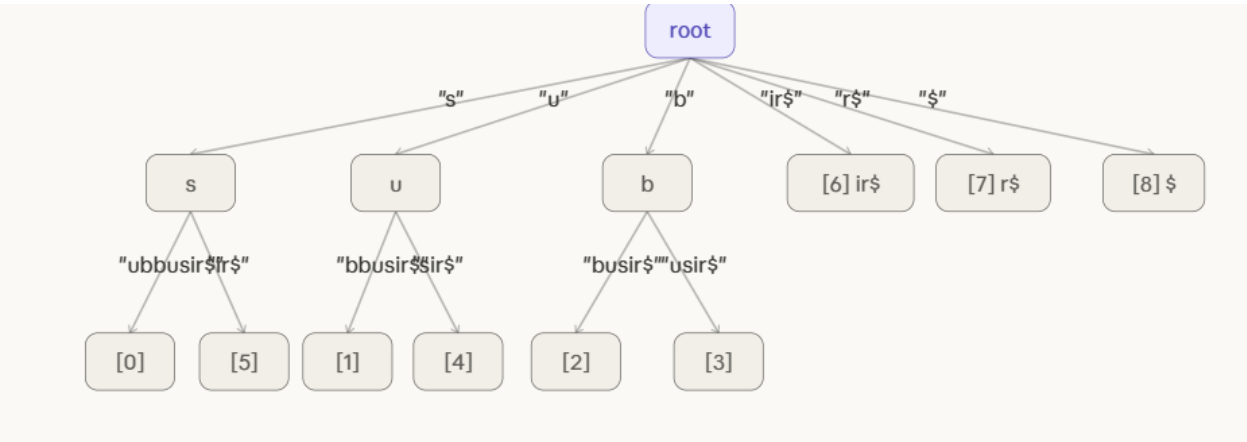
Deletion Algorithm

DELETE_SUFFIX(root, S, i):

Step 1 — Find the leaf:
Walk from root following suffix S[i..end] character by character.
If not found → error, suffix not in tree.

Step 2 — Remove the leaf:
Detach the leaf node and its incoming edge from its parent.

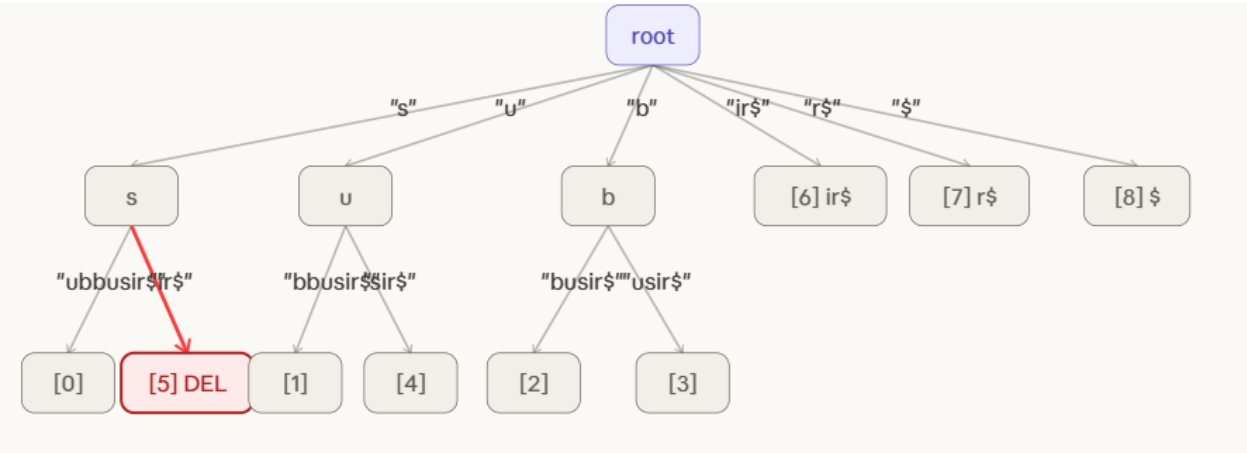
Step 3 — Check parent for merge:
parent = the node just above the removed leaf.
If parent now has exactly 1 child remaining
AND parent is NOT the root
AND parent is NOT an end-of-word marker:
→ MERGE:
new_label = edge(grandparent → parent) + edge(parent → sole_child)
Remove parent node.
Connect grandparent directly to sole_child with new_label.
Else:
→ STOP. No merge needed.



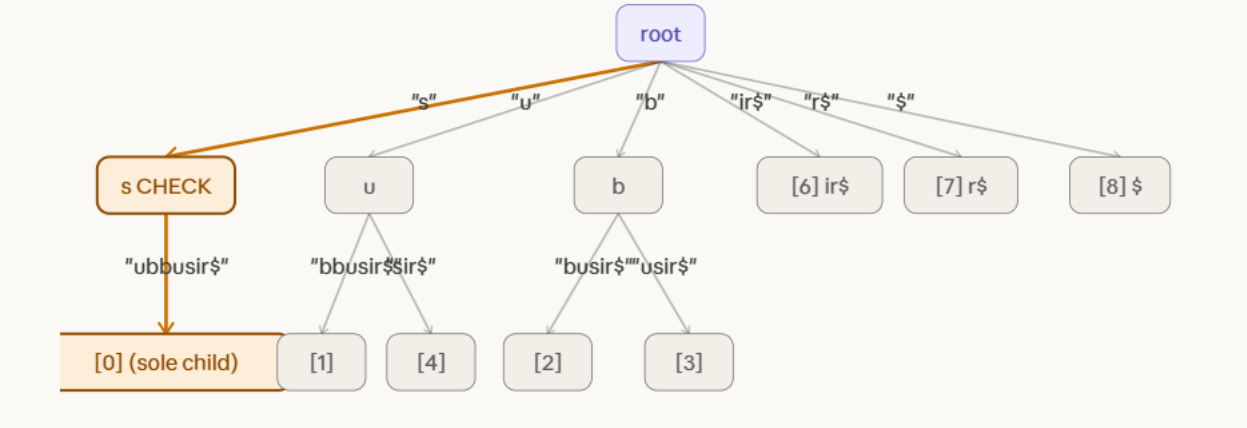
Delete sir$

DELETE 1 — Suffix "sir$" (leaf [5]).

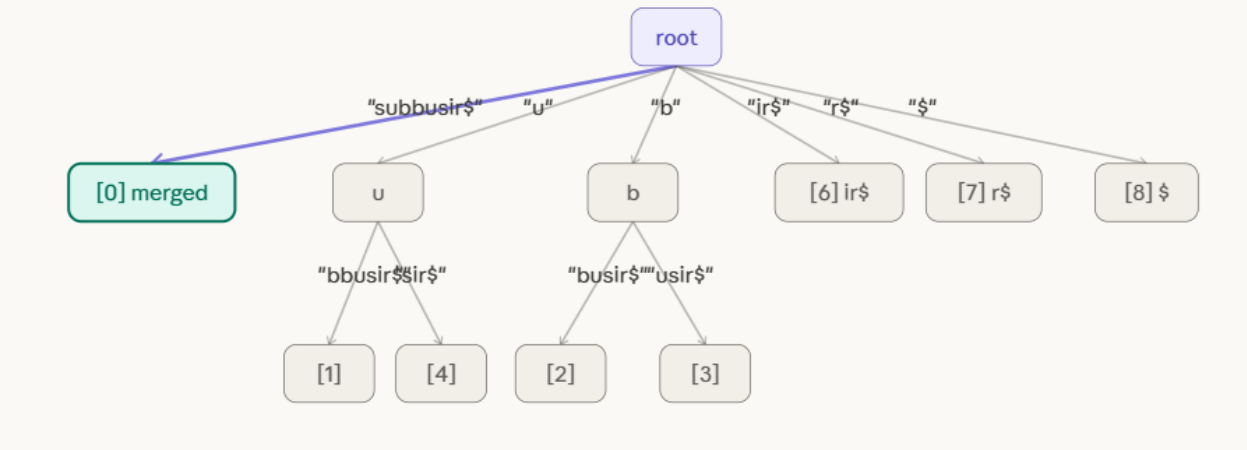
Step 1: Locate leaf [5]. Path: root → edge "s" → node [s] → edge "ir$" → leaf [5]. Found! Leaf [5] is marked for removal (shown in red).



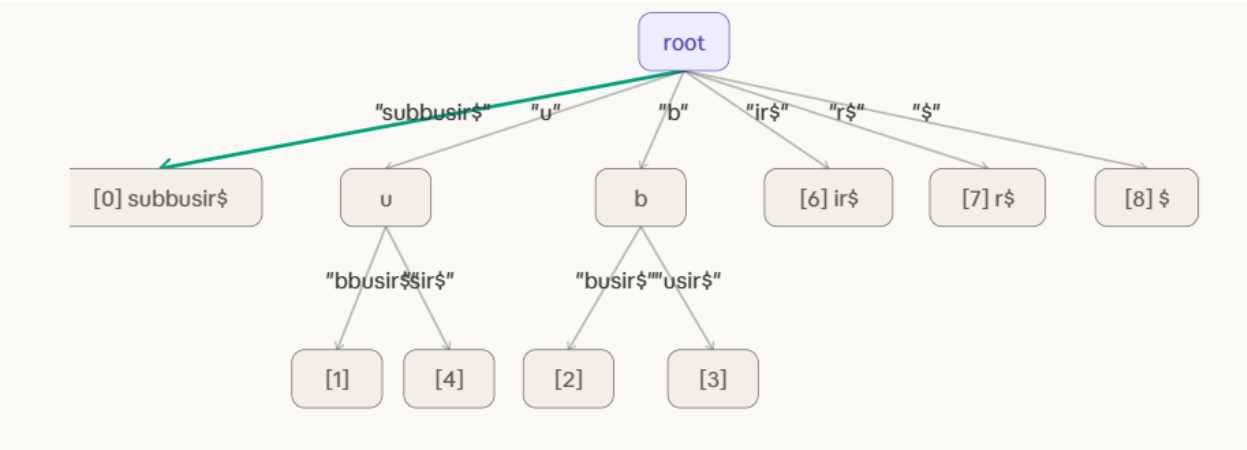
Step 2: Remove leaf [5] and its incoming edge "ir$". Check parent [s]: it now has only 1 child remaining (leaf [0] via "ubbusir$"). [s] is NOT end-of-word. → MERGE condition triggered!



Step 3: MERGE. Node [s] had edge "s" from root + edge "ubbusir$" to leaf[0]. Concatenate: "s" + "ubbusir$" = "subbusir$". Remove [s] node. Root now connects directly to leaf[0] with edge "subbusir$". [5] fully deleted.



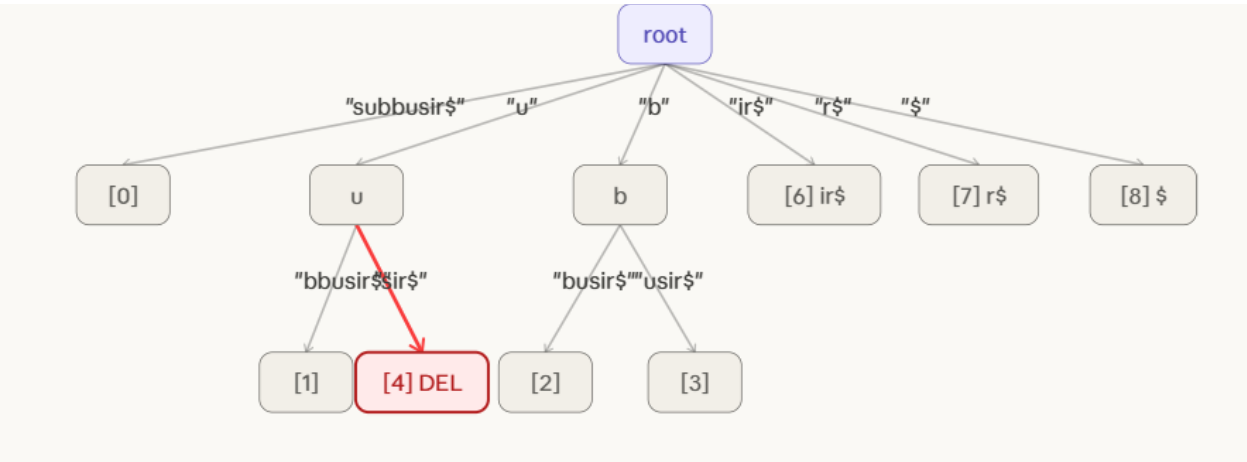
After Delete 1 — "sir$" successfully removed. Internal node [s] merged away. Tree now has 8 leaves ([0]–[4], [6]–[8]). Root has 6 direct edges. Structure is still a valid compressed suffix tree.



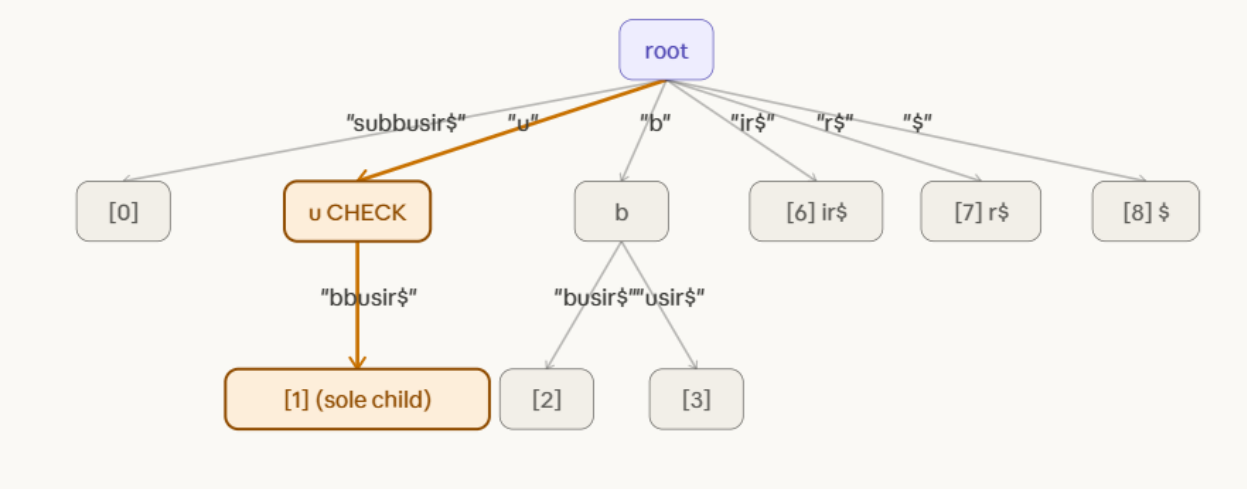
DELETE : usir

Suffix "usir$" (leaf [4]).

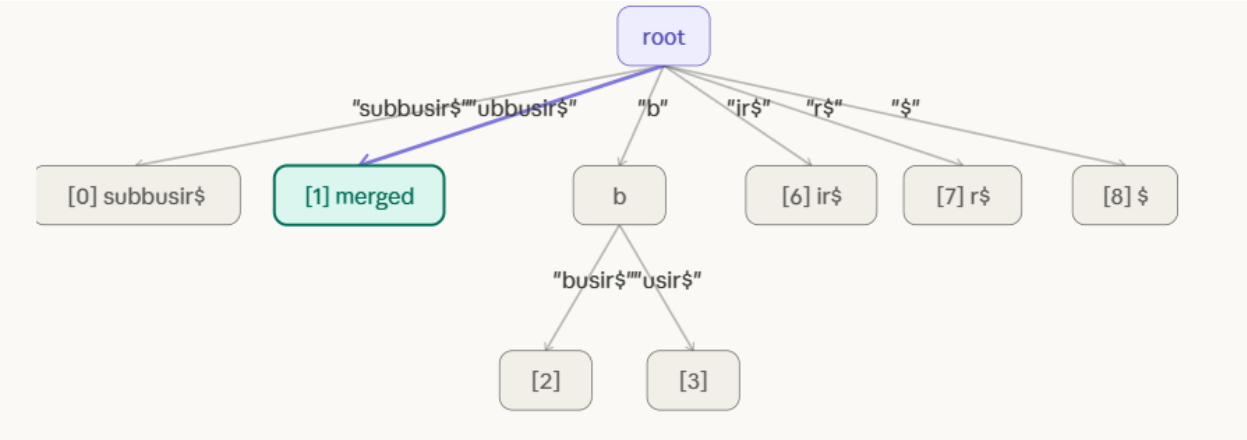
Step 1: Locate leaf [4]. Path: root → edge "u" → node [u] → edge "sir$" → leaf [4]. Found! Marked for deletion (red).



Step 2: Remove leaf [4] and its edge "sir$". Check parent [u]: it now has only 1 child remaining (leaf [1] via "bbusir$"). [u] is NOT end-of-word. → MERGE condition triggered again!



Step 3: MERGE. Node [u] had edge "u" from root + edge "bbusir$" to leaf[1]. Concatenate: "u" + "bbusir$" = "ubbusir$". Remove [u] node. Root connects directly to leaf[1] with "ubbusir$". [4] fully deleted.



FINAL STATE

Both deletions complete. Suffixes "sir$" (pos 5) and "usir$" (pos 4) removed. Tree has 7 leaves ([0],[1],[2],[3],[6],[7],[8]). Both internal nodes [s] and [u] were merged away. Only [b] remains as an internal node. Tree is still valid and compressed.